University of Applied Sciences Ulm
Faculty of Electrical Engineering and Information Technology



Master Thesis

submitted for the degree of

Master of Engineering

In System Engineering and Management

A Concept and Implementation of a Proactive Asset Administration Shell with Multi-Agent System Integration using the Belief-Desire-Intention approach

by

Ahmed Ibrahim Almohamed, BEng
Matriculation Nr.: 6005151

Submission Date:	February 4, 2026
Supervisors:	Prof. Dr. Sven Völker
	Prof. Dr.-Ing. Hartwig Baumgärtel

Abstract

As industries move toward Industry 4.0 (I4.0), intelligent autonomous systems capable of managing physical assets are increasingly essential. The Asset Administration Shell (AAS) has emerged as a key technology for standardizing the representation and management of digital twins. Among AAS types, the Type 3 Asset Administration Shell (Type3AAS) is distinguished by its capacity to encapsulate not only an asset's data and functions but also its proactivity. This thesis proposes an approach to implementing Type3AAS using Multi Agent system (MAS) based on the Belief-Desire-Intention (BDI) architecture. A systems engineering approach was taken to define the requirements, concept, and implementation of the proposed solution. The work establishes foundational submodels for the AAS and demonstrates their application through a production resource use case involving drill machines. The proof of concept is validated by the ability of the implemented Type3AAS to autonomously manage and optimize drill machine operations in a simulated production environment. This thesis provides groundwork for further research and development in this area.

List of Acronyms

AAS Asset Administration Shell

Type3AAS Type 3 Asset Administration Shell

BDI Belief-Desire-Intention

IIoT Industrial Internet of Things

I4.0 Industry 4.0

RAMI4.0 Reference Architectural Model Industrie 4.0

MAS Multi Agent system

DT Digital Twin

XML eXtensible Markup Language

JSON JavaScript Object Notation

RDF Resource Description Framework

API Application Programming Interface

CRUD Create, Read, Update, Delete

SYNC Synchronous

ASYNC Asynchronous

LLC Low-Level Control

HLC High-Level Control

FAAAST Fraunhofer Advanced AAS Tools for Simplified Interoperability

PLC Programmable Logic Controller

ACL Agent Communication Language

FIPA Foundation for Intelligent Physical Agents

OPC UA Open Platform Communications Unified Architecture

MQTT Message Queuing Telemetry Transport

HTTP Hypertext Transfer Protocol

REST Representational State Transfer

XMPP Extensible Messaging and Presence Protocol

JADE JAVA Agent Development Framework

JADEX JADE eXtension

SPADE Smart Python Agent Development Environment

CPS Cyber Physical Systems

IDTA Industrial Digital Twin Association

ACL Agent Communication Language

List of Figures

1.1 Reference Architectural Model Industrie 4.0 (RAMI 4.0) [1] 16

2.1 The Industry 4.0 Component: Asset and its Administration Shell [2] . 20
2.2 Asset Administration Shell Meta Model [3] 20
2.3 Submodel Meta Model [3] 22
2.4 Submodel Element Meta Model [3] 23
2.5 Data Types Meta Model [3] 27
2.6 Types of Asset Administration Shells [4] 27
2.7 Concept of Type3 AAS [5] 35
2.8 Industrial Agent Layers [6] 35
2.9 BDI Reasoning Cycle [7] 46
2.10 Capability-Skill-Service Information Model [8] 48

3.1 Use Case Diagram for the SmartDrillSystem 59

4.1 Two-Level System Architecture 68
4.2 SmartDrillSystem Architecture Overview 68
4.3 Component Interaction Patterns 75
4.4 Resource Agent Component Structure 75
4.5 Resource Agent Internal Architecture 78
4.6 Operational State Submodel Structure 78
4.7 Knowledge Submodel Structure 80
4.8 Capability Submodel Structure 82
4.9 Interaction and Scheduling Submodel Structure 82
4.10 BDI Reasoning Cycle in Type 3 AAS 88

4.11 Resource Agent Repository Class Hierarchy 91
4.12 Resource Agent Core Class Architecture 94
4.13 Resource Agent State Machine 97
4.14 Resource Simulator Integration Architecture 98
4.15 Production Agent Component Structure 103
4.16 Production Agent Internal Architecture 103
4.17 Process Plan Submodel Structure 104
4.18 Execution Tracking Submodel Structure 106
4.19 Interface Endpoints Submodel Structure 106
4.20 Execution Core Class Architecture 107
4.21 Negotiation Core Class Architecture 110
4.22 Production Agent - Resource Agent - System Interaction State Machine 114
4.23 Current Implementation: Linear Process Plan 119
4.24 Precedence Graph with Parallel Branches (Not Implemented) 119

5.1 System Infrastructure Overview 126

6.1 Test Process Plan: Move → Drill 137

Listings

4.1 Belief Loading from AAS Submodels 83
4.2 Initial Goal in AgentSpeak 84
4.3 Desire: Respond to CFP 86
4.4 Desire: Execute Skills 86
4.5 Custom Action Registration 86
4.6 Context-Dependent Intention Selection 87
8.1 Docker Compose Network Configuration 159
8.2 Operation Invocation Delegation Qualifier 160
8.3 Operation Invoker Endpoint Structure 160
8.4 Prosody Installation 161
8.5 Prosody Host Configuration 161
8.6 PubSub Component Configuration 161
8.7 Admin Configuration 161
8.8 Certificate Generation 162
8.9 XMPP User Creation 162
8.10 PubSub Topic Dispatch Pattern 162
8.11 Docker Compose Configuration (excerpt) 163

List of Tables

2.1 Submodel Element Types and Definitions [3] 24
2.2 Specification of *type2 Application Programming Interface (API)* [3] . . 26
2.3 Gap Analysis: Type 2 Capabilities vs. Type 3 Requirements 42
2.4 Comparison of Asset Administration Shell frameworks 46
2.5 Comparison of Multi-Agent System and BDI frameworks 46
2.6 Key Relationships in the CSS Information Model 51
2.7 Summary of AAS Type 3 implementations based on MAS concepts . 52

3.1 Traceability Matrix: Contributions to Requirements 64

4.1 System Components Mapped to RAMI4.0 Hierarchy Levels 71
4.2 System Elements Mapped to RAMI4.0 Life Cycle Phases 72
4.3 System Implementation Mapped to RAMI4.0 Layers 72
4.4 BDI to AAS-Agent Mapping 85
4.5 AAS Submodel Elements to Agent Beliefs 85
4.6 Resource Simulator Configuration 100
4.7 Production Agent Internal Communication Topics 116
4.8 Production-Resource Communication Topics 116
4.9 Generic vs. Scenario-Specific Submodel Elements 117
4.10 User Request Message Structure 122

5.1 Implementation Technology Stack 127
5.2 Deployment Components 130

6.1 Test Environment Configuration 132
6.2 Test Scenario Overview 134

6.3 Negotiation Test Results (TS-1) . . . 134
6.4 ResourceAAS Submodel State Before Execution . . . 135
6.5 ProductionAAS Submodel State Before Execution . . . 136
6.6 Detailed AAS Updates During Execution (TS-2) . . . 136
6.7 AAS Submodel State After Execution . . . 136
6.8 Process Plan Execution Trace (TS-3) . . . 137
6.9 Token Position During Execution . . . 138
6.10 Simulator Operation Logs . . . 138
6.11 Test Limitations and Future Work . . . 139
6.12 Requirements Coverage Summary . . . 140
6.13 Submodel Support for Proactive Agent Behavior . . . 141
6.14 Thesis Contribution Validation Summary . . . 143
6.15 AAS Type Comparison . . . 144

8.1 Use Case UC1: System Startup . . . 149
8.2 Use Case UC3: Drilling Order . . . 150
8.3 Use Case UC5: System Shutdown . . . 152
8.4 System Specification Requirements . . . 154
8.5 ResourceAAS Requirements . . . 155
8.6 ProductionAAS Requirements . . . 156
8.7 MAS/BDI Requirements . . . 158
8.8 Non-Functional Requirements . . . 158

Contents

1 Introduction **13**

2 Fundamentals and State of the Art **17**

2.1 Asset Administration Shells(AAS) 17

2.1.1 Basic concept of Asset Administration Shell 17

2.1.2 Submodels and data types 21

2.1.3 Types of Asset Administration Shells 26

2.1.4 Asset Administration Shell Type3 28

2.1.5 Summary 29

2.2 Multi-Agent Systems (MAS) 30

2.2.1 Agents and Their Properties 30

2.2.2 Multi-Agent Systems 31

2.2.3 Agent Architectures 31

2.2.4 Agent Communication 32

2.2.5 Agent Coordination and Cooperation 33

2.2.6 Industrial Agents 34

2.2.7 Summary 34

2.3 BDI (Belief-Desire-Intention) Approach 37

2.3.1 Foundations of the Belief-Desire-Intention Model 37

2.3.2 The Belief-Desire-Intention Reasoning Cycle 38

2.3.3 Belief-Desire-Intention for Industrial and Cyber-Physical Systems 39

2.3.4 Summary 40

2.4 Software Frameworks and Platforms 40

2.4.1 Asset Administration Shell Frameworks . . . 40
2.4.2 Multi-Agent System Frameworks . . . 43
2.4.3 Framework selection considerations . . . 45
2.4.4 Summary and Framework Selection . . . 45
2.5 Capability-Skill-Service Information Model . . . 48
2.5.1 Model Overview . . . 48
2.5.2 Product-Process-Resource (PPR) . . . 49
2.5.3 Capability Layer . . . 49
2.5.4 Skill Layer . . . 50
2.5.5 Service Layer . . . 50
2.5.6 Key Relationships . . . 51
2.5.7 Relevance for Type 3 AAS . . . 51
2.6 Integration of Asset Administration Shell with Multi-Agent Systems . 52
2.6.1 Critical Analysis of Existing Approaches . . . 54
2.6.2 Addressed Research Gap . . . 56

3 Use Cases and System Requirements 57
3.1 From Research Questions to Use Cases . . . 57
3.2 System Components . . . 58
3.3 Use Cases . . . 58
3.3.1 UC1: System Startup . . . 59
3.3.2 UC3: Drilling Order . . . 59
3.3.3 UC5: System Shutdown . . . 60
3.3.4 Included Use Cases . . . 60
3.4 System Requirements . . . 61
3.4.1 Functional Requirements . . . 61
3.4.2 Non-Functional Requirements (NFR_1 – NFR_5) . . . 62
3.4.3 Considerations for Production Deployment . . . 63
3.5 Requirements Traceability . . . 64
3.6 Summary . . . 64

4 Concept **66**

4.1 System Overview . . . 66

4.1.1 Architectural Layers . . . 66

4.1.2 System Components . . . 67

4.1.3 Component Interactions . . . 69

4.1.4 Bidirectional AAS Integration . . . 70

4.1.5 Mapping to RAMI4.0 . . . 71

4.2 Resource Agent Concept . . . 73

4.2.1 Resource Agent Overview . . . 74

4.2.2 Resource Agent Submodels . . . 76

4.2.3 BDI-Submodel Integration . . . 81

4.2.4 Resource Agent Repositories . . . 90

4.2.5 Resource Agent Core . . . 93

4.2.6 Resource Simulators . . . 96

4.3 Production Agent Concept . . . 101

4.3.1 Production Agent Overview . . . 101

4.3.2 Production Agent Submodels . . . 102

4.3.3 Production Agent Execution Core . . . 105

4.3.4 Production Agent Negotiation Core . . . 109

4.4 System Interaction . . . 113

4.4.1 Execution Core Flow . . . 115

4.4.2 Negotiation Core Flow . . . 115

4.4.3 Inter-Core Communication . . . 116

4.4.4 Resource Agent Interaction . . . 116

4.5 Submodel Design Considerations . . . 117

4.5.1 Generic vs. Scenario-Specific Elements . . . 117

4.5.2 Capability Matching Mechanism . . . 118

4.5.3 Process Plan Structure and Limitations . . . 119

4.5.4 Token Position and Execution State . . . 120

4.5.5 Knowledge Submodel Simplifications . . . 121

4.5.6 Assembly Processes and Bill of Materials . . . 122

4.5.7 User Request Definition . . . 122
4.5.8 Concrete Example: Drill Operation . . . 123

5 Implementation **125**
5.1 Infrastructure Overview . . . 125
5.2 Asset Administration Shell Server . . . 127
5.2.1 Operation Invocation Delegation . . . 127
5.3 Agent Communication Infrastructure . . . 128
5.4 Agent Implementation Challenges . . . 128
5.4.1 BDI vs. Finite State Machine . . . 128
5.4.2 SPADE Belief Storage Limitation . . . 129
5.4.3 PubSub Subscriber Crosstalk . . . 129
5.5 Deployment Architecture . . . 130
5.6 Summary . . . 130

6 Evaluation **132**
6.1 Test Environment . . . 132
6.2 Report Generation System . . . 132
6.2.1 Production Reports . . . 133
6.2.2 Simulator Reports . . . 133
6.3 Functional Test Results . . . 133
6.3.1 Test Scenarios . . . 133
6.3.2 Test Scenario TS-1: Multi-Agent Negotiation . . . 134
6.3.3 Test Scenario TS-2: AAS State Documentation . . . 135
6.3.4 Test Scenario TS-3: Multi-Node Process Plan . . . 137
6.3.5 Simulator Verification . . . 138
6.3.6 Test Limitations . . . 138
6.3.7 Summary . . . 139
6.4 Requirements Validation . . . 139
6.5 Validation of Thesis Contributions . . . 140
6.5.1 Contribution 1: Bidirectional AAS Integration . . . 141
6.5.2 Contribution 2: Foundational Submodels for Proactive Behavior 141

6.5.3 Contribution 3: Proof-of-Concept with Production Resources . 142
6.5.4 Summary . . . 142
6.6 Discussion . . . 143
6.6.1 Achievements . . . 143
6.6.2 Limitations . . . 143
6.6.3 Comparison with Type 1 and Type 2 AAS . . . 144
6.6.4 Validation of Thesis Contributions . . . 144
6.7 Summary . . . 145

7 Outlook **146**
7.1 Physical Hardware Integration . . . 146
7.2 Enhanced Negotiation Mechanisms . . . 146
7.3 Scalability and Performance . . . 147
7.4 Advanced BDI Capabilities . . . 147
7.5 Standardization Alignment . . . 147
7.6 Security and Access Control . . . 148
7.7 Visualization and Monitoring . . . 148

8 Appendix **149**
8.1 Detailed Use Cases and Requirements . . . 149
8.1.1 Use Case Specifications . . . 149
8.1.2 System Specifications . . . 154
8.1.3 ResourceAAS Specifications . . . 155
8.1.4 ProductionAAS Specifications . . . 156
8.1.5 MAS/BDI Specifications . . . 157
8.1.6 Non-Functional Requirements . . . 158
8.2 Implementation Details . . . 159
8.2.1 Operation Invocation Delegation . . . 159
8.2.2 XMPP Server Configuration . . . 161
8.2.3 PubSub Dispatch Pattern . . . 162
8.2.4 Deployment Configuration . . . 163

Chapter 1

Introduction

The manufacturing industry is undergoing a fundamental transformation driven by the fourth industrial revolution, commonly referred to as Industry 4.0. This transformation is characterized by Cyber Physical Systems (CPS), intelligent automation, and the convergence of technologies including mechatronic systems, controllers, on-board computation, big data, machine learning, and the Industrial Internet of Things (IIoT) [9].

Industry 4.0 holds the promise of increased flexibility in manufacturing, along with mass customization, better quality, and improved productivity [10]. Its vision encompasses ecosystems of smart factories with intelligent and autonomous shop-floor entities [11]. To provide a structured approach for implementing this vision, the Platform Industrie 4.0 working group developed the Reference Architectural Model Industrie 4.0 (Reference Architectural Model Industrie 4.0 (RAMI4.0)) [12].
RAMI4.0 is a three-dimensional map showing how to approach the deployment of Industry 4.0 in a structured manner, ensuring that all participants involved share a common perspective and develop a common understanding [1]. The model encompasses three axes: the product life cycle and value stream (distinguishing between types and instances), the hierarchy levels representing different participants within the network, and six interoperability layers covering everything from business processes to communication standards to physical components [1].

Central to this architectural model is the concept of the Digital Twin (DT). A digital twin can be defined as an evolving digital profile of the historical and current

behavior of a physical object or process that helps optimize business performance [13]. Digital twins allow industry to detect physical issues sooner, predict outcomes more accurately, and build better products [13]. As the adoption of Industry 4.0 advances, digital twins prove invaluable for testing and simulating new parameters and design variants [14].

To fully realize the potential of digital twins in industrial settings, standardization is essential [15]. The AAS has emerged as a key enabler in this regard. The AAS is the logical representation of an asset that enables it to communicate in an Industry 4.0-compliant manner [15]. Together with the asset, the AAS forms the Industry 4.0 Component (I4.0 Component) [15].

The AAS is the standard digital representation of an industrial asset that plays a pivotal role in enabling interoperable communication among I4.0 components across the value chain [15]. The interoperability between I4.0 components depends crucially on the contents of the administration shell, and so it is the main task of the AAS to register and make available the data and functions of all relevant assets in a standardized way throughout their life cycle [16].

However, as manufacturing systems become increasingly complex, autonomous production becomes essential. As [17] note, the increasing complexity caused by global competition, product variety, and individualization demands production systems that are self-organized and capable of autonomous decision-making [17]. In such environments, it becomes impossible to anticipate and automate all relevant constellations through centralized control [18].

While the AAS provides a robust framework for standardizing asset representation and communication, current implementations predominantly focus on passive data exchange and reactive behavior [19]. Most existing solutions are limited to passive (providing static metadata) or reactive (responding to external events) functionalities, lacking the active behaviors required to support fully autonomous interactions and intelligent decision-making [20]. The Type3AAS, which is characterized by its proactive capabilities, remains largely unexplored in practice, still in its early stages and lacking formal specifications and comprehensive implementation guidelines [19]. Realizing truly autonomous I4.0 components requires not only stan-

dardized data models but also decision-making capabilities that allow assets to act proactively based on their goals and beliefs. This gap motivates the integration of agent-based technologies with the AAS concept. MAS offer a promising paradigm for implementing autonomous behavior in distributed systems. In particular, the BDI architecture provides a well-established model for rational agents that reason about their beliefs, desires, and intentions to select appropriate actions [21]. By combining the standardized structure of the AAS with the proactive reasoning capabilities of BDI agents, it becomes possible to realize Type3AAS that can autonomously manage and optimize asset operations.

This thesis proposes an approach to implementing Type3AAS using MAS based on the BDI architecture. The main contributions are:

- A conceptual integration of AAS with BDI-based agents
- Foundational submodels for enabling proactive behavior in AAS
- A proof-of-concept implementation demonstrated through a production resource use case involving drill machines

The remainder of this thesis is structured as follows: Chapter 2 provides a literature review of AAS, MAS, and the BDI architecture. Chapter **??** examines the current state of the art, including existing frameworks and integration approaches. Chapter 3 defines the system requirements and use case. Chapter 4 presents the conceptual design of the proposed solution. Chapter **??** details the implementation. Chapter **??** evaluates the proof of concept. Finally, Chapter 7 concludes with an outlook on future research directions.

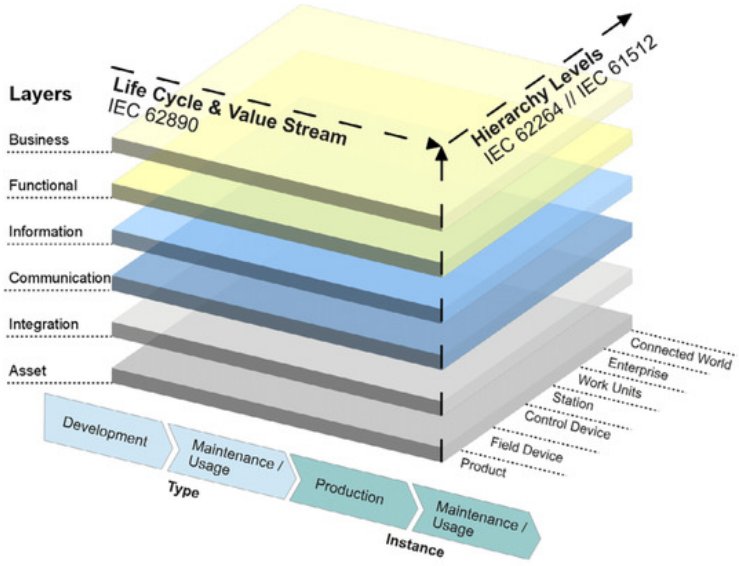


Figure 1.1: Reference Architectural Model Industrie 4.0 (RAMI 4.0) [1]

Chapter 2

Fundamentals and State of the Art

2.1 Asset Administration Shells(AAS)

The AAS is one of the central concepts in the principles of Industry 4.0, providing a methodology for designing and implementing the digital representation of manufacturing process components [22]. This section explores the fundamental concepts of the AAS, its structure, and its role in enabling interoperability within Industry 4.0 ecosystems. This section cover the specifications in [3], [4] that are of relevance to this thesis.

2.1.1 Basic concept of Asset Administration Shell

The AAS is a standardized digital representation of an asset, which can be a physical or virtual entity within an industrial environment. It serves as a cornerstone for achieving interoperability and seamless communication of the asset with other components in the Industry 4.0 ecosystem [2]. Together with the asset it represents, the AAS forms the so-called I4.0 Component (I4.0-Component), as illustrated in Figure 2.1. The asset can be any physical or non-physical entity that has value for an organization, including production systems, equipment, machines, components, products, raw materials, business processes, software, documents, and even human personnel [2].

At its core, the AAS is an information model (meta model) that is exchanged

between I4.0 components to enable interoperability [3]. This information model could be represented as a *eXtensible Markup Language (XML)*, *JavaScript Object Notation (JSON)* or *Resource Description Framework (RDF)* document [3].

The figure 2.2 illustrates the meta model of the AAS, which defines its structure and key components. According to [3], the AAS has an *Identifiable* attribute, which translate to an element that is referable via a unique identifier.This attribute ensures that each AAS can be distinctly identified within the Industry 4.0 ecosystem which also makes them unambiguous and easy to find. Also it make the AAS into an *AdministrativeInformation* ,which contains information about the versioning , who created the last changes and information about the languages available in case the element contains text for translation [3]. Also each AAS has a *HasDataSpecification* attribute, which means that it can be extended by using data specification templates , those define a named set of additional attributes that an element may or shall have [3]. The AAS also has a *derivedFrom* relation, which allows it to reference another AAS from which it is derived. This relation is useful for establishing hierarchies or relationships between different AASs. The most important attribute of the AAS is the *HasSubmodel* attribute, which indicates that the AAS contains one or more submodels. Submodels are modular components within the AAS that encapsulate specific functionalities and data related to the asset [3]. Finally the AAS has a *AssetInformation* attribute, which provides essential meta data about the asset itself, including the *AssetKind*, which classify the asset into a *Type AAS* or *Instance AAS* [3]. A *Type AAS* represents a hardware or software element which specifies the common attributes shared by all instances of the type [3]. An *Instance AAS* represents a specific occurrence of an asset, which contains a concrete , clearly identifiable component of the type [3]. The *AssetInformation* attribute also includes the *globalAssetId*, this attribute is required as soon as the AAS is exchanged via partners in the life cycle of the asset. In a first phase of the life cycle the asset might not yet have a global ID but already an internal identifier. The internal identifier would be modelled via [3].

In summary, the AAS provides a standardized framework for representing assets in a digital format, enabling interoperability and seamless communication within

Industry 4.0 environments. Its meta model defines the structure and key components of the AAS, laying the foundation for effective asset management and integration in modern industrial systems.

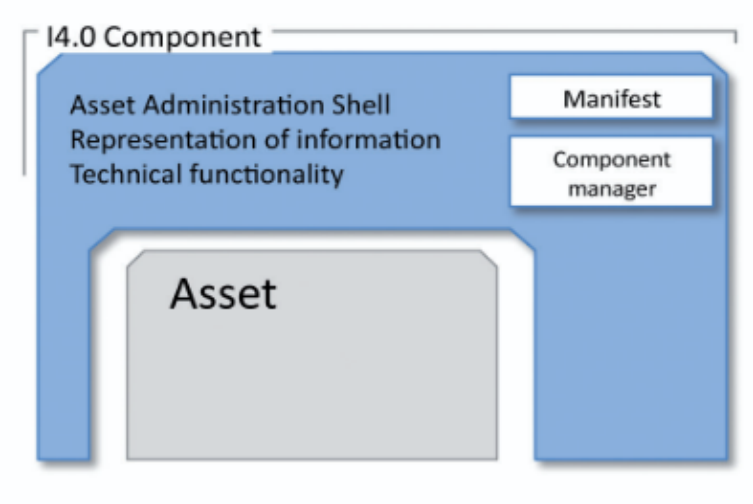


Figure 2.1: The Industry 4.0 Component: Asset and its Administration Shell [2]

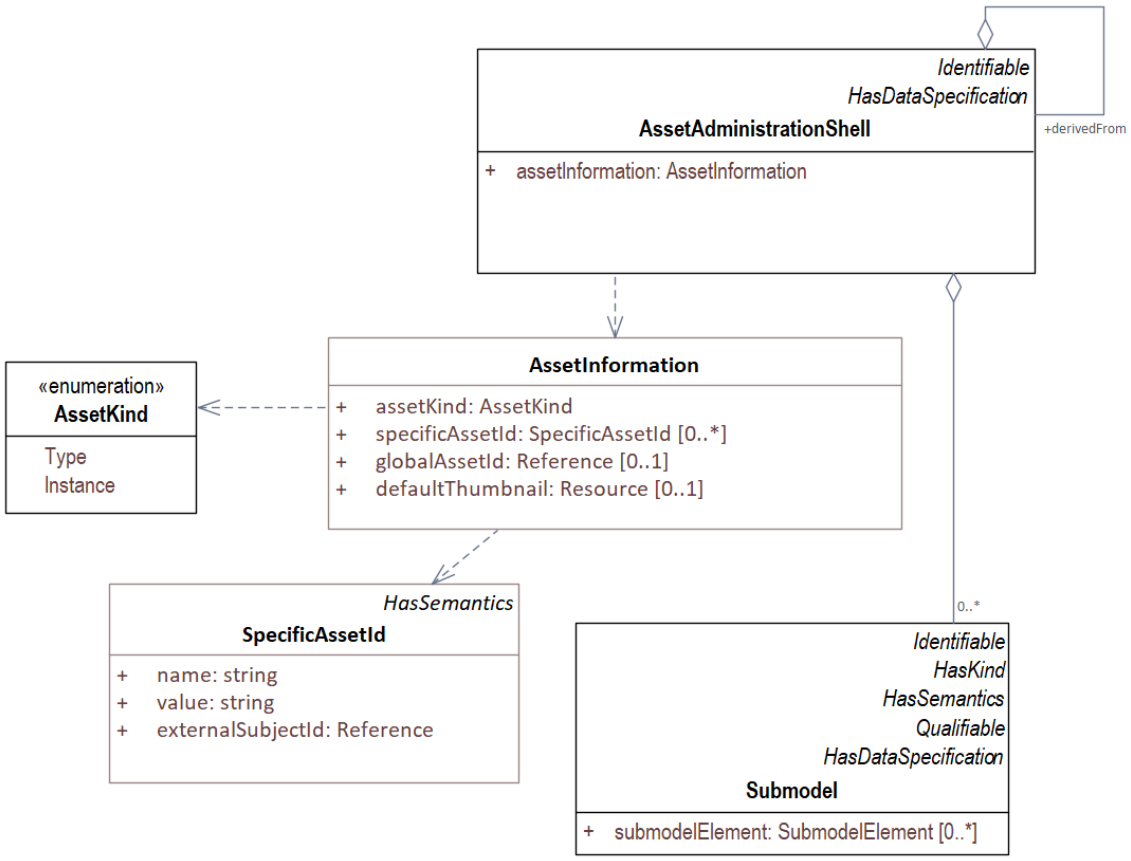


Figure 2.2: Asset Administration Shell Meta Model [3]

2.1.2 Submodels and data types

A submodel is used to structure the digital representation and technical functionality of an Administration Shell into distinguishable parts. Each submodel refers to a well-defined domain or subject matter. Submodels can become standardized and, thus, become submodels templates [3]. In the figure 2.3, the meta model of a submodel is illustrated. Each submodel contains mainly *submodel elements*, which are the building blocks of the submodel structure. *Submodel elements* can represent various types of data and functionalities, including *properties*, *operations*, *events*, and *collections* etc.. [3]. The Submodel also is an *Identifiable* element, which means it has a unique identifier within the context of the AAS [3]. Similar to the AAS, the Submodel also has an *AdministrativeInformation* attribute, which contains versioning and language information [3]. Additionally, the Submodel has a *HasDataSpecification* attribute, allowing it to be extended with data specification templates [3]. This extensibility enables the inclusion of additional attributes and characteristics specific to the submodel's domain [3]. The Submodel can also have a *Qualifier* attribute, which provides additional context or metadata about the submodel element [3]. Finally, the Submodel has a *SemanticId* attribute, which links the submodel to a semantic definition, enabling better understanding and interoperability of the submodel's content [3].

The *SubmodelElement* is the base element for all elements that can be part of a submodel. The *SubmodelElement* can be polymorphic, meaning it takes on different types as shown in figure 2.4.

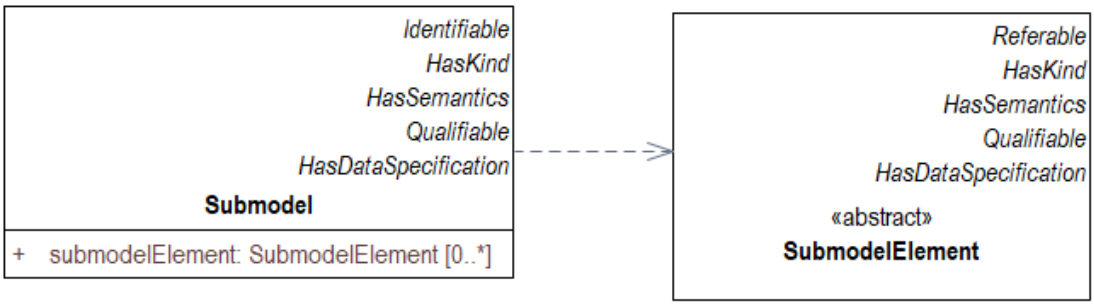


Figure 2.3: Submodel Meta Model [3]

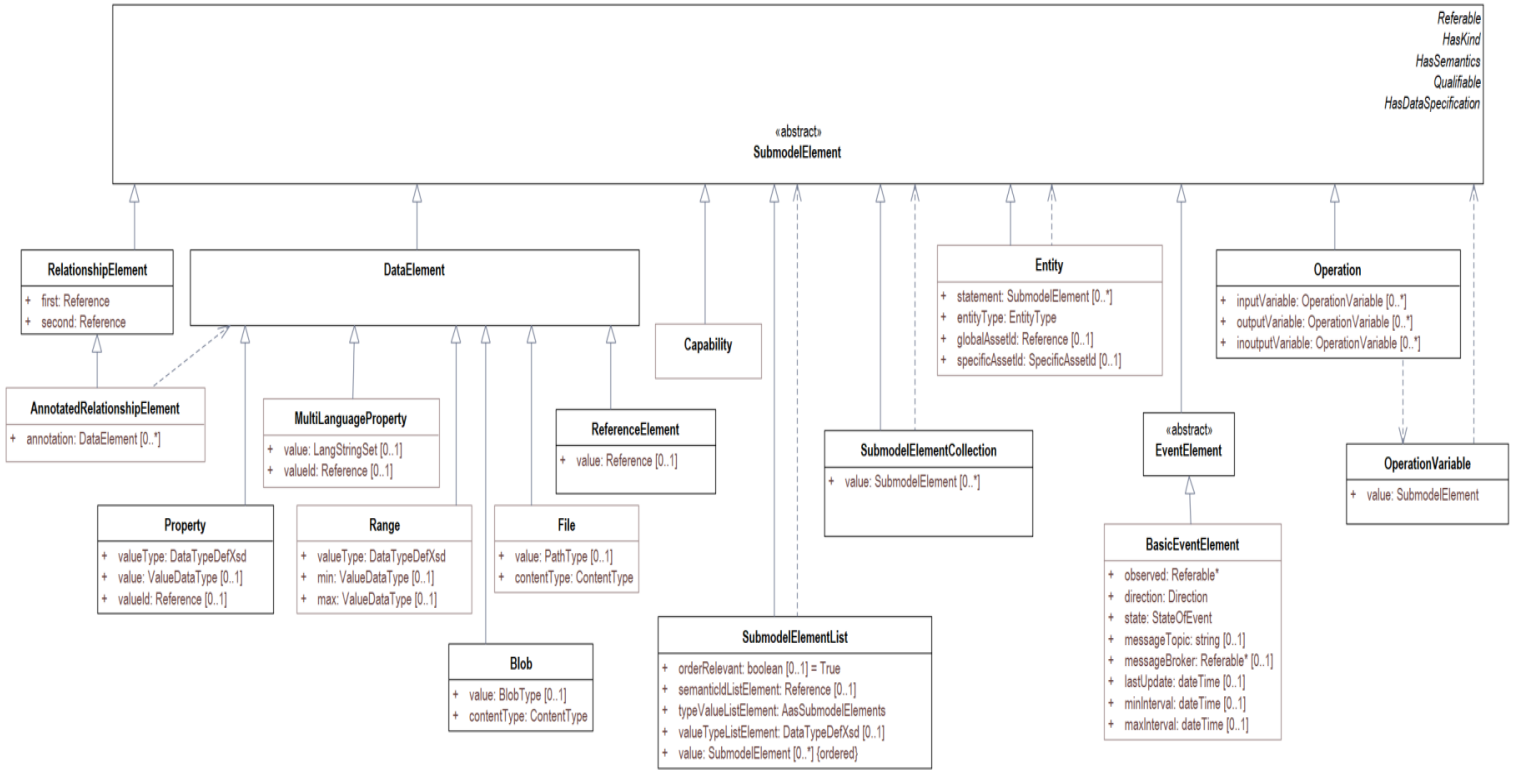


Figure 2.4: Submodel Element Meta Model [3]

In table 2.1, a brief description of the different types of *SubmodelElements* is provided.

Table 2.1: Submodel Element Types and Definitions [3]

Type	Definition	Specification
Property	A data element with a simple value	valueType: DataTypeDefXsd [1..1]
Range	A data element with min and max values	min: ValueDataType [0..1], max: ValueDataType [0..1]
Blob	Binary large object	value: BlobType [0..1], contentType: ContentType
File	Reference to an external file	value: PathType [0..1], contentType: ContentType
ReferenceElement	Reference to another element	value: Reference [0..1]
SubmodelElement-Collection	A collection of submodel elements	value: SubmodelElement [0..*]
SubmodelElement-List	An ordered list of submodel elements	value: SubmodelElement [0..*]
Entity	Represents an entity in a hierarchy	entityType: EntityType [1..1]
RelationshipElement	Defines a relationship between two elements	first: Reference, second: Reference
Annotated-RelationshipElement	Relationship with annotations	annotation: DataElement [0..*]

Continued on next page

Table 2.1 – continued from previous page

Type	Definition	Specification
Capability	Describes a capability of the asset	–
Operation	Executable operation	inputVariable, outputVariable, inoutputVariable
BasicEventElement	Describes an event	observed: Reference [1..1], direction: Direction
MultiLanguage-Property	Property with multi-language support	value: LangStringSet [0..1]
DataElement	Abstract base for data elements	(abstract class)

Along with the different types of *SubmodelElements*, various *data types* are used to define the values and attributes of these elements. These *data types* ensure consistency and standardization in representing information within the AAS and shown in Figure 2.5.

Those *data types* are divided into three main categories:

- **decimalBuildInTypes**: These include basic data types such as *string*, *integer*, *float*, *boolean*, etc., which are used to represent simple values.
- **durationBuildInTypes**: These types are used to represent time durations and intervals, such as *dayTimeDuration* and *yearMonthDuration*.
- **PrimitiveTypes**: This category includes specialized data types like *DateTime*, *Date*, *Time*, *base64Binary*, etc., which provide more specific representations for certain kinds of data.

2.1.3 Types of Asset Administration Shells

The AAS can be classified into three types based on the interaction between the AASs [4]. In Figure 2.6, the three types of AAS are illustrated.

The *type1* AAS represents a basic AAS that is covered in previous sections, where those are simply file exchanges between I4.0 components. The *type2* AAS introduces communication capabilities between AASs,where an *API* specification is defined for that purpose [4]. The *type2* AAS allows for the active exchange of information and services between AASs, enabling more dynamic interactions within the Industry 4.0 ecosystem on the contrary of *type1* which is more passive [4]. The *type3* AAS builds on the capabilities of *type2* and the specifications of *type1* where it creates a peer-to-peer network of AASs which allows for a proactive exchange of the system [4]. According to [4], the *type2* AAS has specification that are listed as in the table 2.2.

Table 2.2: Specification of *type2 API* [3]

Specification	Description	Reference
Create, Read, Update, Delete (CRUD) AAS	create, read, update, delete AAS	[3]
CRUD Submodels	create, read, update, delete submodels	[3]
Synchronous (SYNC) Invokers	Invoke methods synchronously	[3]
Asynchronous (ASYNC) Invokers	Invoke methods asynchronously	[3]
Eventing	subscribe and unsubscribe to events	[3]

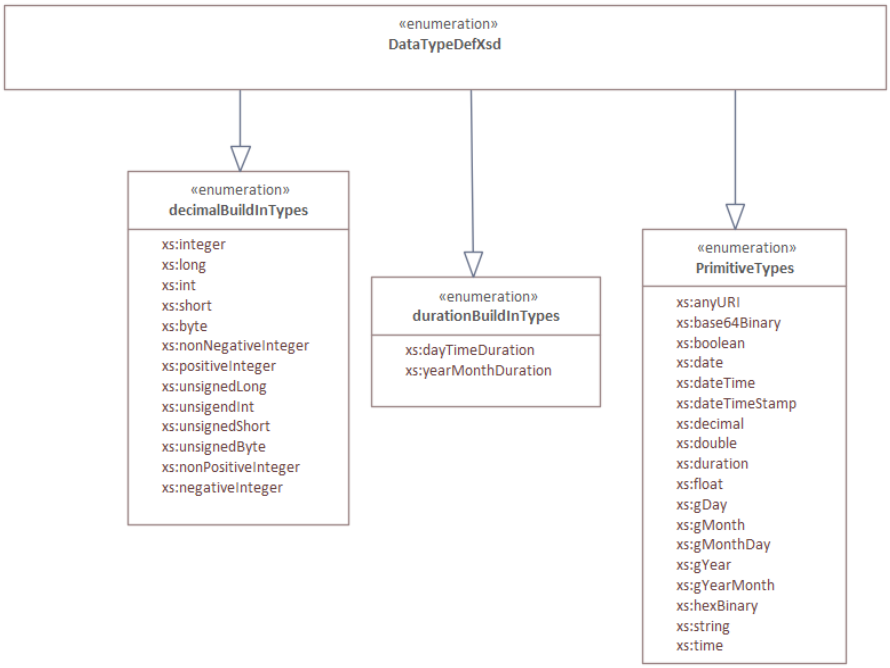


Figure 2.5: Data Types Meta Model [3]

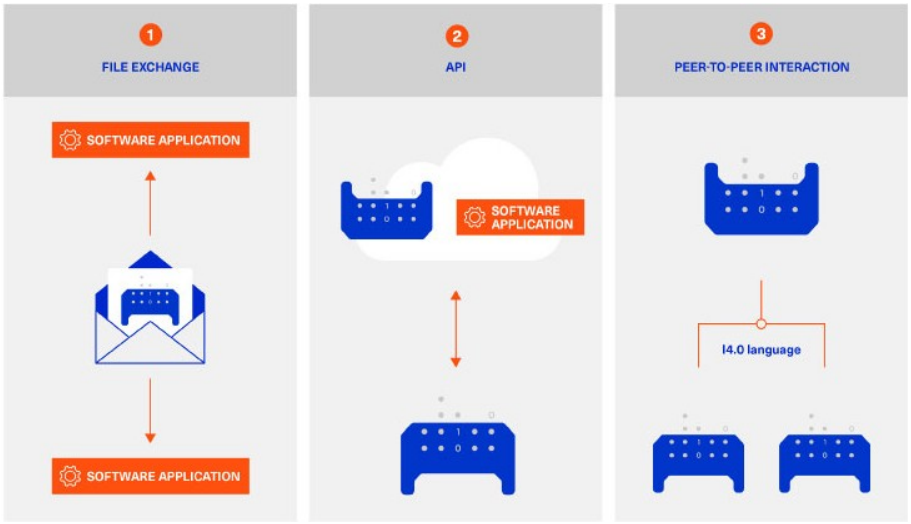


Figure 2.6: Types of Asset Administration Shells [4]

2.1.4 Asset Administration Shell Type3

The *type3* AAS builds upon the capabilities of *type2* AAS by introducing a peer-to-peer network of AASs [5]. The *type3* represents a more advanced and extended form of AAS, acting as a decision-making entity that autonomously interacts with other AASs to exchange information and collaborate (e.g., through negotiation) [5]. The core idea of the Asset Administration Shell AAS concept is that every industrial asset is represented by its own AAS, thereby transforming the asset into an I4.0 component [5]. Because all AASs follow a unified, standardized structure and expose I4.0-compliant interfaces, they enable seamless information provision and exchange across the entire lifecycle of an asset. This standardization facilitates interoperability and real-time data sharing among heterogeneous assets, independent of manufacturer or application domain [23]. In figure 2.7, the concept of *type3* AAS is illustrated. Through their AAS representations, assets are thus envisioned to autonomously collaborate, coordinate processes, and contribute to decentralized decision-making within industrial systems [23]. However, such autonomous, intelligent, and cooperative behavior corresponds primarily to AAS *type3*, which unlike AAS *type1* and *type2* remains comparatively underexplored and currently lacks dedicated specifications and comprehensive implementation guidelines [5]. The *type3* AAS revolves around three main requirements [5] , [23], [24], [25]:

- **Autonomy**: Each AAS should operate independently, makin g decisions and executing actions without human intervention.
- **Intelligence**: AASs should possess cognitive capabilities, enabling them to learn from their environment, adapt to changes, and optimize their performance over time.
- **Collaboration**: AASs should be able to communicate and collaborate with other AASs, forming a network of interconnected agents that work together to achieve common goals.

The *type3* is currently an area of active research and development, with ongoing efforts to define its specifications and explore its potential applications in Industry

4.0 environments [5]. There is to be yet no standardized framework or guidelines for implementing *type3* AAS, making it a promising avenue for future exploration and innovation in the field of industrial automation and digitalization [5].

2.1.5 Summary

This section has introduced the AAS as the standardized digital representation of industrial assets within the Industry 4.0 ecosystem. The key concepts covered include:

- The AAS meta-model, which defines the structure for representing assets through identifiable, self-describing information models that can be exchanged in XML, JSON, or RDF formats.
- Submodels as modular components that encapsulate specific aspects of an asset, including properties, operations, events, and capabilities. Standardized submodel templates enable interoperability across different vendors and application domains.
- The classification of AAS into three types based on their interaction capabilities: Type 1 (file exchange), Type 2 (API-based communication), and Type 3 (peer-to-peer proactive interaction).
- The requirements of Type3AAS—autonomy, proactive behavior, and collaboration—which extend beyond the current specifications and require additional technologies to be realized.

Of particular relevance to this thesis is the Type3AAS, which envisions assets as autonomous entities capable of proactive decision-making and peer-to-peer collaboration. However, as noted, the Type3AAS currently lacks formal specifications and implementation guidelines. The standardized Operation element defined in the AAS meta-model provides a potential interface for executing agent actions, yet this capability remains underutilized in existing implementations. The following sections introduce MAS and the BDI architecture as enabling technologies for realizing the proactive capabilities required by Type3AAS.

2.2 Multi-Agent Systems (MAS)

The concept of autonomous agents and MAS has emerged as a powerful paradigm for designing and implementing complex distributed systems that require autonomous behavior, coordination, and adaptability [26]. This section introduces the fundamental concepts of agents and MAS, establishing the theoretical foundation for understanding their integration with AAS to realize Type3AAS capabilities.

2.2.1 Agents and Their Properties

An agent is commonly defined as "a computer system that is situated in some environment, and that is capable of autonomous action in this environment in order to meet its design objectives" [26]. Unlike conventional software components that execute predefined instructions in response to method calls, agents exhibit a higher degree of autonomy and environmental awareness.

According to [27], an intelligent agent possesses the following essential properties:

- **Autonomy**: Agents operate without direct intervention from humans or other agents. They have control over their own actions and internal state, making independent decisions based on their goals and perceptions.
- **Reactivity**: Agents perceive their environment through sensors or data interfaces and respond in a timely manner to changes that occur in it. This enables agents to adapt to dynamic conditions.
- **Proactivity**: Beyond merely reacting to environmental changes, agents can exhibit goal-directed behavior by taking initiative. They can anticipate future states and act to achieve their objectives without external prompting.
- **Social Ability**: Agents can interact with other agents (and potentially humans) through some form of agent communication language to coordinate activities, negotiate, and cooperate toward shared or individual goals.

These properties distinguish agents from traditional software objects and make them particularly suitable for applications requiring autonomous decision-making

and adaptive behavior, such as the proactive capabilities envisioned for Type3AAS.

2.2.2 Multi-Agent Systems

A MAS is a system composed of multiple interacting agents that work together within a shared environment [26]. Each agent in a MAS has its own goals, capabilities, and knowledge, and the collective behavior of the system emerges from the interactions among individual agents.

MAS offer several advantages over centralized systems [28]:

- **Decentralization**: There is no single point of control or failure. Each agent operates autonomously, making the system more robust and resilient to local failures.
- **Scalability**: New agents can be added to the system without requiring fundamental changes to existing agents, allowing the system to grow incrementally.
- **Flexibility**: Agents can dynamically reorganize and adapt their interactions in response to changing requirements or environmental conditions.
- **Modularity**: Each agent encapsulates specific functionality, promoting separation of concerns and facilitating maintenance and evolution of the system.

These characteristics align closely with the requirements of modern manufacturing systems, where flexibility, adaptability, and resilience are essential to cope with increasing product variety, shorter product lifecycles, and unpredictable disruptions [28].

2.2.3 Agent Architectures

Agent architectures define the internal structure of agents and how they process perceptions to generate actions. Three main categories of agent architectures have been identified in the literature [26]:

Reactive Architectures implement a direct mapping from perceptions to actions without maintaining an internal model of the world. These architectures, such

as the subsumption architecture proposed by Brooks [29], are computationally efficient and provide fast responses but lack the ability to reason about complex goals or plan ahead.

Deliberative Architectures maintain an explicit symbolic model of the world and use logical reasoning to determine appropriate actions. These architectures can handle complex planning tasks but may be computationally expensive and slow to respond to rapidly changing environments.

Hybrid Architectures combine reactive and deliberative components to balance responsiveness with reasoning capabilities. A typical hybrid architecture consists of multiple layers: a reactive layer for immediate responses, a deliberative layer for planning and reasoning, and a coordination layer that mediates between them. The BDI architecture, discussed in Section 2.3, represents a well-established hybrid approach that provides both reactive capabilities and goal-directed reasoning.

2.2.4 Agent Communication

Communication is fundamental to MAS, enabling agents to share information, coordinate activities, and negotiate agreements. The Foundation for Intelligent Physical Agents (FIPA) has established standards for agent communication that are widely adopted in industrial applications [30].

The FIPA Agent Communication Language (ACL) defines a set of performatives (communicative acts) that specify the intended meaning of messages exchanged between agents. Common performatives include:

- `INFORM`: Used to share information or beliefs with other agents
- `REQUEST`: Used to ask another agent to perform an action
- `PROPOSE`: Used to make a proposal during negotiation
- `ACCEPT-PROPOSAL` / `REJECT-PROPOSAL`: Used to respond to proposals
- `CFP` (Call for Proposals): Used to initiate a negotiation or auction

These standardized performatives enable interoperability between agents developed using different frameworks and facilitate the implementation of coordination protocols such as the Contract Net Protocol [31], which is commonly used for task allocation in manufacturing systems.

2.2.5 Agent Coordination and Cooperation

In MAS, agents must coordinate their activities to achieve common goals and avoid conflicts. Several coordination mechanisms have been developed [26]:

Contract Net Protocol: A market-based mechanism where a manager agent announces a task, receives bids from contractor agents, and awards the contract to the most suitable bidder. This protocol is widely used in manufacturing for dynamic task allocation [31].

Auction-Based Coordination: Agents participate in auctions to acquire resources or tasks. Different auction types (English, Dutch, sealed-bid) offer trade-offs between communication overhead and optimality of allocation.

Negotiation: Agents engage in a dialogue to reach mutually acceptable agreements. Negotiation is essential when agents have conflicting interests but need to cooperate for mutual benefit.

Organizational Structures: Agents can be organized into hierarchies, teams, or coalitions with defined roles and responsibilities. Organizational models such as Moise+ [32] provide formal frameworks for specifying agent organizations.

Terminology Clarification. The Contract Net Protocol is classified as a *negotiation* mechanism rather than an auction because it involves a dialogue between the manager and contractors: contractors submit proposals, and the manager selects based on criteria that may include non-price factors such as availability, capability match, and reliability. In contrast, pure auctions typically determine allocation solely based on price/bid value. This thesis implements the Contract Net Protocol for resource allocation and therefore uses the term "negotiation" throughout to describe the coordination process between Production Agent and Resource Agents.

2.2.6 Industrial Agents

The application of agent technology in industrial settings has given rise to the concept of industrial agents, which are agents specifically designed for manufacturing and production environments [28]. Industrial agents typically operate within a two-layered architecture as illustrated in Figure 2.8:

- **Low-Level Control (LLC)**: This layer handles real-time control of physical assets, including sensor monitoring, actuator control, and safety functions. It is typically implemented using Programmable Logic Controller (PLC)s or industrial PCs with real-time operating systems.
- **High-Level Control (HLC)**: This layer implements the agent logic, including decision-making, coordination, and communication with other agents. It provides intelligence and adaptability while delegating time-critical operations to the LLC layer.

This separation allows agents to leverage the reliability and determinism of industrial control systems while adding intelligent, adaptive behavior through the agent layer. The interface between these layers can be implemented using industrial communication standards such as Open Platform Communications Unified Architecture (OPC UA), which aligns well with the communication capabilities specified for AAS [6].

Industrial agents have been successfully applied in various manufacturing scenarios, including flexible manufacturing systems, supply chain management, and production scheduling [28]. Their ability to operate autonomously, coordinate with peers, and adapt to changing conditions makes them natural candidates for implementing the proactive behavior required by Type3AAS.

2.2.7 Summary

This section has introduced the fundamental concepts of agents and MAS, including agent properties, architectures, communication, and coordination mechanisms. The key characteristics of agents autonomy, reactivity, proactivity, and social ability align

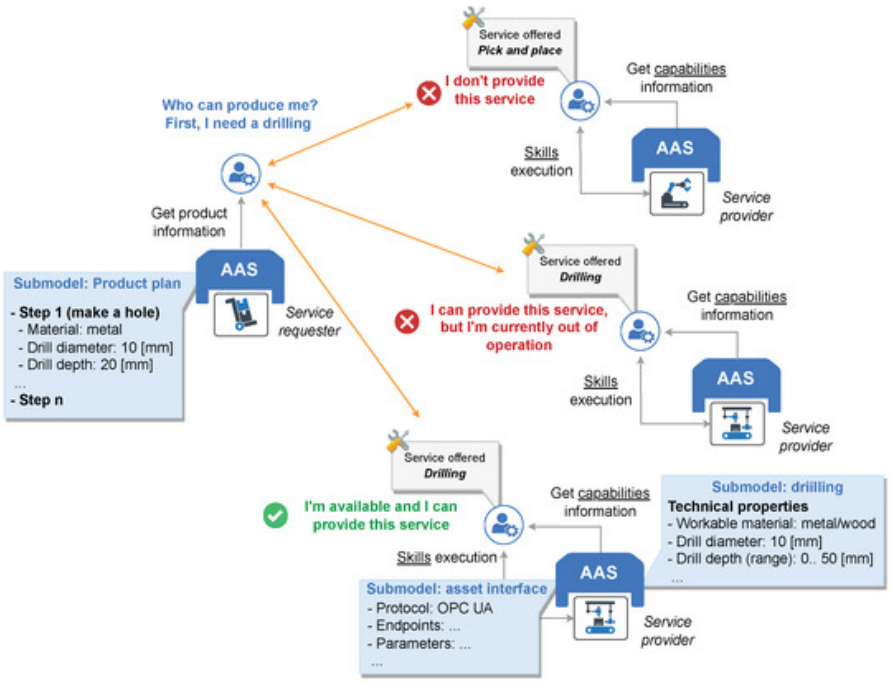


Figure 2.7: Concept of Type3 AAS [5]

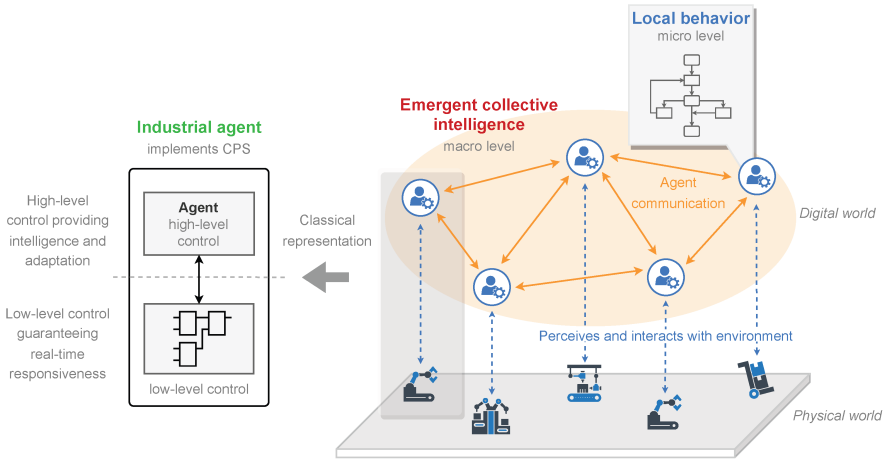


Figure 2.8: Industrial Agent Layers [6]

directly with the requirements identified for Type3AAS: autonomous operation, intelligent decision-making, and collaborative interaction. Industrial agents provide a proven approach for deploying agent technology in manufacturing environments, with established patterns for integrating intelligent behavior with real-time control systems. The following section examines the BDI architecture, which provides a specific model for implementing rational, goal-directed agents suitable for Type3AAS applications.

2.3 BDI (Belief-Desire-Intention) Approach

The BDI architecture represents one of the most influential and well-established models for implementing rational agents capable of autonomous decision-making [27]. Originally rooted in the philosophical theory of human practical reasoning developed by Bratman [33], the BDI model has been successfully adapted into a computational framework for programming intelligent agents that exhibit goal-directed behavior [21].

2.3.1 Foundations of the Belief-Desire-Intention Model

The BDI architecture is grounded in three core mental attitudes that guide an agent's reasoning and behavior [33]:

- **Beliefs**: Represent the agent's knowledge about itself, other agents, and its environment. Beliefs are informational states that the agent holds to be true based on its perceptions and prior knowledge. In the context of AAS, beliefs can be derived from submodel data, sensor readings, or information received from other I4.0 components.

- **Desires**: Represent the goals or objectives that the agent would like to achieve. Desires are motivational states that drive the agent's behavior. For a Type3AAS, desires might include optimizing production throughput, minimizing energy consumption, or completing manufacturing orders.

- **Intentions**: Represent the desires that the agent has committed to pursue. Intentions are deliberative states that the agent has decided to act upon, providing a form of commitment that ensures persistence in goal pursuit while allowing for reconsideration when circumstances change significantly.

The interplay between these three attitudes enables agents to reason about their goals, select appropriate plans, and execute actions in a rational manner [21]. This separation of concerns allows for a clear distinction between what the agent knows (beliefs), what it wants to achieve (desires), and what it is currently trying to do (intentions).

2.3.2 The Belief-Desire-Intention Reasoning Cycle

A BDI agent operates through a continuous reasoning cycle that processes perceptions from the environment and generates actions [34]. The typical reasoning cycle consists of the following steps:

1. **Perception**: The agent perceives changes in its environment and updates its belief base accordingly.
2. **Belief Revision**: Based on new perceptions, the agent revises its beliefs to maintain consistency with the current state of the world.
3. **Option Generation**: The agent generates possible options (potential desires) based on its current beliefs and existing goals.
4. **Deliberation**: The agent deliberates over the available options and selects which goals to commit to as intentions.
5. **Plan Selection**: For each intention, the agent selects an appropriate plan from its plan library that can achieve the intended goal given current beliefs.
6. **Action Execution**: The agent executes the next action in the selected plan, which may affect the environment and trigger new perceptions.

This cycle enables BDI agents to exhibit both reactive behavior (responding to environmental changes) and proactive behavior (pursuing long-term goals), which aligns with the requirements of Type3AAS [27].

2.3.3 Belief-Desire-Intention for Industrial and Cyber-Physical Systems

The application of BDI agents in industrial settings has gained significant attention due to their ability to handle complex decision-making in dynamic environments [6]. In the context of CPS and Industry 4.0, BDI agents offer several advantages:

- **Autonomy**: BDI agents can operate independently, making decisions without constant human supervision, which is essential for autonomous manufacturing systems.
- **Adaptability**: The belief revision mechanism allows agents to adapt to changing conditions in real-time, responding to unexpected events such as machine failures or order changes.
- **Explainability**: The explicit representation of beliefs, desires, and intentions provides a transparent model of agent behavior, facilitating debugging, validation, and trust in autonomous systems [35].
- **Goal-Directed Behavior**: The intention-based commitment strategy ensures that agents persistently pursue their goals while maintaining the flexibility to reconsider when necessary.

2.3.4 Summary

The BDI architecture provides a well-founded theoretical and practical framework for implementing rational agents capable of autonomous decision-making. Its clear separation of beliefs, desires, and intentions enables transparent reasoning processes that are essential for industrial applications where explainability and trust are paramount. The availability of mature programming frameworks such as Jason and JaCaMo facilitates the development of BDI-based multi-agent systems. When integrated with the AAS concept, BDI agents can leverage standardized asset information and communication interfaces to realize the proactive, collaborative behavior envisioned for Type3AAS. This integration forms the conceptual basis for the approach proposed in this thesis.

2.4 Software Frameworks and Platforms

The realization of Type3AAS requires not only theoretical foundations but also practical software tools and frameworks. This section reviews the current state of available frameworks for implementing AAS, MAS, and BDI agents, providing the technical context for the implementation decisions made in this thesis.

2.4.1 Asset Administration Shell Frameworks

Several open-source and commercial frameworks have emerged to support the development and deployment of AAS implementations. The most prominent frameworks include:

Eclipse BaSyx is an open-source middleware platform developed under the Eclipse Foundation that provides a comprehensive implementation of the AAS specification [36]. BaSyx offers:

- Complete implementation of the AAS meta-model and API specification
- Support for both Type 1 (file-based) and Type 2 (API-based) AAS
- Registry and discovery services for AAS components

- Integration capabilities with industrial protocols (OPC UA, Message Queuing Telemetry Transport (MQTT), Hypertext Transfer Protocol (HTTP),Representational State Transfer (REST))
- Modular architecture allowing selective deployment of components

BaSyx is widely adopted in research and industry due to its open-source nature and active community support. However, it does not natively support Type3AAS capabilities such as autonomous decision-making or proactive behavior.

Fraunhofer Advanced AAS Tools for Simplified Interoperability (FAAAST) is another open-source framework that focuses on ease of use and rapid prototyping [37]. It provides:

- Lightweight implementation suitable for resource-constrained environments
- Support for multiple serialization formats (JSON, XML, RDF)
- Built-in synchronization with asset data sources
- Extensible architecture for custom functionality

AASX Package Explorer is a desktop application developed by the Industrial Digital Twin Association (IDTA) for creating, viewing, and editing AAS packages [38]. While primarily a development and visualization tool rather than a runtime framework, it serves as an important reference implementation and is commonly used for:

- Creating and validating AAS models
- Importing and exporting AAS packages in standardized formats
- Visualizing submodel structures and relationships
- Testing interoperability between different AAS implementations

Limitations for Type 3 AAS. While current AAS frameworks provide the technical infrastructure for Type 2 functionality including event notification (via

MQTT) and operation invocation (via REST API) they lack the higher-level capabilities required for truly autonomous, proactive behavior. Table 2.3 clarifies what is present in existing frameworks and what is missing for Type 3 realization.

Table 2.3: Gap Analysis: Type 2 Capabilities vs. Type 3 Requirements

Aspect	Type 2 (Available)	Type 3 (Missing)
Event Handling	Publish/subscribe via MQTT; external systems react to events	Internal goal generation from events; autonomous decision which goals to pursue
Operation Invocation	External clients call AAS Operations	AAS autonomously decides *when* and *which* operations to invoke
Data Access	Submodel read/write via API	Submodel data interpreted as beliefs for reasoning
Coordination	Point-to-point request/response	Multi-agent negotiation protocols (e.g., CNP)
Behavior	Reactive: responds when called	Proactive: initiates actions based on internal goals
Decision Logic	External (in MES, ERP, or human)	Internal (embedded reasoning engine)

The fundamental distinction is *where* the decision-making logic resides:

- **Type 2**: The AAS exposes data and operations, but an *external* system (MES, orchestrator, or human operator) decides what to do and when. The AAS is a *servant* that executes commands.
- **Type 3**: The AAS contains an *internal* reasoning engine that autonomously decides what goals to pursue based on its current beliefs and desires. The AAS is an *agent* that acts on its own behalf.

Concretely, existing frameworks lack:

1. **Goal-Oriented Reasoning**: No mechanism to represent goals, evaluate alternatives, and commit to intentions. Frameworks provide *mechanisms* (events, operations) but not *policies* for when to use them.

2. **Belief Management**: Submodel data exists, but there is no inference engine to derive new beliefs, detect inconsistencies, or reason about uncertain information.

3. **Plan Libraries**: No way to define reusable plans that specify how to achieve goals under different conditions.

4. **Negotiation Protocols**: While messaging infrastructure exists (MQTT, HTTP), there are no built-in protocols for multi-party negotiation such as Contract Net Protocol, auctions, or coalition formation.

5. **Autonomy Lifecycle**: No support for agent initialization, goal adoption, plan selection, intention execution, and replanning on failure.

This analysis confirms that realizing Type 3 AAS requires integration with agent technologies that provide these missing capabilities. The BDI architecture, in particular, offers a well-established model for goal-oriented reasoning, belief management, and intention execution filling precisely the gaps identified above. This motivates the approach taken in this thesis: combining the standardized data model and communication infrastructure of AAS frameworks with the autonomous reasoning capabilities of BDI agents.

2.4.2 Multi-Agent System Frameworks

Several mature frameworks exist for developing MAS, each with different design philosophies and target applications:

JAVA Agent Development Framework (JADE) is one of the most widely used MAS platforms, providing a FIPA-compliant infrastructure for agent development in Java [39]. Key features include:

- Full compliance with FIPA specifications for agent communication
- Distributed agent platform with support for multiple containers
- Built-in agent management and directory services

- Graphical tools for debugging and monitoring agent behavior
- Behavior-based agent programming model

JADE has been extensively used in industrial applications and research projects. However, its behavior-based programming model does not directly support cognitive agent architectures such as BDI.

Smart Python Agent Development Environment (SPADE) is a modern MAS platform built on Python and Extensible Messaging and Presence Protocol (XMPP) [40]. SPADE offers:

- Asynchronous agent execution using Python's asyncio
- XMPP-based communication providing robust, scalable messaging
- Web-based interface for agent monitoring
- Flexible behavior system with support for cyclic, one-shot, and finite-state behaviors
- Easy integration with Python's extensive ecosystem of libraries

SPADE's use of XMPP provides advantages for industrial deployments, including message persistence, presence detection, and established security mechanisms. Its Python foundation also facilitates rapid prototyping and integration with data science tools.

JADE eXtension (JADEX) extends JADE with BDI reasoning capabilities, combining JADE's robust infrastructure with cognitive agent programming [41]. JADEX supports:

- BDI agent development using XML-based agent definitions
- Integration with JADE's communication infrastructure
- Goal-oriented programming with automatic plan selection
- Active components model for service-oriented architectures

2.4.3 Framework selection considerations

The selection of appropriate frameworks for implementing Type3AAS depends on several factors:

- **AAS Compliance**: The AAS framework must support the current specification, including submodel operations and standardized APIs.
- **Agent Architecture Support**: For BDI-based implementations, the MAS framework should either natively support BDI or allow integration with BDI reasoning engines.
- **Communication Capabilities**: Industrial deployments require robust, scalable communication mechanisms that can handle asynchronous messaging and potential network disruptions.
- **Integration Flexibility**: The ability to connect AAS and MAS components through well-defined interfaces is essential for realizing bidirectional integration.
- **Development Ecosystem**: Practical considerations such as documentation quality, community support, and available tooling influence development efficiency.

Tables 2.4,2.5 provides a comparative overview of the discussed frameworks.

2.4.4 Summary and Framework Selection

This section has reviewed the current landscape of software frameworks for AAS, MAS, and BDI agent development. While mature frameworks exist for each domain individually, no single framework provides integrated support for Type3AAS implementations. AAS frameworks such as Eclipse BaSyx offer comprehensive support for Type 1 and Type 2 functionality but lack autonomous behavior capabilities. MAS frameworks provide the agent infrastructure needed for autonomy and collaboration, with platforms like SPADE offering modern, asynchronous communication.

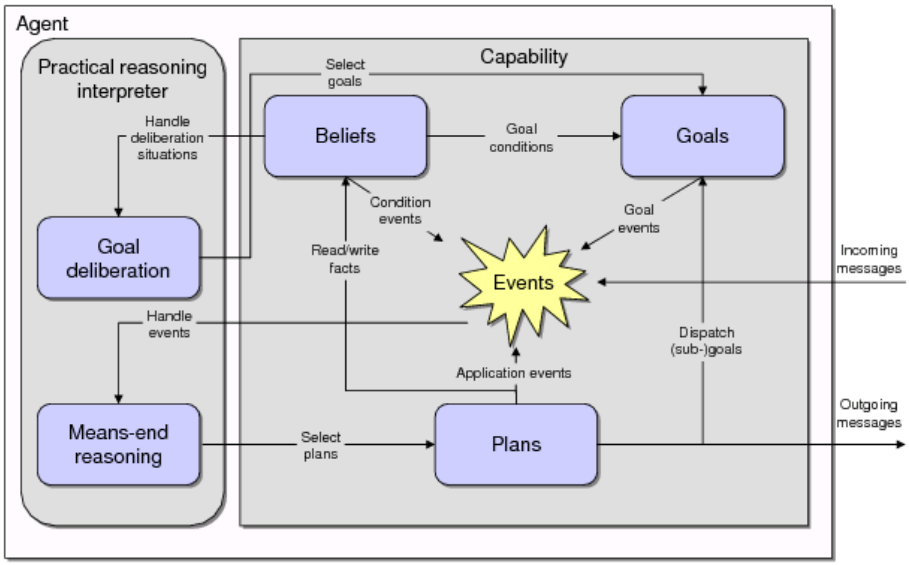


Figure 2.9: BDI Reasoning Cycle [7]

Table 2.4: Comparison of Asset Administration Shell frameworks

Framework	Language	AAS Types	Communication
Eclipse BaSyx	Java / Python	Type 1, Type 2	HTTP/REST, MQTT, OPC UA
FA³ST Service	Java	Type 1, Type 2	HTTP/REST, OPC UA
AASX Package Explorer	C#	Type 1	File-based (AASX)

Table 2.5: Comparison of Multi-Agent System and BDI frameworks

Framework	Language	BDI Support	Communication
JADE	Java	No (ext.)	FIPA ACL
SPADE	Python	Extension (spade-bdi)	XMPP
JADEX	Java/XML	Yes (native)	FIPA ACL

Selected Technology Stack. Based on the framework comparison and the requirements of this thesis, the following technologies were selected for the Type3AAS implementation:

- **Eclipse BaSyx** for AAS hosting, providing a mature, standards-compliant server with REST API, registry services, and MongoDB persistence. The BaSyx Python SDK enables programmatic submodel creation and access.
- **SPADE with spade-bdi extension** for the agent runtime, combining Python's asyncio-based execution with BDI reasoning capabilities. SPADE's XMPP foundation provides robust publish-subscribe messaging essential for multi-agent coordination.
- **Prosody** as the XMPP server, selected for its compatibility with SPADE and support for the PubSub extension (XEP-0060) required for event-driven agent communication.

This combination bridges the gap between AAS and MAS domains: BaSyx provides the standardized digital twin infrastructure while SPADE enables the autonomous, goal-directed behavior characteristic of Type3AAS. The detailed configuration and integration of these components is presented in Chapter 5.

2.5 Capability-Skill-Service Information Model

The Plattform Industrie 4.0 and IDTA working groups have developed a standardized information model for describing manufacturing capabilities, skills, and services. This model, published in the document "Information Model for Capabilities, Skills & Services" [8], provides a technology-independent framework for flexible manufacturing systems. Understanding this model establishes the theoretical foundation for capability-based production coordination.

2.5.1 Model Overview

The Capability-Skill-Service (CSS) information model organizes manufacturing functionality into three distinct abstraction layers, as illustrated in Figure 2.10. Each layer serves a specific purpose in describing what a manufacturing system can do, how it does it, and how these functions are offered to external requesters.

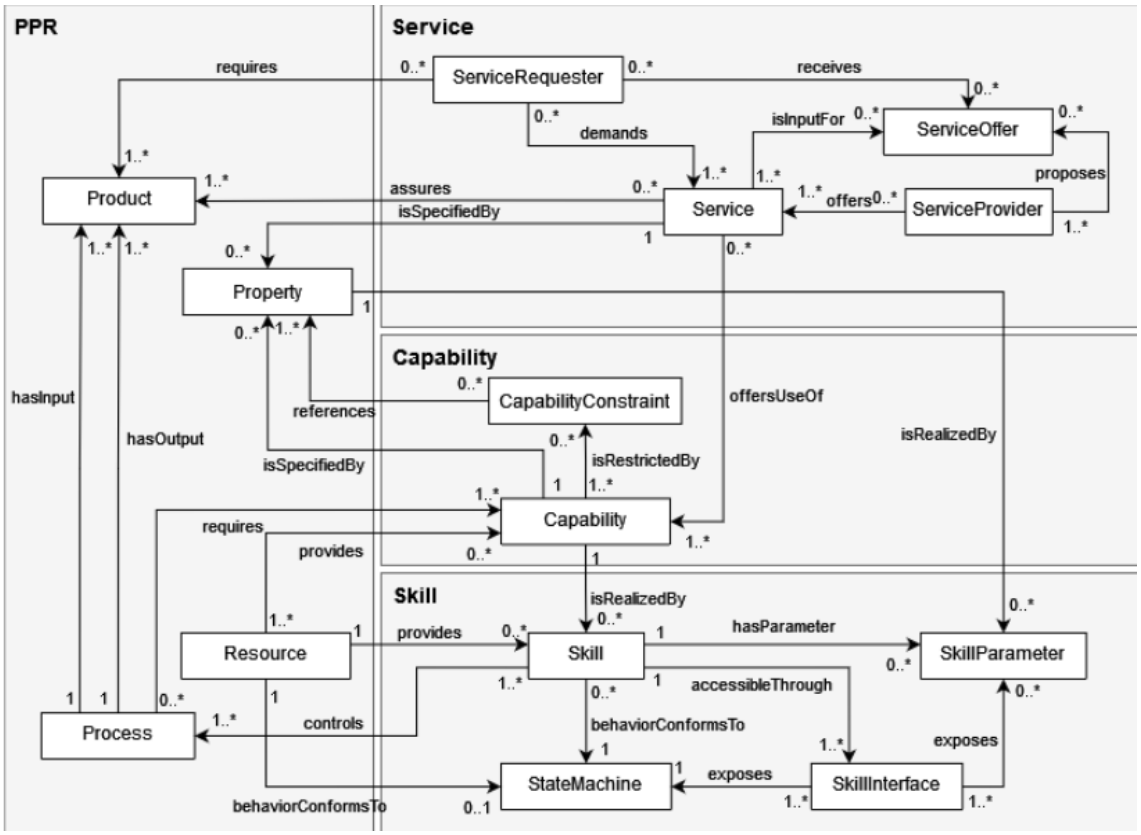


Figure 2.10: Capability-Skill-Service Information Model [8]

2.5.2 Product-Process-Resource (PPR)

The left side of the model represents the classic manufacturing triad:

Product. The item being manufactured, defined by its required inputs and expected outputs. Products specify which capabilities are needed for their production through the *requires* relationship.

Process. The sequence of manufacturing steps that transform inputs into outputs. Processes control resources and conform to defined behavioral specifications.

Resource. The physical or virtual equipment that performs manufacturing operations. Resources *provide* both capabilities (abstract) and skills (concrete), establishing the link between what is required and what is available.

2.5.3 Capability Layer

Capabilities represent **what** a resource can do at an abstract, technology-independent level:

Capability. A machine-interpretable specification of a function that can potentially be performed. Capabilities are described in terms of their effects rather than their implementation. For example, a "drilling" capability specifies the ability to create holes without defining the specific machine or method.

CapabilityConstraint. Restrictions that limit a capability's applicability, such as maximum drilling depth, material constraints, or dimensional tolerances. These constraints enable capability matching during production planning.

The key characteristic of capabilities is their **technology neutrality**—they describe manufacturing functions independently of how these functions are realized. This abstraction enables:

- Capability matching between product requirements and resource offerings
- Resource substitution when multiple resources provide equivalent capabilities
- Long-term production planning without binding to specific equipment

2.5.4 Skill Layer

Skills represent **how** a capability is concretely implemented on a specific resource:

Skill. An executable implementation of a capability bound to a particular resource. A skill is the actual function that can be invoked to perform work. Multiple skills may realize the same capability (e.g., different drilling machines implementing the drilling capability).

SkillParameter. Input and output parameters that configure skill execution. Parameters define the interface contract for skill invocation (e.g., target coordinates, drilling depth, spindle speed).

SkillInterface. The technical interface through which a skill is accessible. This specifies the communication protocol (REST, OPC UA, MQTT) and endpoint information required for invocation.

StateMachine. Defines the behavioral specification of a skill, including its states (idle, executing, completed, error) and valid transitions. Skills conform to their state machine specification, enabling predictable execution monitoring.

The relationship *Capability isRealizedBy Skill* connects the abstract and concrete layers, allowing production systems to reason about capabilities while executing skills.

2.5.5 Service Layer

Services provide a business-oriented view for coordinating manufacturing functions:

Service. A higher-level offering that combines one or more capabilities into a coherent function. Services represent what is offered to external systems or users.

ServiceProvider. An entity (resource, agent, or system) that offers services to requesters. Providers advertise their services and respond to service requests.

ServiceRequester. An entity that demands services to fulfill production goals. Requesters discover available services and negotiate with providers.

ServiceOffer. A concrete proposal from a provider in response to a service request, including availability, constraints, and terms.

The service layer enables market-like coordination where requesters discover

providers, negotiate terms, and establish agreements—patterns well-suited to multi-agent systems.

2.5.6 Key Relationships

Table 2.6 summarizes the principal relationships in the CSS model:

Table 2.6: Key Relationships in the CSS Information Model

Relationship	From → To	Meaning
requires	Product → Capability	Product needs this capability
provides	Resource → Capability	Resource offers this capability
provides	Resource → Skill	Resource has this implementation
isRealizedBy	Capability → Skill	Abstract is implemented by concrete
isRestrictedBy	Capability → Constraint	Capability has limitations
hasParameter	Skill → SkillParameter	Skill has input/output parameters
accessibleThrough	Skill → SkillInterface	How to invoke the skill
behaviorConformsTo	Skill → StateMachine	Skill follows state behavior
offersUseOf	Service → Capability	Service provides access to capability

2.5.7 Relevance for Type 3 AAS

The CSS information model provides a theoretical foundation for autonomous AAS implementations:

- **Capability Matching**: Agents can reason about abstract capabilities to find suitable resources without knowledge of specific skill implementations.
- **Service Negotiation**: The ServiceRequester/ServiceProvider pattern aligns with multi-agent negotiation protocols like the Contract Net Protocol.
- **Skill Invocation**: The SkillInterface concept maps directly to the AAS Operation invocation mechanism.
- **State Tracking**: The StateMachine concept supports execution monitoring and agent belief updates.

The IDTA has published submodel templates that implement portions of this model within the AAS framework, enabling standardized capability and skill descriptions across Industry 4.0 systems.

2.6 Integration of Asset Administration Shell with Multi-Agent Systems

The preceding sections have established the foundational concepts of AAS as standardized digital representations of industrial assets, MAS as a paradigm for autonomous and collaborative systems, and the BDI architecture as a model for goal-directed agent behavior. This section examines how these concepts can be combined to realize Type3AAS implementations, reviewing existing approaches and identifying the research gaps that this thesis addresses.

As discussed in Section 2.1.4, the requirements of Type3AAS autonomy, proactive behavior, and collaboration align closely with the core properties of agents identified in Section 2.2.1. This alignment has motivated several research efforts to implement Type3AAS using MAS concepts. Table 2.7 summarizes the key approaches found in the literature.

MAS represent a decentralized approach to problem-solving, by distributing tasks and responsibilities among multiple agents autonomously working towards common objectives, on the contrary of traditional monolithic systems where a single entity controls the entire system [28]. There are several attempts to implement *type3* AAS based on MAS concepts. Those attempts are summarized in the table 2.7.

Table 2.7: Summary of AAS Type 3 implementations based on MAS concepts

Reference	Approach	Key Features
Vogel-Heuser et al. (2021) [24]	Approach using AAS as knowledge base for the agents	Demonstrated on a stack-crane system

Continued on next page

Table 2.7 – continued from previous page

Reference	Approach	Key Features
Sakurada et al. (2022) [42]	Method for implementing AASs using MAS	Decentralized production control, peer-to-peer communication, demonstrated on simulation
Siatras et al. (2024) [43]	Agents operate in MAS to solve scheduling problems where the AAS provide production and agent information	Demonstrated on real production line
Sidorenko et al. (2023) [44]	A framework that combines MAS with AAS to enable human-centered manufacturing	Demonstrated on robot with human collaboration
Jungbluth et al. (2022) [45]	Modeling AAS submodels for resource and production agents	Demonstrated on an assembly line

The literature shows consensus that submodels serve as standardized knowledge sources for agents, capturing asset-related information in formats accessible to both humans and machines [5]. Agents leverage these submodels to support decision-making and enrich their knowledge bases. Standardized submodels, such as the Control Component Instance [46], Asset Interfaces Description [47], and Time-Series Data [48], describe the skills offered by an asset, the methods for interfacing with it, and the mechanisms for managing operational data.

Beyond standardized submodels, custom submodels can be defined to address specific application requirements and represent agent-specific information, such as agent behaviors, decision-making strategies, or interaction protocols, thereby enhancing transparency and understanding of agent capabilities [5].

The review further reveals that communication capabilities represent one of the most emphasized and valuable features of agents. This inherent characteristic positions agents as powerful enablers for achieving the decentralized and collaborative interactions particularly through negotiation strategies envisioned in Type3AAS implementations [5].

2.6.1 Critical Analysis of Existing Approaches

The existing approaches to implementing Type3AAS based on MAS concepts, as summarized in Table 2.7, demonstrate promising directions but also reveal significant limitations that motivate further research.

Strengths of Current Approaches

Several positive aspects emerge from the reviewed literature. First, there is a clear consensus on the architectural separation between the AAS as a standardized knowledge repository and the agent as an autonomous decision-making entity [24,42]. This separation allows each component to leverage its respective strengths: the AAS provides interoperability and standardized data access, while agents contribute autonomous behavior and coordination capabilities.

Second, the use of submodels as structured knowledge sources for agents has proven effective across multiple implementations [5, 45]. Standardized submodels such as the Control Component Instance [46] provide a common vocabulary for describing asset capabilities, enabling agents to interpret and reason about heterogeneous assets in a uniform manner.

Third, several approaches have successfully demonstrated decentralized production control through agent negotiation and collaboration [42,43]. This validates the fundamental premise that MAS can enable the peer-to-peer interactions envisioned for Type3AAS.

Limitations and Open Challenges

Despite these advances, the reviewed approaches exhibit several common limitations:

Unidirectional AAS Usage. Current approaches primarily use the AAS as a passive, read-only data source. Agents retrieve information from submodels to inform their decisions, but the potential of the AAS as an interface for executing actions remains largely unexplored. The AAS specification defines Operations as executable functions that can be invoked synchronously or asynchronously [4], yet existing implementations do not fully leverage this capability for agent action execution.

Lack of Standardized Submodels for Agent Behavior. While standardized submodels exist for describing asset capabilities (e.g., Control Component Instance [46]) and interfaces (e.g., Asset Interfaces Description [47]), there are no established submodel templates specifically designed to represent agent-related information such as goals, desires, or behavioral parameters. This limits the transparency and interoperability of agent-based Type3AAS implementations.

Validation Predominantly in Simulation. The majority of approaches have been validated only in simulated environments [42] or on simplified laboratory setups [24]. While simulation provides a controlled environment for initial validation, it does not fully capture the complexities, uncertainties, and real-time constraints of actual industrial deployments.

Lack of Standardized Agent-AAS Integration Patterns. Each reviewed approach implements the integration between agents and AAS in a different manner, with no established design patterns or reference architectures. This heterogeneity hinders reusability, comparison, and adoption across different industrial contexts.

Missing Connection Between AAS Operations and Agent Actions. A critical gap exists in how agent intentions are translated into standardized AAS operations. Current implementations typically use proprietary interfaces for agent-asset interaction, bypassing the standardized Operation interface defined in the AAS specification. This undermines the interoperability benefits that the AAS standard aims to provide.

2.6.2 Addressed Research Gap

This thesis addresses the identified limitations by proposing a systematic integration of AAS with BDI-based agents that fully leverages the AAS standard. Specifically, this work contributes:

1. **Bidirectional AAS Integration**: Rather than using the AAS solely as a data source, this thesis proposes a bidirectional integration where agents both read from submodels (to form beliefs) and invoke Operations (to execute intentions). This approach fully utilizes the capabilities specified in the AAS standard, enabling agents to interact with assets through standardized interfaces rather than proprietary solutions.

2. **Foundational Submodels for Proactive Behavior**: This work defines submodel structures specifically designed to support agent-based Type3AAS implementations. These submodels represent goals, capabilities, and behavioral parameters that enable proactive decision-making while maintaining compliance with the AAS meta-model. This contribution addresses the lack of standardized templates for agent-related information.

3. **Proof-of-Concept with Production Resources**: The proposed approach is validated through a concrete use case involving drill machines as production resources. This demonstration shows the feasibility of implementing BDI-based Type3AAS in a realistic production scenario, demonstrating feasibility in a realistic production scenario with discrete-event simulators that model actual machine behavior and timing.

By addressing these gaps, this thesis aims to advance the state of the art toward truly interoperable, proactive I4.0 components that leverage the full potential of the AAS standard while benefiting from the autonomous capabilities provided by agent technology.

Chapter 3

Use Cases and System Requirements

The preceding chapter established the theoretical foundations and identified the research gaps that this thesis addresses. Based on these findings, this chapter derives concrete use cases and system requirements for implementing a Type3AAS using BDI-based MAS. Following the systems engineering approach, the use cases are derived from the research questions identified in Section 2.6.1, and the system requirements are subsequently derived from these use cases. Detailed specifications are provided in Appendix 8.1.

3.1 From Research Questions to Use Cases

The critical analysis in Chapter 2 identified three main contributions that this thesis aims to provide: bidirectional AAS integration enabling agents to both read from submodels and invoke Operations, foundational submodels supporting proactive behavior, and a proof-of-concept validation with production resources.

To demonstrate and validate these contributions, a production scenario involving drill machines (in virtual environment) was selected. The *SmartDrillSystem* serves as the proof-of-concept implementation, consisting of multiple AAS-represented drill machines (ResourceAAS) that can be orchestrated by production orders (ProductionAAS). This scenario allows the demonstration of all key Type3AAS capabili-

ties: autonomous behavior, peer-to-peer communication, negotiation, and proactive decision-making.

3.2 System Components

The SmartDrillSystem comprises four main components that work together to realize Type3AAS functionality:

ResourceAAS represents the digital twin of each drill machine, combining the standardized AAS structure with a BDI agent runtime. Each ResourceAAS maintains submodels describing the asset's operational state, capabilities, and interaction parameters, while the embedded agent provides autonomous decision-making and negotiation capabilities.

ProductionAAS manages production orders and coordinates the allocation of tasks to available resources. When a drilling order is received, the ProductionAAS agent queries the registry for available resources, initiates negotiations based on defined criteria (such as time availability), and delegates the production plan to the selected ResourceAAS.

AAS Registry provides a central discovery service where all AAS components register themselves upon initialization. This enables dynamic discovery of available resources without hardcoded dependencies between components.

Event Broker implements an MQTT-based messaging infrastructure that enables asynchronous, event-driven communication between components. This supports real-time status updates and reactive behavior across the distributed system.

3.3 Use Cases

The SmartDrillSystem is specified through three primary use cases that cover the complete lifecycle of the system: Startup, Drilling Order, and Shutdown. Additionally, two included use cases (AAS Registration and Drill Operation) are invoked by the primary use cases to handle specific sub-processes. The relationship between these use cases is illustrated in Figure 3.1.

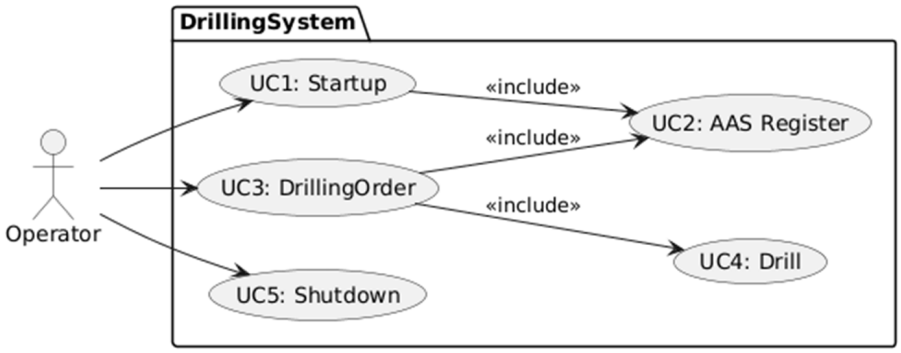


Figure 3.1: Use Case Diagram for the SmartDrillSystem

The detailed use case specifications are provided in Appendix 8.1.1.

3.3.1 UC1: System Startup

The System Startup use case describes the initialization sequence when an operator starts the SmartDrillSystem. During startup, the system initializes all components including the Event Broker, AAS Registry, ResourceAAS instances, and ProductionAAS. The startup process includes the *AAS Registration* (UC2) sub-process, where each ResourceAAS registers itself with the central registry, making it discoverable for subsequent production orders. Upon successful initialization, the system transitions to the ready state and awaits drilling orders. If any component fails to initialize, the system displays appropriate error messages and attempts retry procedures.

3.3.2 UC3: Drilling Order

The Drilling Order use case represents the core production scenario and demonstrates the key Type3AAS capabilities. When an operator submits a drilling order with specified parameters (depth, diameter, material type), the system delegates this order to the ProductionAAS. The ProductionAAS agent then retrieves available resources from the registry and initiates a negotiation process with all registered ResourceAAS agents. Using time availability as the primary negotiation criterion, the ProductionAAS selects the most suitable resource and assigns the production plan.

The selected ResourceAAS then executes the *Drill Operation* (UC4) sub-process,

which handles the actual drilling execution through the bidirectional AAS integration. During this sub-process, the ResourceAAS agent updates its beliefs from submodels, forms intentions, and invokes standardized AAS Operations on the physical asset. Throughout execution, the ResourceAAS publishes status events, and upon completion, the system returns to the ready state for the next order. Alternative flows handle invalid input parameters and drilling failures with appropriate error handling and recovery procedures.

3.3.3 UC5: System Shutdown

The System Shutdown use case ensures graceful termination of the SmartDrillSystem. When an operator initiates shutdown, the system stops all active production processes, completes or cancels pending drilling orders, deregisters all ResourceAAS from the registry, and terminates all components in the proper sequence. This controlled shutdown procedure prevents data loss and ensures system integrity.

3.3.4 Included Use Cases

In addition to the primary use cases described above, two included use cases support the main workflows:

UC2: AAS Registration is included in UC1 (System Startup) and handles the dynamic registration of ResourceAAS instances with the central registry. Each ResourceAAS sends its identification information, capabilities, and communication endpoints to the registry, enabling participation in subsequent negotiation processes.

UC4: Drill Operation is included in UC3 (Drilling Order) and describes the execution of drilling operations by a ResourceAAS. This use case demonstrates the bidirectional integration between the BDI agent and the AAS, where beliefs are derived from submodels and intentions are executed through standardized Operations.

3.4 System Requirements

Based on the use cases defined above, this section summarizes the system requirements organized into functional and non-functional categories. The complete requirements specification with detailed descriptions is provided in Appendix 8.1.

3.4.1 Functional Requirements

The functional requirements are organized into four groups addressing different aspects of the system.

System Specifications (REQ_1 – REQ_8)

The system-level requirements define the overall architecture and fundamental capabilities of the SmartDrillSystem. These requirements specify that the system must comprise ResourceAAS, ProductionAAS, an Event Broker, and an AAS Registry as its core components. The system must provide standardized communication between AAS instances and support event-based messaging through the broker. Furthermore, the requirements mandate that AAS components operate autonomously, possess decision-making capabilities, and collaborate through interdependent processes. These requirements directly address the Type3AAS characteristics of autonomy, intelligence, and collaboration identified in Chapter 2.

ResourceAAS Specifications (REQ_10 – REQ_20)

The ResourceAAS requirements specify the structure and behavior of drill machine representations. Each ResourceAAS must contain an agent interface supporting FIPA ACL communication, a BDI agent runtime, standardized submodels, and an asset interface for physical device communication. The required submodels include an Operational State Submodel (tracking current state and performance KPIs), a Knowledge Submodel (capturing resource constraints and environmental conditions), a Capability Submodel (describing available skills and service parameters), and an Interaction and Scheduling Submodel (defining communication endpoints and collaboration protocols). Additionally, the ResourceAAS must support event

publishing and subscription, and participate in negotiations with ProductionAAS using time availability as the decision criterion.

ProductionAAS Specifications (REQ_21 – REQ_31)

The ProductionAAS requirements define the structure for production order management. Similar to ResourceAAS, each ProductionAAS must contain an agent interface, agent runtime, and specialized submodels. The required submodels include an Identification Submodel (containing order metadata), a Process Plan Submodel (defining tasks, precedences, and required capabilities), a Services Catalog Submodel (listing available machine skills), an Interface and Endpoints Submodel (specifying communication methods), and an Execution State and Tracking Submodel (monitoring current progress and status). The ProductionAAS must coordinate with ResourceAAS through FIPA ACL negotiation protocols.

MAS/BDI Specifications (REQ_32 – REQ_34)

The MAS/BDI requirements specify the integration between agent concepts and the AAS structure. Each ResourceAAS must maintain defined belief sets derived from its submodels, representing the agent's knowledge about itself and its environment. Agent desires (objectives) must be defined for each resource, representing the goals that drive agent behavior. Finally, agent intentions (plans) must be specified, defining the procedures the agent executes to achieve its objectives. These requirements ensure that the BDI architecture is systematically integrated with the AAS meta-model.

3.4.2 Non-Functional Requirements (NFR_1 – NFR_5)

Beyond functional capabilities, the system must satisfy quality requirements essential for industrial deployment. The interoperability requirement mandates compliance with IDTA AAS specifications to ensure compatibility with the broader Industry 4.0 ecosystem. Scalability requirements ensure that new ResourceAAS can be added without modifying existing components. Reliability requirements specify

graceful handling of component failures with appropriate error messages. Maintainability requirements call for modular architecture supporting future extensions. Finally, performance requirements specify that agent negotiations must complete within acceptable time bounds for the proof-of-concept demonstration.

3.4.3 Considerations for Production Deployment

The requirements defined above focus on demonstrating the feasibility of Type3AAS in a proof-of-concept context. A production-ready system deployed in an industrial environment would need to address additional requirements that are beyond the scope of this thesis but are acknowledged here for completeness.

Availability and Fault Tolerance. Production systems require high availability, often specified as 99.9% uptime or higher. This necessitates redundant components, failover mechanisms, and the ability to recover from partial system failures without losing production data or state. The current prototype does not implement redundancy or automatic recovery.

Information Security. Industrial systems must protect against unauthorized access, data manipulation, and cyber attacks. This includes authentication and authorization for all AAS access, encrypted communication channels (TLS/SSL), audit logging of all operations, and compliance with standards such as IEC 62443 (Industrial Cybersecurity). The prototype uses unencrypted communication and does not implement access control.

Performance and Scalability. While NFR_5 specifies negotiation completion within 5 seconds for the proof-of-concept, production systems may require sub-second response times for real-time control scenarios. Additionally, the system must scale to hundreds or thousands of resources while maintaining performance. Load testing and performance optimization are not addressed in this work.

Integration with Enterprise Systems. Production deployments typically require integration with Manufacturing Execution Systems (MES), Enterprise Resource Planning (ERP), and other business systems. This includes standardized interfaces, data format transformations, and transaction management.

Regulatory Compliance. Depending on the industry, additional requirements

may arise from regulations such as FDA 21 CFR Part 11 (pharmaceutical), ISO 13485 (medical devices), or automotive quality standards (IATF 16949).

These considerations are discussed further in the outlook (Chapter 7), where future work directions are identified for transitioning the proof-of-concept toward production readiness.

3.5 Requirements Traceability

Table 3.1 provides a traceability matrix linking the thesis contributions to use cases and requirements, ensuring complete coverage of the research objectives.

Table 3.1: Traceability Matrix: Contributions to Requirements

Thesis Contribution	Use Cases	Key Requirements
Bidirectional AAS Integration	UC3, UC4	REQ_5, REQ_14, REQ_32, REQ_34
Foundational Submodels for Proactive Behavior	UC3, UC4	REQ_12, REQ_16–19, REQ_23, REQ_26–30
Proof-of-Concept with Production Resources	UC1–UC5	All requirements

3.6 Summary

This chapter has derived use cases and system requirements for implementing a Type3AAS using BDI-based MAS. Three primary use cases were defined (Startup, Drilling Order, and Shutdown), supported by two included use cases (AAS Registration and Drill Operation) that handle specific sub-processes. From these use cases, 26 functional requirements and 5 non-functional requirements were derived, organized into System Specifications, ResourceAAS Specifications, ProductionAAS Specifications, and MAS/BDI Specifications.

The requirements directly address the research gaps identified in Chapter 2. Bidirectional AAS integration is addressed through requirements for agent-asset communication and the mapping of BDI components to AAS elements. Foundational submodels are specified through detailed submodel requirements for both ResourceAAS

and ProductionAAS. The proof-of-concept is enabled through the complete system specification covering all operational scenarios. The following chapter presents the conceptual design that realizes these requirements.

Chapter 4

Concept

This chapter presents the conceptual design for implementing Type3AAS using BDI-based MAS. The design realizes the requirements specified in Chapter 3 through a layered architecture that integrates standardized AAS components with autonomous agent behavior. Section 4.1 provides an overview of the complete system architecture. Sections 4.2 and 4.3 detail the design of the ResourceAAS and ProductionAAS components respectively. Finally, Section 4.4 describes the communication and interaction mechanisms that enable collaboration between agents.

4.1 System Overview

4.1.1 Architectural Layers

The system architecture follows a two-level design pattern that separates production coordination from resource execution, as illustrated in Figure 4.1. This separation aligns with the industrial agent architecture discussed in Section 2.2.6, where high-level control handles coordination and decision-making while low-level control manages physical asset interaction.

Product Level. The product level contains the ProductionAAS, which is responsible for managing production orders and coordinating the allocation of tasks to available resources. When a drilling order is received, the ProductionAAS queries the AAS Registry to discover available resources, retrieves their capabilities, and

initiates negotiations to select the most suitable resource for the task. The ProductionAAS subscribes to events published by ResourceAAS instances to monitor execution progress and react to status changes.

Resource Level. The resource level contains one or more ResourceAAS instances, each representing a physical drill machine. ResourceAAS components register themselves with the AAS Registry upon initialization, making their capabilities discoverable. During operation, ResourceAAS instances publish status events (started, completed, fault) to the XMPP Broker, enabling real-time monitoring by the ProductionAAS and other system components. When assigned a production task, the ResourceAAS agent autonomously executes the drilling operation by invoking standardized AAS Operations on the physical asset.

4.1.2 System Components

The complete system comprises five main components that work together to realize Type3AAS functionality. Figure 4.2 illustrates the overall system architecture and the relationships between components.

ProductionAAS. The ProductionAAS serves as the coordination hub for production orders. It combines standardized AAS submodels with a BDI agent runtime that provides autonomous decision-making capabilities. The ProductionAAS contains submodels for identification, process planning, service cataloging, interface endpoints, and execution tracking. Its agent component implements orchestration logic for task decomposition and negotiation protocols for resource selection. Communication with other components occurs through the Agent Interface, which translates between AAS data structures and agent beliefs.

ResourceAAS. Each ResourceAAS represents a digital twin of a physical drill machine, integrating AAS standardization with autonomous agent behavior. The ResourceAAS contains submodels describing the asset's operational state, knowledge constraints, capabilities, and interaction parameters. A BDI agent runtime provides goal-directed behavior, enabling the resource to autonomously respond to production requests, participate in negotiations, and execute assigned tasks. The AAS Machine Interface connects the digital representation to the physical asset,

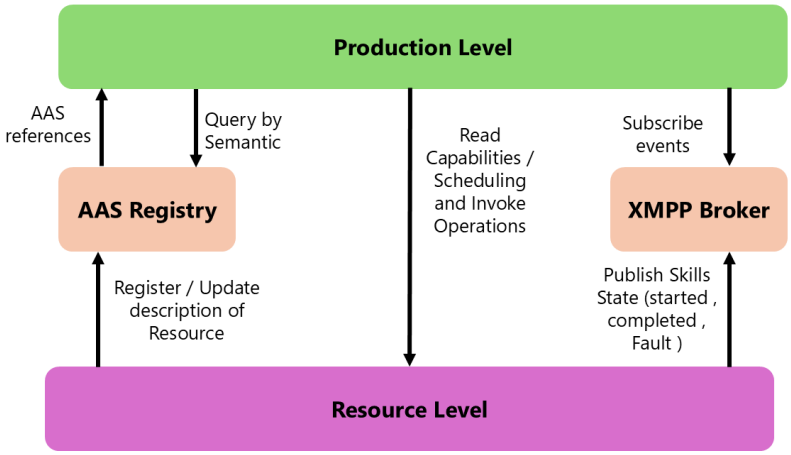


Figure 4.1: Two-Level System Architecture

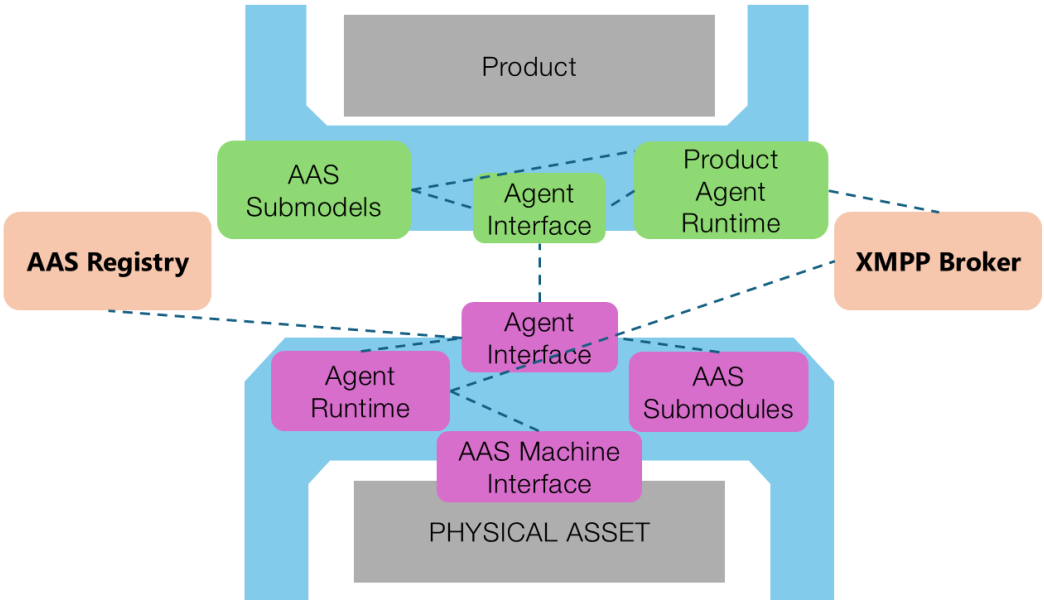


Figure 4.2: SmartDrillSystem Architecture Overview

enabling bidirectional data exchange and operation invocation.

AAS Registry. The AAS Registry provides a centralized discovery service compliant with the AAS specification. Upon initialization, each ResourceAAS registers its identification information, endpoint addresses, and capability descriptions with the registry. The ProductionAAS queries the registry to discover available resources that match required capabilities. This decoupled discovery mechanism enables dynamic system reconfiguration without hardcoded dependencies between components.

XMPP Broker. The XMPP Broker implements the event-driven communication infrastructure using the XMPP protocol. This choice aligns with the SPADE framework used for agent implementation, which natively supports XMPP for agent communication. The broker enables publish-subscribe messaging patterns where ResourceAAS instances publish status events (skill started, completed, or failed) and the ProductionAAS subscribes to receive these updates. This asynchronous communication model supports reactive agent behavior and real-time monitoring.

Physical Assets. The physical drill machines represent the actual manufacturing resources controlled by the system. Each asset connects to its corresponding ResourceAAS through a machine interface that abstracts the underlying communication protocol (such as OPC UA or proprietary interfaces). The ResourceAAS reads sensor data and operational status from the asset and invokes operations to execute drilling tasks.

4.1.3 Component Interactions

Figure 4.3 illustrates the interaction patterns between system components during typical operation.

The system employs four primary interaction patterns:

Registration. During system startup, each ResourceAAS registers itself with the AAS Registry and the XMPP Broker. Registration with the registry enables discovery by the ProductionAAS, while registration with the broker enables event publication and subscription.

Discovery. When the ProductionAAS receives a production order, it queries

the AAS Registry to retrieve references to all registered ResourceAAS instances. The registry returns the endpoint addresses and capability descriptions needed to initiate direct communication with resources.

Direct Communication. The ProductionAAS communicates directly with ResourceAAS instances for operations requiring request-response patterns, such as reading capabilities, scheduling information, and invoking operations. This communication uses the standardized AAS REST API, ensuring interoperability with the AAS specification.

Event-Based Communication. ResourceAAS instances publish status events to the XMPP Broker, and the ProductionAAS subscribes to receive these events. This pattern supports asynchronous notification of state changes without requiring polling, enabling efficient real-time monitoring of distributed resources.

4.1.4 Bidirectional AAS Integration

A key contribution of this design is the bidirectional integration between BDI agents and the AAS standard. Unlike existing approaches that use AAS primarily as a passive data source, this architecture enables agents to both read information from submodels and execute actions through standardized Operations.

Beliefs from Submodels. Agent beliefs are derived from AAS submodel data. The Operational State Submodel provides current asset status, the Knowledge Submodel provides constraints and environmental conditions, and the Capability Submodel provides information about available skills. The Agent Interface continuously synchronizes submodel data with the agent's belief base, ensuring that agent decisions reflect the current system state.

Intentions through Operations. Agent intentions are executed through AAS Operations defined in the submodels. When an agent commits to executing a drilling task, it invokes the corresponding Operation on the physical asset through the standardized AAS API. This approach leverages the Operation element defined in the AAS specification, which supports both synchronous and asynchronous invocation patterns.

This bidirectional integration ensures that the autonomous behavior provided

by the BDI architecture operates within the standardized framework defined by the AAS specification, addressing the interoperability requirements identified in Chapter 2.

4.1.5 Mapping to RAMI4.0

The proposed Type 3 AAS architecture can be positioned within the RAMI4.0 reference model introduced in Chapter 1. Figure 1.1 presents the three-dimensional RAMI4.0 structure consisting of Hierarchy Levels, Life Cycle & Value Stream, and Layers. This subsection maps the system components to each axis.

Hierarchy Levels Axis

The system operates across multiple hierarchy levels as defined by IEC 62264:

Table 4.1: System Components Mapped to RAMI4.0 Hierarchy Levels

Hierarchy Level	Component	Role
Work Centers	Production Agent	Coordinates production orders, orchestrates resources
Station	Resource Agent	Manages individual drill machine operations
Field Device	Simulators / PLCs	Executes physical operations (drill, move)

The Production Agent operates at the *Work Centers* level, receiving production orders and coordinating their execution across available resources. Resource Agents operate at the *Station* level, each representing a single manufacturing resource (drill machine). The simulators (or physical PLCs in a production deployment) represent the *Field Device* level, executing the actual machining operations.

Life Cycle & Value Stream Axis

The RAMI4.0 life cycle axis distinguishes between *Type* (design-time definitions) and *Instance* (runtime representations):

The submodel structures (Capability, Knowledge, Process Plan) represent *Type* information reusable templates that define what a resource can do and how processes

Table 4.2: System Elements Mapped to RAMI4.0 Life Cycle Phases

Phase	Type (Design)	Instance (Runtime)
Development	Submodel templates, Capability definitions	–
Maintenance	Process Plan structure, Knowledge constraints	–
Production	–	AAS instances, Agent beliefs, Execution state

are structured. At runtime, these become *Instance* data: concrete AAS instances with actual parameter values, agent beliefs reflecting current state, and execution tracking data that changes during operation.

Layer Axis

The six RAMI4.0 layers describe different aspects of an I4.0 component. Table 4.3 maps the system implementation to each layer:

Table 4.3: System Implementation Mapped to RAMI4.0 Layers

Layer	Implementation
Business	Production orders (User Requests) triggering manufacturing processes
Functional	BDI agent logic: goal-directed reasoning, plan execution, CNP negotiation
Information	AAS submodels: Capability, Knowledge, Process Plan, Execution Tracking
Communication	XMPP PubSub (agent messaging), REST API (AAS access), HTTP (skill invocation)
Integration	ResourceAASClient, invocationDelegation pattern, BaSyx SDK
Asset	Drill machine simulators (proof-of-concept) / Physical PLCs (production)

Business Layer. Production orders represent business-level requests that initiate the manufacturing process. The User Request message defines what capability is needed and when.

Functional Layer. The BDI agent architecture provides the functional logic. Agents reason about their beliefs (from submodels), form desires (respond to CFP,

execute skills), and commit to intentions (negotiate, invoke operations). This layer realizes the "proactive" behavior that distinguishes Type 3 from Type 1/2 AAS.

Information Layer. The AAS submodels provide the standardized information model. All data exchanged between components conforms to the AAS meta-model, ensuring interoperability with other I4.0-compliant systems.

Communication Layer. Three protocols handle different communication needs: XMPP PubSub for asynchronous agent messaging and event notification, the AAS REST API for submodel access, and HTTP for skill invocation via the invocation-Delegation pattern.

Integration Layer. The ResourceAASClient and ProductionAASClient classes bridge the gap between the AAS server and the agent runtime. The invocationDelegation qualifier pattern connects AAS Operations to external execution endpoints.

Asset Layer. The physical drill machines (represented by simulators in the proof-of-concept) form the asset layer. These execute the actual manufacturing operations when invoked through the integration layer.

Type 3 AAS in the RAMI4.0 Context

The key contribution of this work within the RAMI4.0 framework is the enhancement of the *Functional Layer*. While Type 1 and Type 2 AAS implementations primarily address the Information, Communication, and Integration layers, the Type 3 AAS adds autonomous decision-making capabilities at the Functional Layer through BDI agent integration.

This positions the Type 3 AAS as an enabler for the RAMI4.0 vision of intelligent, self-organizing production systems where I4.0 components can autonomously coordinate without centralized control.

4.2 Resource Agent Concept

The ResourceAAS represents the core component for integrating autonomous agent behavior with standardized AAS representations of manufacturing resources. This section presents the conceptual design of the ResourceAAS, describing its overall

architecture, submodel structure, agent core, and the mechanisms that enable bidirectional integration between the BDI agent and the AAS.

4.2.1 Resource Agent Overview

The ResourceAAS combines three main architectural elements: the AAS structure providing standardized data representation, the BDI agent core enabling autonomous behavior, and the interfaces connecting these components to each other and to external systems. Figure 4.4 illustrates the high-level component structure of the ResourceAAS.

Asset Administration Shell. The AAS component provides the standardized digital representation of the drill machine using the Eclipse BaSyx Python SDK. It contains the shell information (identification, asset kind, global asset ID) and a collection of submodels that describe different aspects of the resource. The AAS is deployed to a BaSyx AAS Environment server, exposing a REST API that enables external access to submodel data and operation invocation, compliant with the AAS API specification defined in Part 2 of the standard.

BDI Core. The BDI core implements the agent's autonomous behavior using the SPADE framework combined with the spade_bdi extension. SPADE (Smart Python Agent Development Environment) provides asynchronous agent communication over XMPP, while spade_bdi adds AgentSpeak-based BDI reasoning capabilities. The agent maintains beliefs derived from submodel data, processes desires triggered by external events (such as Call-For-Proposals), and executes intentions through SPADE behaviors. Agent plans are defined in AgentSpeak Language (ASL) files that specify reactive rules for handling events and achieving goals.

AAS-Agent Interface (ResourceAASClient). The ResourceAASClient serves as the translation layer between the AAS REST API and the agent's internal representation. This component implements an asynchronous HTTP client using aiohttp that communicates with the BaSyx server via GET and POST requests. It provides repository classes for reading submodel elements (SubmodelElementRepositoryGetters), updating values (SubmodelElementRepositoryUpdate), and invoking operations. The interface translates AAS data structures into Python dictionaries that

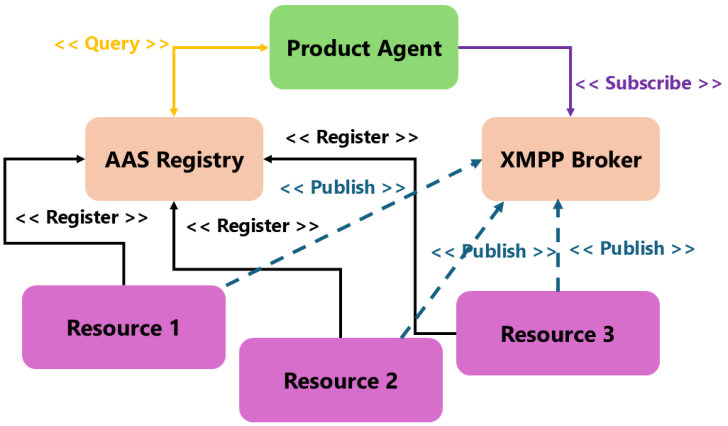


Figure 4.3: Component Interaction Patterns

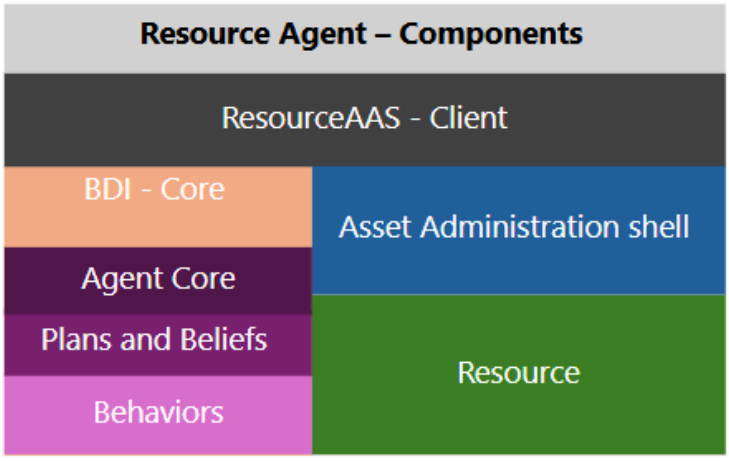


Figure 4.4: Resource Agent Component Structure

can be converted to agent beliefs.

Operation Invokers. The physical asset operations are exposed through FastAPI-based HTTP endpoints that implement the invocationDelegation pattern specified in the AAS Operation qualifiers. When the BaSyx server receives an operation invocation request, it delegates the call to the corresponding invoker endpoint (e.g., `/drill_invocation_1`), which executes the operation on the physical asset or simulator and returns the result.

Figure 4.5 illustrates the internal connections and data flows within the ResourceAAS architecture.

The architecture supports three primary data flows:

Belief Acquisition. During agent initialization, the AgentInitializationbehavior reads submodel data via the ResourceAASClient and populates the agent's belief base. Beliefs include: `current_state` from the Operational State Submodel, `supported_skills` from the Capabilities Submodel, `skills_constraints` from the Knowledge Submodel, and `free_time_slots` / `booked_time_slots` from the Operational State time slot collection.

Event Processing. When the agent receives a Call-For-Proposal (CFP) from the ProductionAAS via XMPP publish-subscribe, the CFPProcessingbehavior validates the request against current beliefs. It checks whether the resource state allows execution, whether the requested skill is supported, and whether input parameters fall within the constraints defined in the Knowledge Submodel.

Operation Execution. When the agent commits to executing a task, the corresponding invoker behavior (e.g., DrillInvokerbehavior) retrieves input parameters from beliefs, calls the operation through the ResourceAASClient, and updates beliefs with the result. The operation invocation follows the AAS standard, passing through the BaSyx server to the delegated FastAPI endpoint.

4.2.2 Resource Agent Submodels

The ResourceAAS contains four submodels that provide the standardized data structures supporting agent operation. These submodels are implemented using the Eclipse BaSyx Python SDK and are designed to map directly to the BDI com-

ponents. The following subsections describe each submodel based on the actual implementation.

Operational State Submodel

The Operational State Submodel captures the current runtime status and scheduling information of the resource. It provides the primary source for agent beliefs about the asset's availability and current condition. Figure 4.6 shows the structure of this submodel.

Current_Operational_State. This Property element tracks the real-time operational status of the drill machine. The value contains the current state as a string ("Idle", "Running", "Error", or "Done"), with the category set to "VARIABLE" indicating runtime mutability. During CFP processing, the agent checks this belief to ensure the resource is in an allowed state ("Idle" or "Free") before accepting new tasks.

TimeSlots_Operational_Data. This SubmodelElementCollection manages the resource's scheduling information through three properties:

- `Free_Slots`: JSON-encoded list of available time slots (e.g., "08:00", "08:30", "09:00")
- `Booked_Slots`: JSON-encoded list of already allocated time slots
- `Slot_Duration`: Integer value representing the duration of each slot in minutes

The TimeSlotDataType class manages slot allocation and release, ensuring that negotiation decisions are based on actual availability.

Knowledge Submodel

The Knowledge Submodel contains constraint information that defines the operational boundaries of the resource. This information is used during CFP validation to determine whether requested parameters are feasible. Figure 4.7 shows the structure of this submodel.

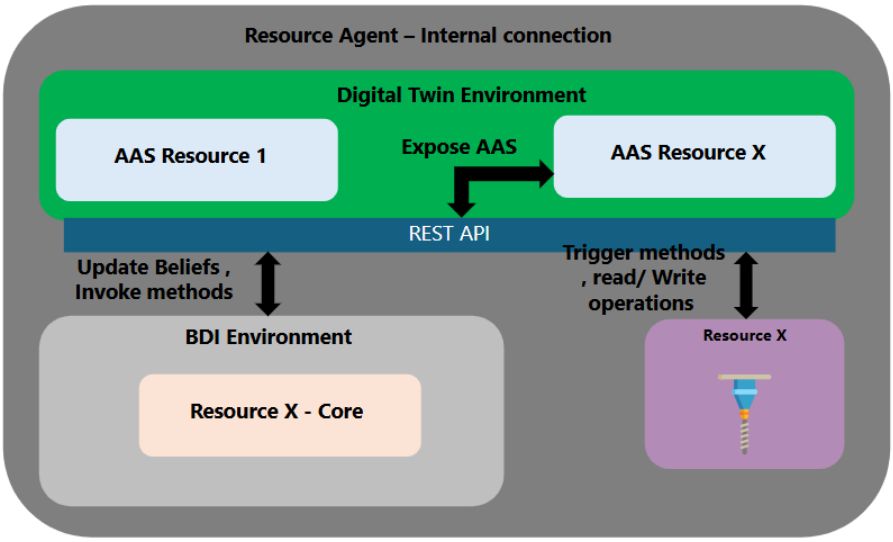


Figure 4.5: Resource Agent Internal Architecture

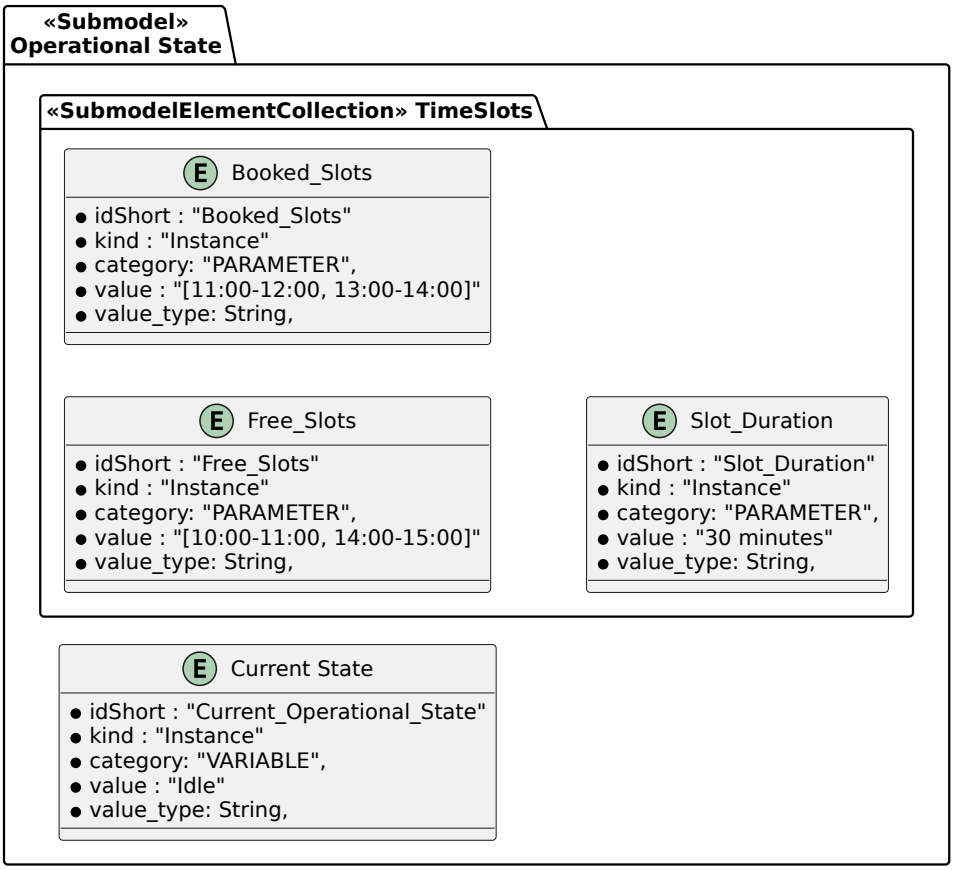


Figure 4.6: Operational State Submodel Structure

Knowledge Submodel

The Knowledge Submodel contains constraint information that defines the operational boundaries of the resource. This information is used during CFP validation to determine whether requested parameters are feasible. Figure 4.7 shows the structure of this submodel.

Resource_Constraints. This SubmodelElementCollection contains nested collections for each constraint type:

- `MaxDepth`: Collection with MaxDepthValue (Double, e.g., 20.0), MaxDepthUnit ("centimeters"), and MaxDepthUnitShort ("cm")
- `MinDepth`: Collection with MinDepthValue (Double, e.g., 1.0) and corresponding unit properties
- `ConstraintType`: Property indicating the parameter being constrained ("Depth")

During CFP processing, the agent retrieves the constraint type and min/max values, then validates that the requested input parameter falls within the allowed range. If validation fails, a `CONSTRAINT_VIOLATION` is added to the negotiation response.

Capability Submodel

The Capability Submodel describes the skills and operations that the resource can perform. This submodel directly relates to agent capabilities and contains the Operation elements that enable bidirectional integration. Figure 4.8 shows the structure of this submodel.

Supported_Skills. This Property lists the capabilities supported by the resource as a comma-separated string (e.g., "drill_capability,movexy_capability"). The ProductionAAS uses this information during resource discovery to filter resources that can fulfill requested operations.

Drill_Capability. This Operation element defines the drilling capability with:

- `invocationDelegation` qualifier: URL of the FastAPI endpoint

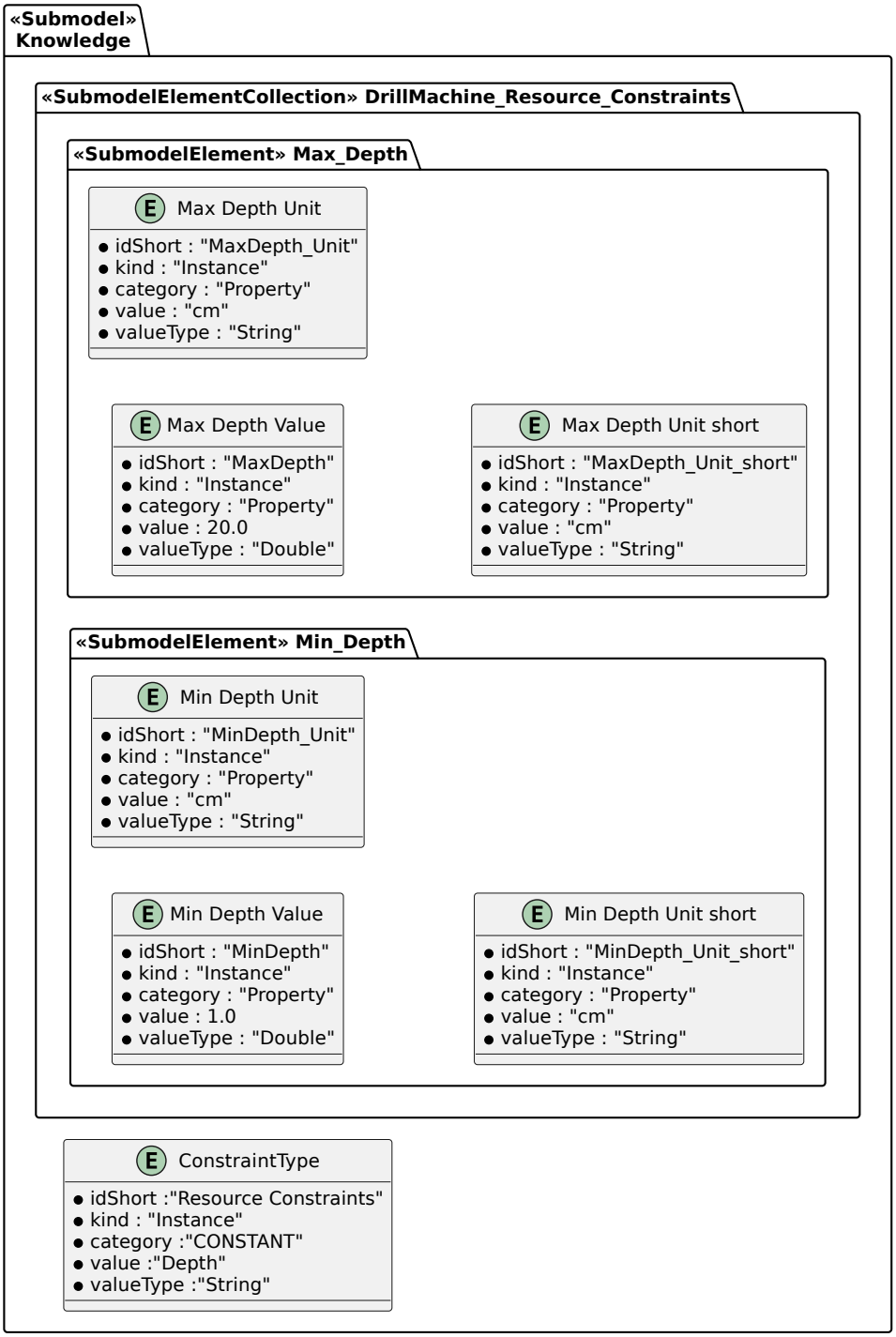


Figure 4.7: Knowledge Submodel Structure

- Input variables: Drill_Depth (Double) and Drill_Speed (Double)
- Output variable: Drill_Result (String)

MoveXY_Capability. This Operation element defines the positioning capability with:

- `invocationDelegation` qualifier: URL of the movement endpoint
- Input variables: Target_X (Double) and Target_Y (Double)
- Output variable: Move_Result (String)

The invocationDelegation pattern enables the BaSyx server to forward operation requests to external services, supporting integration with physical assets or simulators without modifying the AAS infrastructure.

Interaction and Scheduling Submodel

The Interaction and Scheduling Submodel provides the communication endpoints through which external systems can interact with the ResourceAAS. Figure 4.9 shows the structure of this submodel.

Endpoints. This SubmodelElementCollection contains properties for different communication protocols:

- `Machine_OPCUA_Endpoint`: OPC UA server address for direct asset communication (placeholder for future integration)
- `MQTT_Broker_Endpoint`: MQTT broker address for event-based communication

The ProductionAAS retrieves these endpoints during the discovery phase to establish appropriate communication channels for negotiation and task assignment.

4.2.3 BDI-Submodel Integration

This section explicitly describes how the BDI architecture manifests in the Type 3 AAS implementation. The title of this thesis references BDI agents, and this section

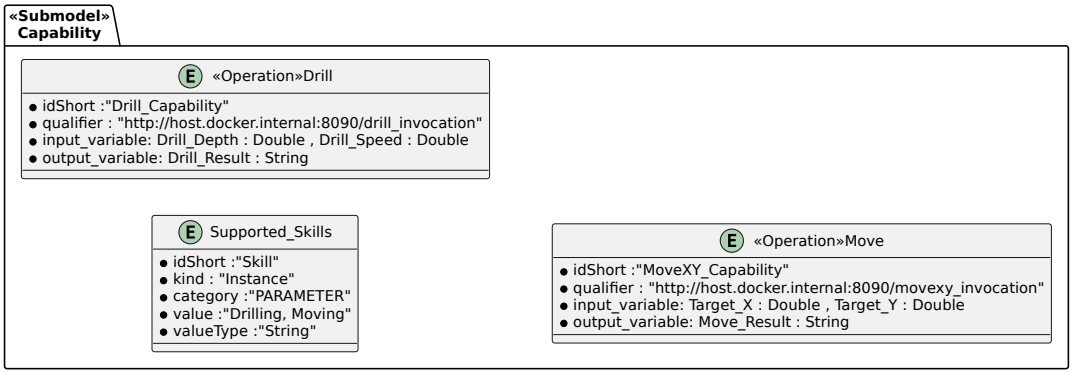


Figure 4.8: Capability Submodel Structure

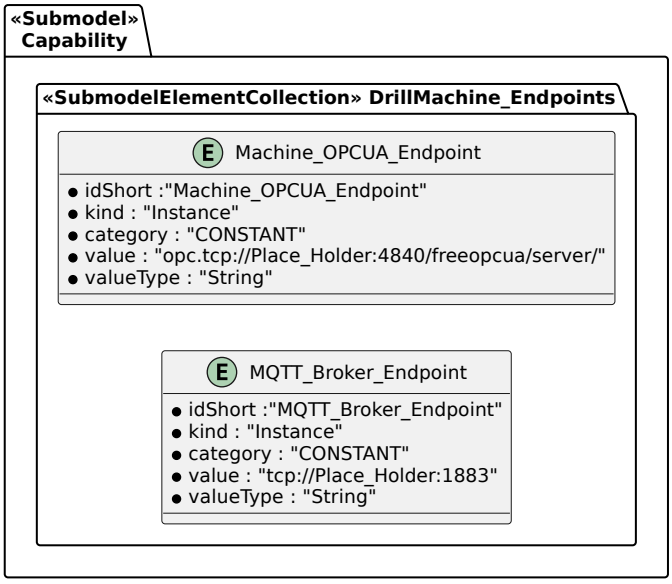


Figure 4.9: Interaction and Scheduling Submodel Structure

clarifies how Beliefs, Desires, and Intentions are concretely realized through the AAS submodels and AgentSpeak plans.

Conceptual Mapping

The BDI model defines three mental attitudes that govern agent behavior:

- **Beliefs**: The agent's knowledge about itself, other agents, and the environment
- **Desires**: The agent's objectives or goals it wishes to achieve
- **Intentions**: The agent's committed plans for achieving its goals

In the Type 3 AAS architecture, these mental attitudes are distributed between the AAS submodels (persistent, shared state) and the AgentSpeak plans (reactive behavior). Table 4.4 provides the concrete mapping.

Beliefs: AAS Submodels as Knowledge Base

The agent's beliefs are loaded from AAS submodels during initialization via the `AgentInitializationbehavior`. This behavior reads submodel data and converts it to agent beliefs:

Listing 4.1: Belief Loading from AAS Submodels

```
class AgentInitializationbehavior(OneShotbehavior):
    async def run(self):
        # Load beliefs from AAS submodels concurrently
        current_state, skills, constraints, ... = await
            ↪ asyncio.gather(
            self.agent.resource_client.getvalue_Current_State
                ↪ (),
            self.agent.resource_client.
                ↪ getvalue_Supported_Skills(),
            self.agent.resource_client.getvalue_MaxMinDepth()
                ↪ ,
```

```
            self.agent.resource_client.getvalue_Free_Slots(),
            ...
        )
        # Set beliefs in BDI agent
        self.agent.bdi.set_belief("current_state",
            ↪ current_state)
        self.agent.bdi.set_belief("supported_skills", skills)
        self.agent.bdi.set_belief("skills_constraints",
            ↪ constraints)
        self.agent.bdi.set_belief("free_time_slots",
            ↪ free_time_slots)
```

Table 4.5 shows the mapping between AAS submodel elements and agent beliefs:

Runtime Beliefs. In addition to persistent beliefs from submodels, the agent maintains runtime beliefs that change during execution:

- `is_cfp_received(true/false)`: Set when a CFP message arrives
- `cfp_skill(Skill)`: The requested capability from the CFP
- `cfp_at_time(TimeSlot)`: The requested time slot
- `current_operation(drill/movexy)`: Tracks which operation is executing
- `drill_completed(true)`, `move_xy_completed(true)`: Operation completion flags

Desires: Goals in AgentSpeak

Desires represent what the agent wants to achieve. In the implementation, desires are expressed through AgentSpeak achievement goals and event-triggered responses:

Initialization Goal. The agent starts with an explicit goal:

Listing 4.2: Initial Goal in AgentSpeak

```
!setup.

+!setup <-
```

Table 4.4: BDI to AAS-Agent Mapping

BDI Element	AAS/Agent Component	Concrete Example
Beliefs	Operational State Submodel	`current_state`, `free_time_slots`
	Knowledge Submodel	`skills_constraints`, `skills_constraints_types`
	Capability Submodel	`supported_skills`
	Runtime Beliefs	`is_cfp_received`, `cfp_skill`, `cfp_at_time`
Desires	AgentSpeak Goals	`!setup` (initialization goal)
	Event-triggered Goals	Respond to CFP, execute skills
	Process Plan	Reach `Exit_Point` from `Entry_Point`
Intentions	AgentSpeak Plans	Plans triggered by belief additions
	Custom Actions	`.initialize_agent`, `.process_cfp`, `.drill`
	SPADE behaviors	`DrillInvokerbehavior`, `MoveXYInvokerbehavior`

Table 4.5: AAS Submodel Elements to Agent Beliefs

Submodel	Element	Belief Predicate
Operational State	Current_State	`current_state(State)`
Operational State	Free_Slots	`free_time_slots(Slots)`
Operational State	Booked_Slots	`booked_time_slots(Slots)`
Capability	Supported_Skills	`supported_skills(Skills)`
Knowledge	Constraints	`skills_constraints(Values)`
Knowledge	Constraints_Type	`skills_constraints_types(Types)`

```
  .print("Agent␣started␣:␣Drilling␣Machine␣1");
  .initialize_agent;
  .check_request.
```

Event-Triggered Desires. Most desires emerge from belief changes rather than explicit goal statements. When a belief is added, it triggers a plan that implicitly represents a desire:

Listing 4.3: Desire: Respond to CFP

```
// Desire: When CFP received, respond appropriately
+is_cfp_received(true) <-
  -is_cfp_received(true);
  .print("EVENT:␣CFP␣received,␣processing␣Time␣slots");
  .process_time_slots;
  .process_cfp.
```

Listing 4.4: Desire: Execute Skills

```
// Desire: When selected, execute the requested skills
+excute_skills(drill_capability) <-
  -excute_skills(drill_capability);
  +current_operation(drill);
  .print("Drilling␣-␣starting␣move_xy");
  .move_xy.
```

Intentions: Committed Plans and Actions

Intentions are the plans the agent commits to execute. They bridge desires to concrete actions through custom actions and SPADE behaviors:

Custom Actions. The `ResourceAgent` class registers custom actions that map AgentSpeak action calls to SPADE behaviors:

Listing 4.5: Custom Action Registration

```
def add_custom_actions(self, actions):
    @actions.add(".initialize_agent", 0)
    def _initialize_agent(agent, term, intention):
```

```
        agent_init_behavior = AgentInitializationbehavior()
        self.add_behavior(agent_init_behavior)
        yield

    @actions.add(".drill", 0)
    def _drill(agent, term, intention):
        drill_behavior = DrillInvokerbehavior()
        self.add_behavior(drill_behavior)
        yield

    @actions.add(".move_xy", 0)
    def _move_xy(agent, term, intention):
        move_xy_behavior = MoveXYInvokerbehavior()
        self.add_behavior(move_xy_behavior)
        yield
```

Intention Execution Flow. When an AgentSpeak plan executes an action like `.drill`, the following occurs:

1. AgentSpeak interpreter calls the registered custom action
2. Custom action instantiates the corresponding SPADE behavior
3. behavior executes asynchronously, invoking the AAS Operation
4. Upon completion, behavior sets a belief (e.g., `drill_completed(true)`)
5. Belief change triggers the next plan in the sequence

Context-Dependent Plans. AgentSpeak supports context conditions that select different plans based on current beliefs:

Listing 4.6: Context-Dependent Intention Selection

```
// When move completes during drill operation -> continue to
   ↪ drill
+move_xy_completed(true) : current_operation(drill) <-
  -move_xy_completed(true);
```

```
  .print("Move completed, now drilling");
  .drill.

// When move completes during movexy-only operation -> done
+move_xy_completed(true) : current_operation(movexy) <-
  -move_xy_completed(true);
  -current_operation(movexy);
  .print("Move completed, operation done");
  .operation_done.
```

This demonstrates how the agent's current beliefs (context) determine which intention is formed, a key characteristic of BDI reasoning.

BDI Reasoning Cycle

Figure 4.10 illustrates how the BDI reasoning cycle operates within the Type 3 AAS:

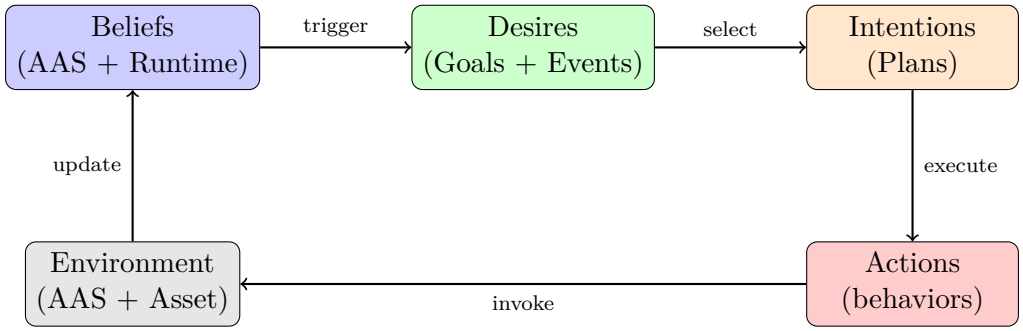


Figure 4.10: BDI Reasoning Cycle in Type 3 AAS

1. **Beliefs**: Agent loads beliefs from AAS submodels and receives runtime belief updates
2. **Trigger**: Belief additions trigger AgentSpeak plan selection
3. **Select**: Agent selects applicable plan based on context conditions
4. **Execute**: Plan body executes, calling custom actions
5. **Invoke**: Custom actions invoke SPADE behaviors that call AAS Operations
6. **Update**: Operation results update beliefs, continuing the cycle

Example: Complete CFP Response Cycle

To illustrate the BDI cycle, consider how a Resource Agent responds to a Call-For-Proposal:

1. **Belief**: `ProductionRequestbehavior` receives CFP message, sets `is_cfp_received(true)`
2. **Plan Triggered**: AgentSpeak matches plan for `+is_cfp_received(true)`
3. **Intention**: Agent commits to plan body:

   ```
   .process_time_slots;
   .process_cfp.
   ```

4. **Action**: `.process_time_slots` → `TimeSlotProcessingbehavior`
   - Checks `free_time_slots` belief against requested time
   - Sets `is_negotiation_message_ready(true)`
5. **Next Plan**: `+is_negotiation_message_ready(true)` triggers response plan
6. **Action**: `.inform_production_agent` → `InformProductionAgent`
   - Publishes proposal to XMPP topic
7. **Belief Update**: Production Agent's decision arrives, setting either:
   - `is_negotiation_response_positive(true)` → prepare execution
   - `is_negotiation_response_positive(false)` → return to waiting

This cycle demonstrates how BDI mental attitudes beliefs from AAS submodels, desires expressed as reactive plans, and intentions executed through behaviors combine to produce autonomous, goal-directed behavior in the Type 3 AAS.

4.2.4 Resource Agent Repositories

The communication between the BDI agent and the AAS server requires a structured data access layer that abstracts the complexity of REST API interactions. The Repository design pattern [49] provides this abstraction, enabling clean separation between agent logic and AAS communication. Figure 4.11 illustrates the repository class hierarchy.

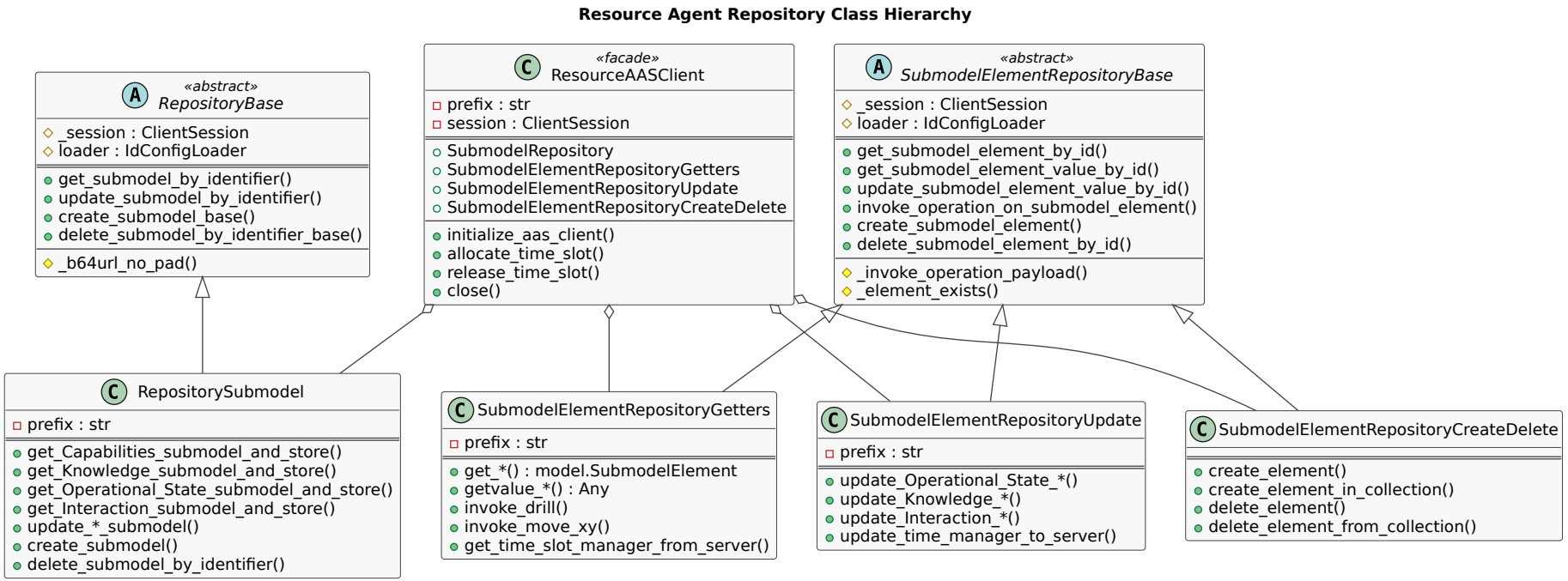


Figure 4.11: Resource Agent Repository Class Hierarchy

Base Classes

RepositoryBase serves as the abstract foundation for Submodel-level operations. It encapsulates HTTP session management using aiohttp, Base64 URL-safe encoding for AAS identifiers (as required by the BaSyx API), and JSON serialization using BaSyx SDK encoders. This class provides generic CRUD operations for retrieving, updating, creating, and deleting entire Submodels.

SubmodelElementRepositoryBase extends the base functionality for SubmodelElement-level operations. It handles path-based element access using dot notation for nested collections (e.g., `Collection.Property`), operation invocation with automatic payload construction, and idempotent element creation with existence checks.

Specialized Repositories

RepositorySubmodel inherits from RepositoryBase and provides Submodel-specific methods for each of the four ResourceAAS submodels. It includes a prefix injection mechanism that enables support for multiple resource instances (e.g., Drill_1, Drill_2) by dynamically modifying submodel identifiers.

SubmodelElementRepositoryGetters provides read-only access to submodel elements. It follows a dual naming convention: `get_*()` methods return complete BaSyx model objects, while `getvalue_*()` methods return only the element values. This class also provides operation invocation methods (`invoke_drill()`, `invoke_move_xy()`) and utility methods like `get_time_slot_manager_from_server()` for complex data retrieval.

SubmodelElementRepositoryUpdate provides write operations for modifying submodel element values. It mirrors the getter structure with update methods for each element type across all submodels.

SubmodelElementRepositoryCreateDelete handles element lifecycle management, supporting dynamic addition and removal of submodel elements during runtime.

ResourceAASClient acts as a facade that aggregates all repository instances. It is initialized with a resource prefix and provides a single entry point for agent behaviors to access AAS data:

```
class ResourceAASClient:
    def __init__(self, prefix: str = ""):
        self.prefix = prefix

    async def initialize_aas_client(self):
        self.session = aiohttp.ClientSession()
        self.SubmodelRepository = RepositorySubmodel(self.session, self.prefix)
        self.SubmodelElementRepositoryGetters = SubmodelElementRepositoryGetters(
            self.session, self.prefix)
        self.SubmodelElementRepositoryUpdate = SubmodelElementRepositoryUpdate(
            self.session, self.prefix)
```

This layered architecture ensures that the agent core remains decoupled from REST API details, facilitating testing and maintenance.

4.2.5 Resource Agent Core

The Resource Agent Core implements the BDI reasoning cycle that enables autonomous behavior. This subsection describes the agent class structure, the AgentSpeak plans, and the SPADE behaviors that execute agent actions. Figure 4.12 illustrates the class architecture.

ResourceAgent Class

The `ResourceAgent` class extends both `BDIAgent` and `PubSubMixin` from the SPADE framework. This dual inheritance combines AgentSpeak-based reasoning with XMPP publish-subscribe messaging. The agent maintains a reference to the `ResourceAASClient` for AAS communication and registers custom actions that map AgentSpeak plan actions to SPADE behaviors.

Custom actions are registered in the `add_custom_actions()` method. Each action corresponds to a specific behavior class. For example, when an AgentSpeak plan executes `.drill`, the agent instantiates and adds a `DrillInvokerbehavior` to its execution queue.

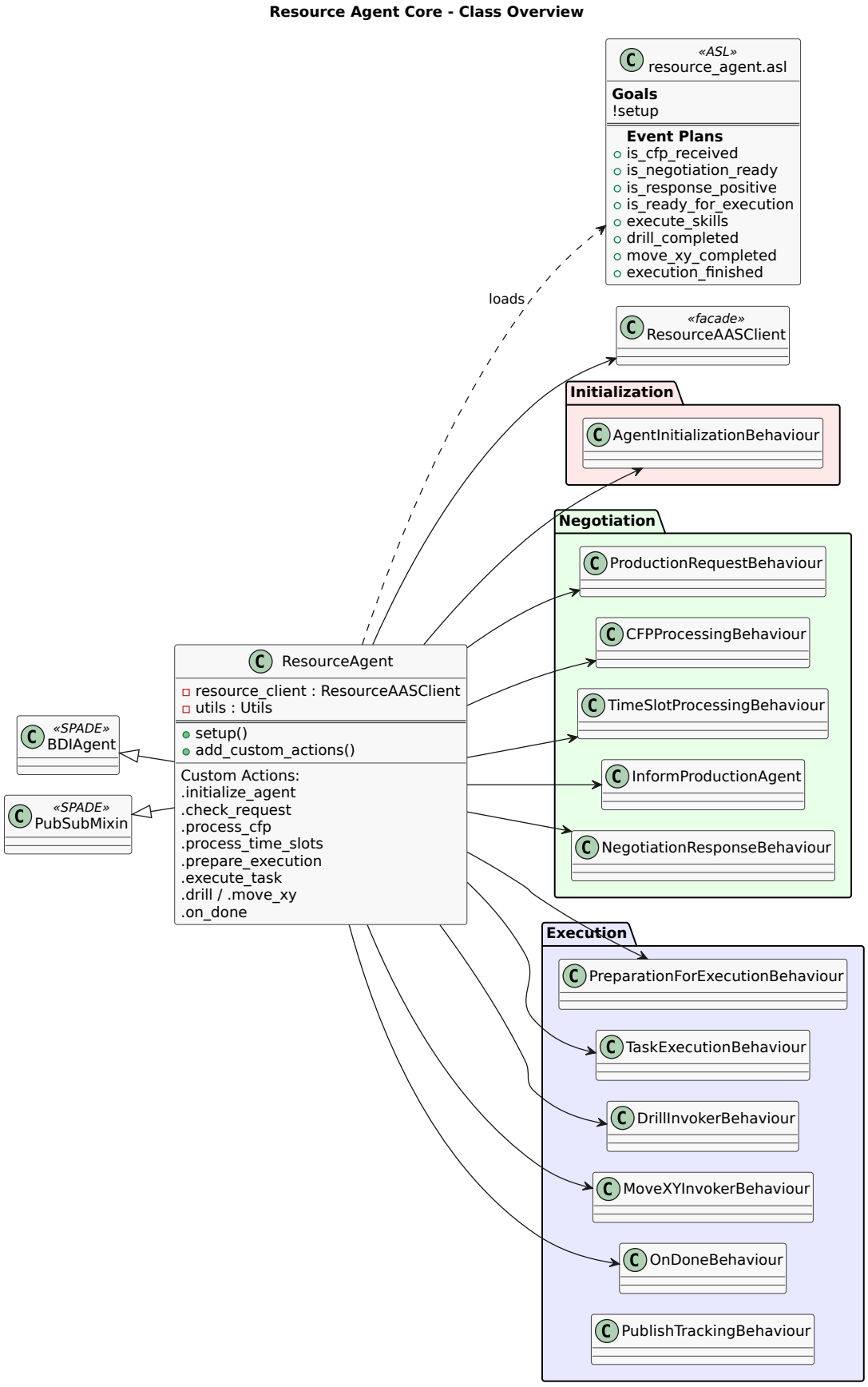


Figure 4.12: Resource Agent Core Class Architecture

AgentSpeak Plans

The agent's reactive behavior is defined in the `resource_agent.asl` file using AgentSpeak syntax. Plans follow the pattern `+trigger : context <- body`, where the trigger is a belief change, the context is an optional condition, and the body contains actions to execute.

Initialization. The agent starts with the goal `!setup`, which triggers belief loading from AAS submodels and initiates request listening.

Negotiation Plans. When a Call-For-Proposal arrives, the belief `is_cfp_received(true)` triggers validation plans that check time slots, current state, and skill constraints. The result is published back to the ProductionAAS.

Execution Plans. Upon receiving a positive negotiation response, execution plans coordinate the operation sequence. For drilling tasks, the agent first executes `move_xy`, then upon completion triggers `drill`. Context conditions ensure correct sequencing:

```
+move_xy_completed(true) : current_operation(drill) <-
    .drill.
```

SPADE behaviors

behaviors implement the actual logic for each agent action. They are organized into three categories:

Initialization behaviors. `AgentInitializationbehavior` reads beliefs from all four submodels concurrently using `asyncio.gather()`, populating the agent's belief base with current state, supported skills, constraints, and time slot availability.

Negotiation behaviors. These behaviors handle the Contract Net Protocol interaction:

- `ProductionRequestbehavior`: Listens for CFP messages via XMPP PubSub
- `TimeSlotProcessingbehavior`: Validates requested time slot availability
- `CFPProcessingbehavior`: Validates state, skills, and constraints against the CFP

- `InformProductionAgent`: Sends the negotiation response with any violations
- `NegotiationResponsebehavior`: Processes the ProductionAAS decision

Execution behaviors. These behaviors execute the actual operations:

- `PreparationForExecutionbehavior`: Prepares the resource (e.g., allocates time slot)
- `TaskExecutionbehavior`: Determines which skill to execute based on the CFP
- `DrillInvokerbehavior`: Invokes the drill operation via the AAS API
- `MoveXYInvokerbehavior`: Invokes the positioning operation
- `PublishTrackingbehavior`: Publishes execution status events
- `OnDonebehavior`: Performs cleanup and releases allocated resources

Agent State Machine

Figure 4.13 shows the complete state machine of the ResourceAgent, illustrating the flow from initialization through negotiation to execution.

The state machine comprises six main phases: initialization (loading beliefs from submodels), waiting (listening for CFP requests), CFP processing (validating the request), negotiation response handling (waiting for ProductionAAS decision), execution (performing the requested operation), and cleanup (releasing resources and returning to waiting state).

4.2.6 Resource Simulators

To enable testing and validation of the ResourceAAS without physical hardware, the system includes discrete-event simulators that emulate the behavior of drill machines and positioning systems. These simulators implement the same HTTP endpoints as real machine controllers, allowing seamless integration with the AAS infrastructure via the invocationDelegation pattern.

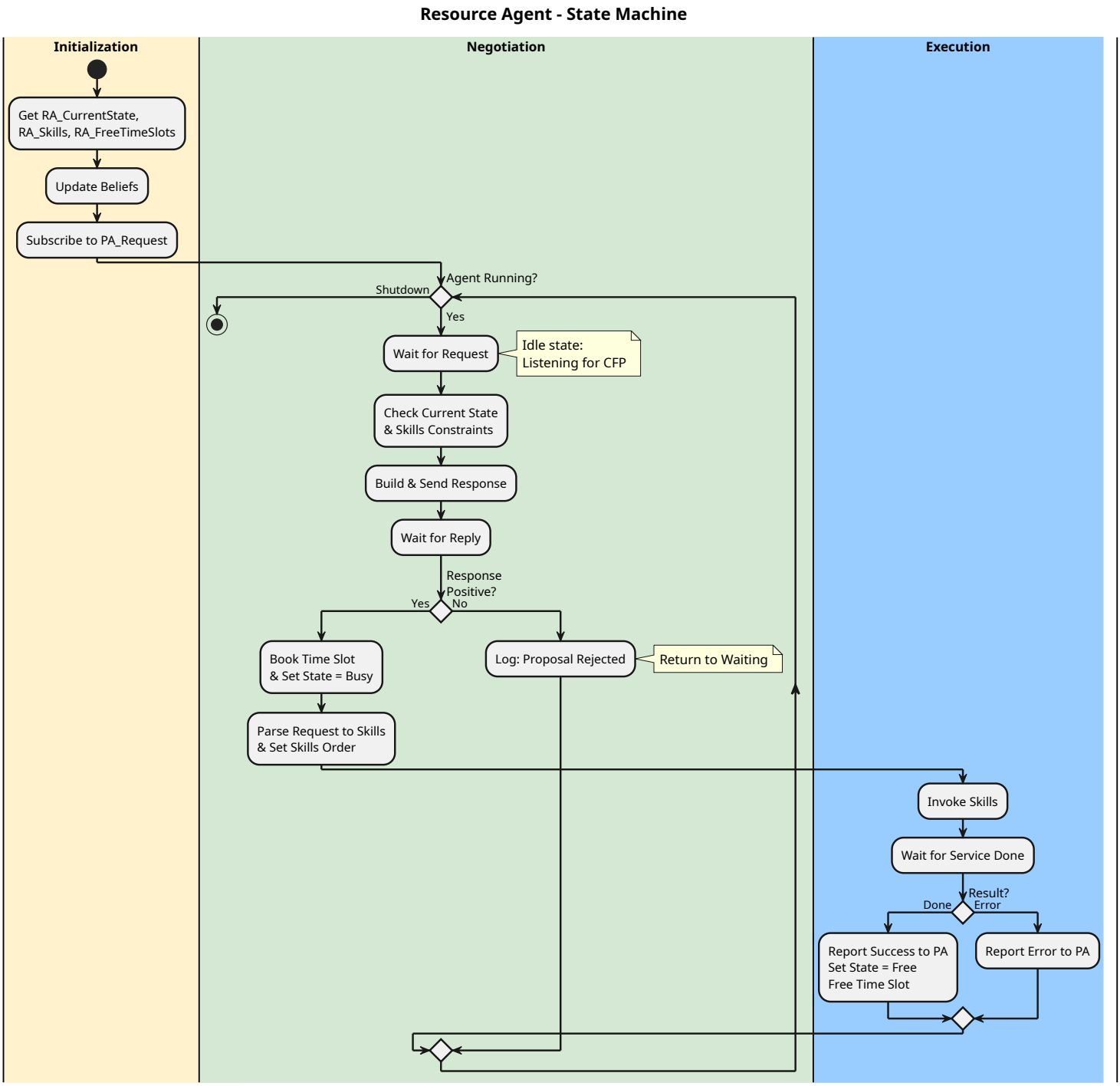


Figure 4.13: Resource Agent State Machine

Simulator Architecture

Each simulator is implemented as a standalone FastAPI application that combines discrete-event simulation with asynchronous HTTP handling. The architecture uses SimPy [50], a process-based discrete-event simulation framework for Python, to model machine operations with realistic timing behavior. Figure 4.14 illustrates the simulator integration.

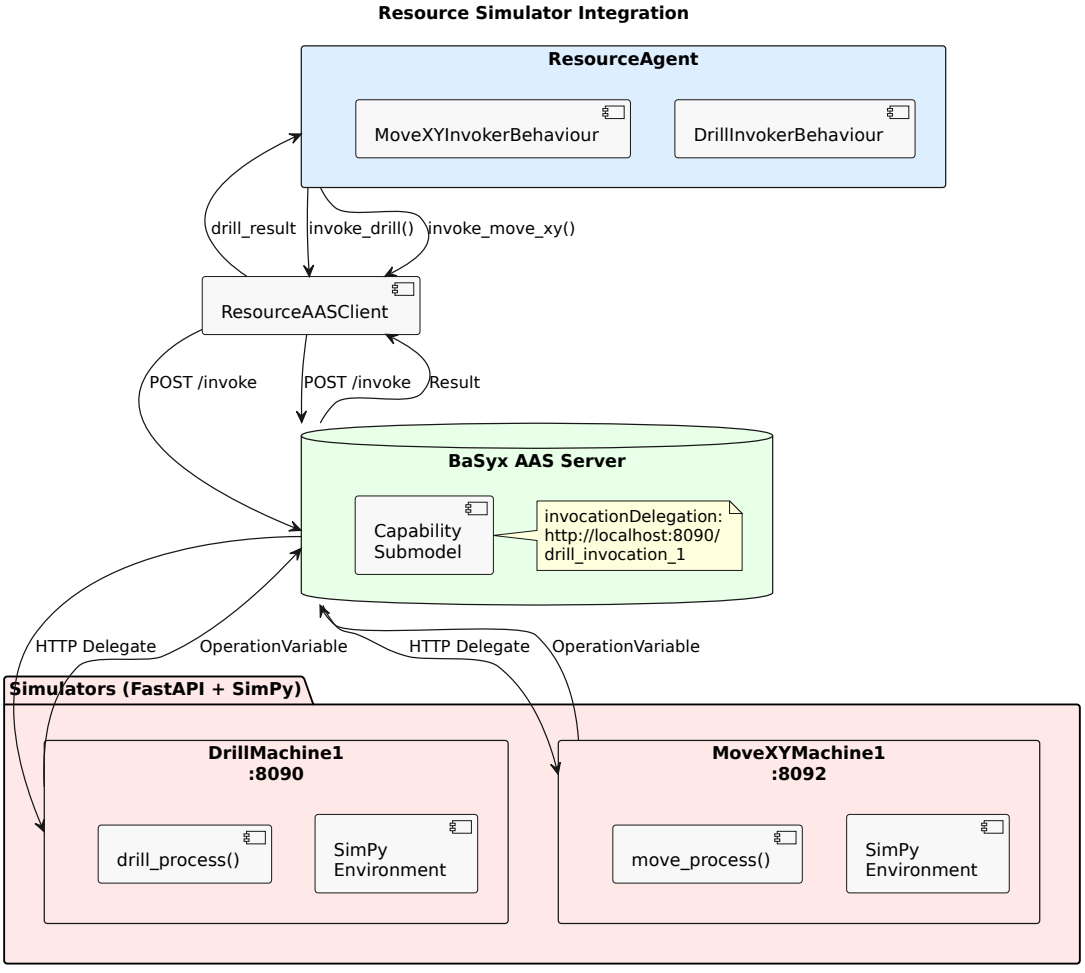


Figure 4.14: Resource Simulator Integration Architecture

The simulators expose HTTP endpoints that match the invocationDelegation URLs defined in the Capability Submodel Operations. When the BaSyx server receives an operation invocation request, it forwards the call to the simulator endpoint, which executes the simulated operation and returns AAS-compliant results.

Drill Machine Simulator

The `DrillMachine` simulator models drilling operations with configurable speed factors and setup times. Key components include:

Machine State. The simulator maintains state information including current operational status (Idle/Drilling), last drill depth, jobs completed, and cumulative drill time.

Drill Process. The SimPy process calculates drill duration based on depth and RPM parameters using a simple simulation formula:

$$t_{drill} = t_{setup} + depth \times factor_{speed} \times \frac{1500}{RPM} \tag{4.1}$$

This formula scales drill time linearly with depth and inversely with RPM, plus a fixed setup time. It is not based on real machining physics but provides a reasonable approximation for simulation purposes, where higher RPM values result in faster drilling and deeper holes require proportionally more time.

Resource Management. A SimPy Resource with capacity 1 ensures mutual exclusion, preventing concurrent operations on the same machine.

MoveXY Simulator

The `MoveXYMachine` simulator models 2D positioning operations. It tracks the current position and calculates movement time based on Euclidean distance:

$$t_{move} = t_{setup} + \sqrt{(x_{target} - x_{current})^2 + (y_{target} - y_{current})^2} \times factor_{move} \tag{4.2}$$

Similar to the drill formula, this is a simplified simulation model where movement time scales linearly with distance traveled, plus a fixed setup time. The simulator maintains position state between operations, enabling realistic simulation of sequential movements.

AAS Integration

Simulators integrate with the AAS infrastructure through the invocationDelegation pattern:

1. The ResourceAgent behavior calls `invoke_drill()` on the ResourceAASClient
2. The client sends an invocation request to the BaSyx server
3. BaSyx reads the invocationDelegation qualifier from the Operation element
4. BaSyx forwards the request to the simulator endpoint (e.g., `http://localhost:8090/drill_invocation_1`)
5. The simulator executes the SimPy process and returns AAS-compliant OperationVariable results
6. The response propagates back through BaSyx to the agent

This design enables transparent switching between simulators and real machine controllers by modifying only the invocationDelegation URL in the Capability Submodel, without changes to agent code.

Simulator Instances

The system includes four simulator instances to support the multi-resource scenario:

Table 4.6: Resource Simulator Configuration

Simulator	Port	Endpoint	Resource
DrillMachine1	8090	/drill_invocation_1	Drill_1
DrillMachine2	8091	/drill_invocation_2	Drill_2
MoveXYMachine1	8092	/movexy_invocation_1	Drill_1
MoveXYMachine2	8093	/movexy_invocation_2	Drill_2

Each drill machine resource has an associated MoveXY simulator, reflecting the physical configuration where positioning and drilling are separate actuators on the same machine.

4.3 Production Agent Concept

The ProductionAAS orchestrates the manufacturing process by coordinating resource agents, managing process plans, and tracking execution progress. Unlike the ResourceAAS which represents physical assets, the ProductionAAS operates at the product level, initiating negotiations with resources and monitoring task completion. This section describes the Production Agent architecture, submodels, and its dual-core design for execution and negotiation.

4.3.1 Production Agent Overview

The ProductionAAS follows a similar architectural pattern to the ResourceAAS but serves a different purpose in the system hierarchy. While ResourceAAS agents represent manufacturing capabilities, the ProductionAAS represents production orders and coordinates the execution of process plans across multiple resources. Figure 4.15 illustrates the Production Agent component structure.

ProductionAASClient. Similar to the ResourceAASClient, this facade aggregates repository classes for accessing Production submodels. It provides methods for reading process plans, updating execution state, and managing interface configurations.

Asset Administration Shell. The Production AAS contains submodels that describe the production order rather than physical capabilities. These include the Process Plan defining the sequence of operations, Execution Tracking for monitoring progress, and Interface Endpoints for communication configuration. The shell is deployed to the same BaSyx environment as ResourceAAS instances, enabling unified access through the AAS API.

Dual-Core Agent Architecture. The Production Agent implements a separation of concerns through two specialized agent cores:

- **Execution Core**: Handles user requests, monitors execution progress, and updates the Execution Tracking submodel based on events from resource agents.

- **Negotiation Core**: Manages the Contract Net Protocol interaction with resource agents, collecting responses and selecting the best-fit resource for task execution.

This separation allows independent scaling and clearer responsibility boundaries compared to a monolithic agent design.

4.3.2 Production Agent Submodels

The ProductionAAS contains three submodels that support process definition, execution monitoring, and communication configuration.

Process Plan Submodel

The Process Plan Submodel defines the sequence of operations required to fulfill a production order. It uses a graph-based representation with nodes, edges, and control flow elements. Figure 4.17 shows the structure of this submodel.

Nodes. A SubmodelElementCollection containing Property elements for each operation in the process. In the drill machine scenario, this includes:

- `Move_Node`: Positioning operation to coordinates X, Y
- `Drill_Node`: Drilling operation with depth and RPM parameters

Edges. A SubmodelElementCollection defining the execution sequence between nodes (e.g., "Move_Node to Drill_Node").

Entry_Point / Exit_Point. Properties indicating the first and last nodes in the process plan, enabling the Execution Core to determine where to begin and when execution is complete.

Preconditions / Postconditions. Properties defining the required state before execution can begin and the expected state after completion.

Required_Capabilities. A list of capabilities needed to execute the process plan (e.g., "[Drill_Capability, MoveXY_Capability]"). The Negotiation Core uses this to filter resource agents during CFP broadcast.

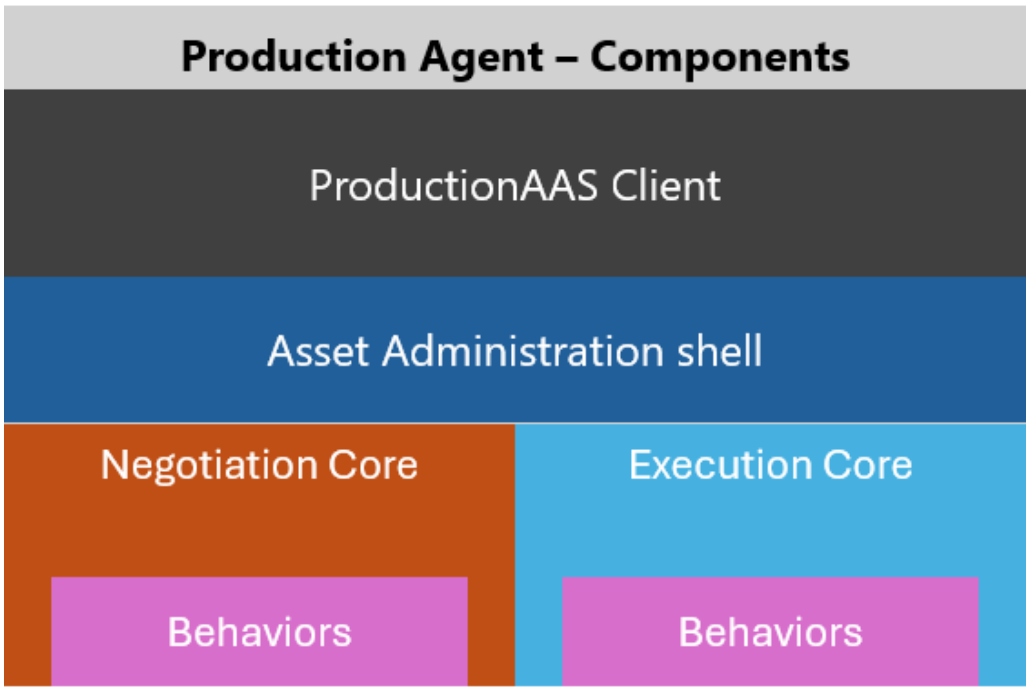


Figure 4.15: Production Agent Component Structure

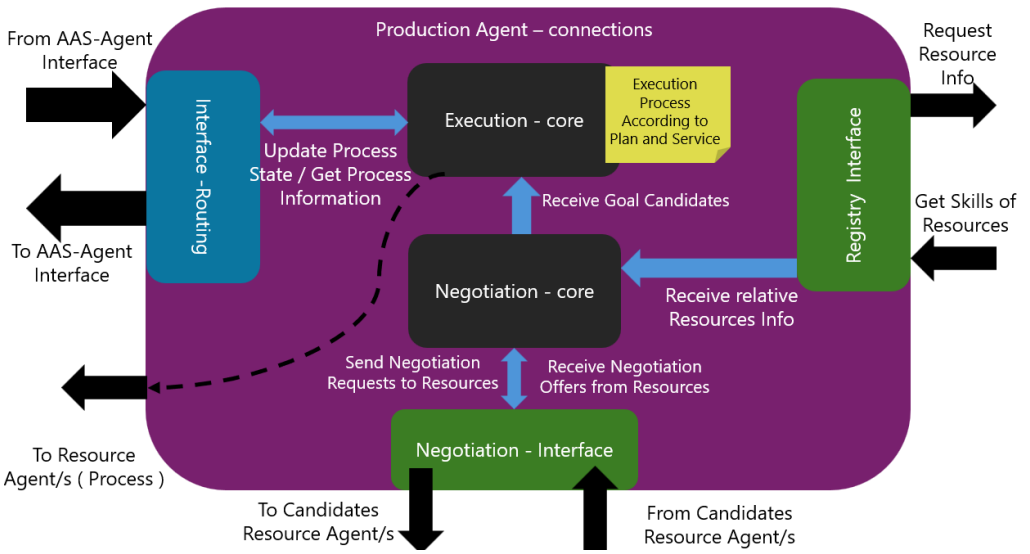


Figure 4.16: Production Agent Internal Architecture

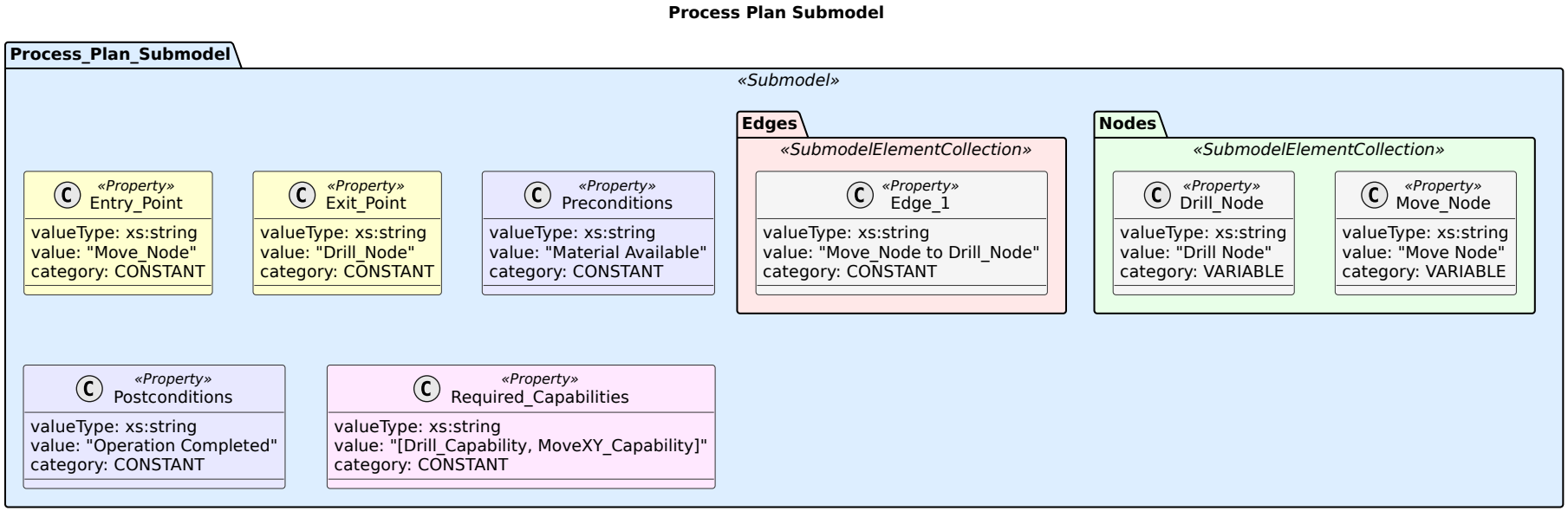


Figure 4.17: Process Plan Submodel Structure

Execution Tracking Submodel

The Execution Tracking Submodel provides real-time visibility into process execution. It is updated by the Execution Core as resource agents report progress. Figure 4.18 shows the structure.

Current_Node. The node currently being executed in the process plan, updated when resource agents publish tracking events.

Token_Position. An integer indicating how many steps have been completed, implementing a simple Petri net-style token tracking mechanism.

Time_Stamp. The timestamp of the last execution update, providing temporal context for monitoring systems.

Step_Status. The status of the current operation: "Not Started", "running", "completed", or "failed".

Interface Endpoints Submodel

The Interface Endpoints Submodel as in figure 4.19 contains configuration for external communication, similar to the Interaction Submodel in ResourceAAS but focused on command invocation endpoints.

Endpoints Collection. Contains properties for:

- `Drill_Command_Invocation_Endpoint`: Base URL for drill operation invokers
- `Move_Command_Invocation_Endpoint`: Base URL for movement operation invokers

4.3.3 Production Agent Execution Core

The Execution Core manages the lifecycle of production requests from receipt through completion. It operates as a SPADE agent extending both `Agent` and `PubSubMixin` for XMPP communication. Figure 4.20 illustrates the class architecture.

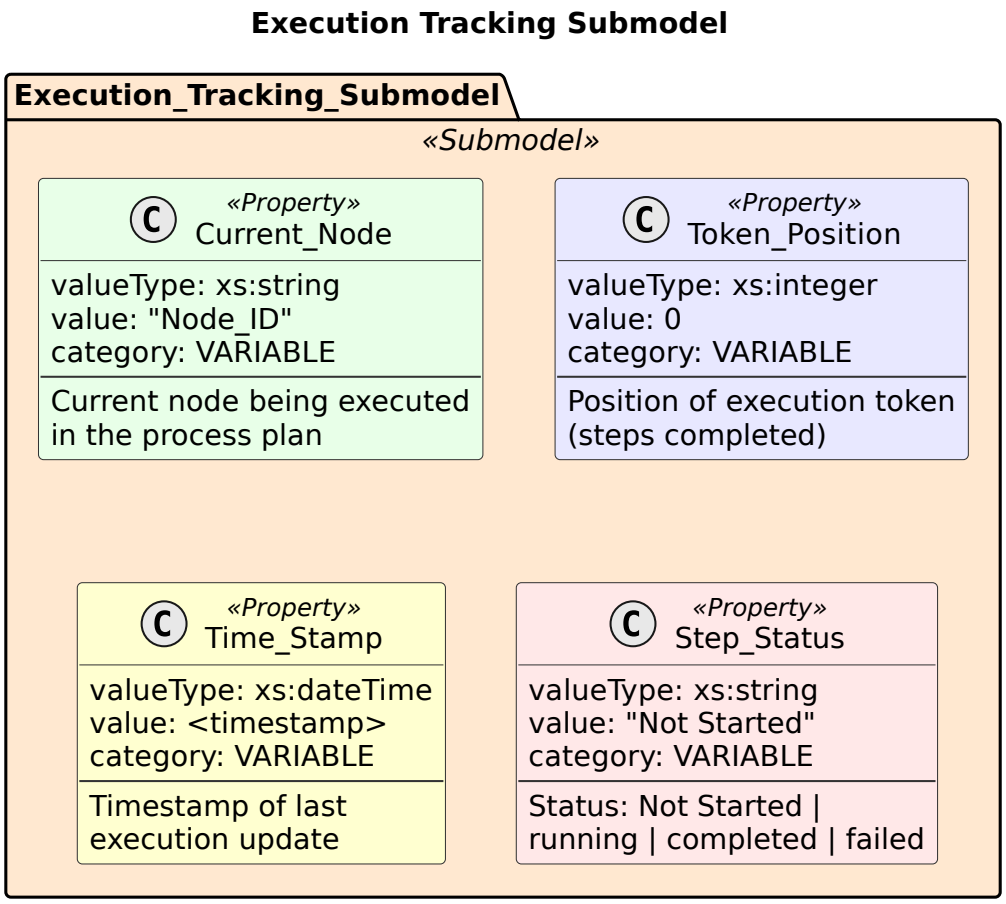


Figure 4.18: Execution Tracking Submodel Structure

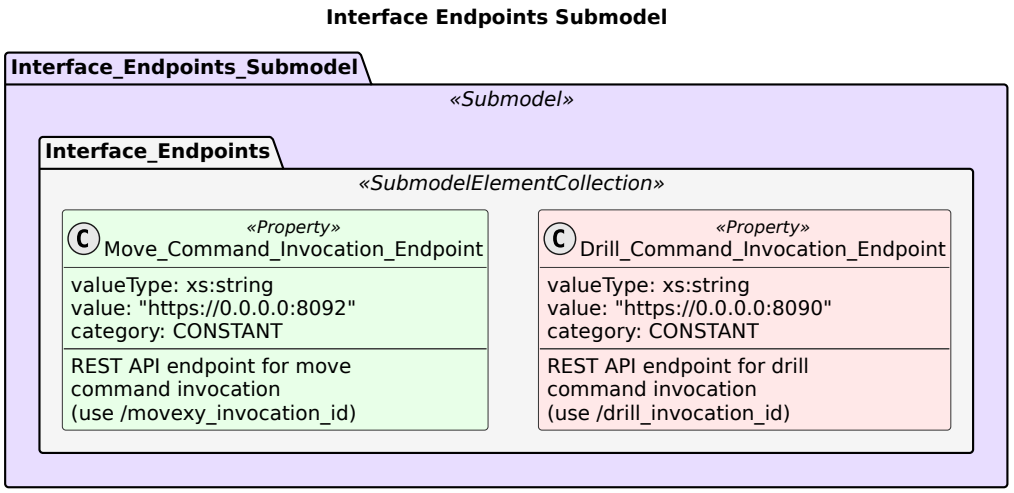


Figure 4.19: Interface Endpoints Submodel Structure

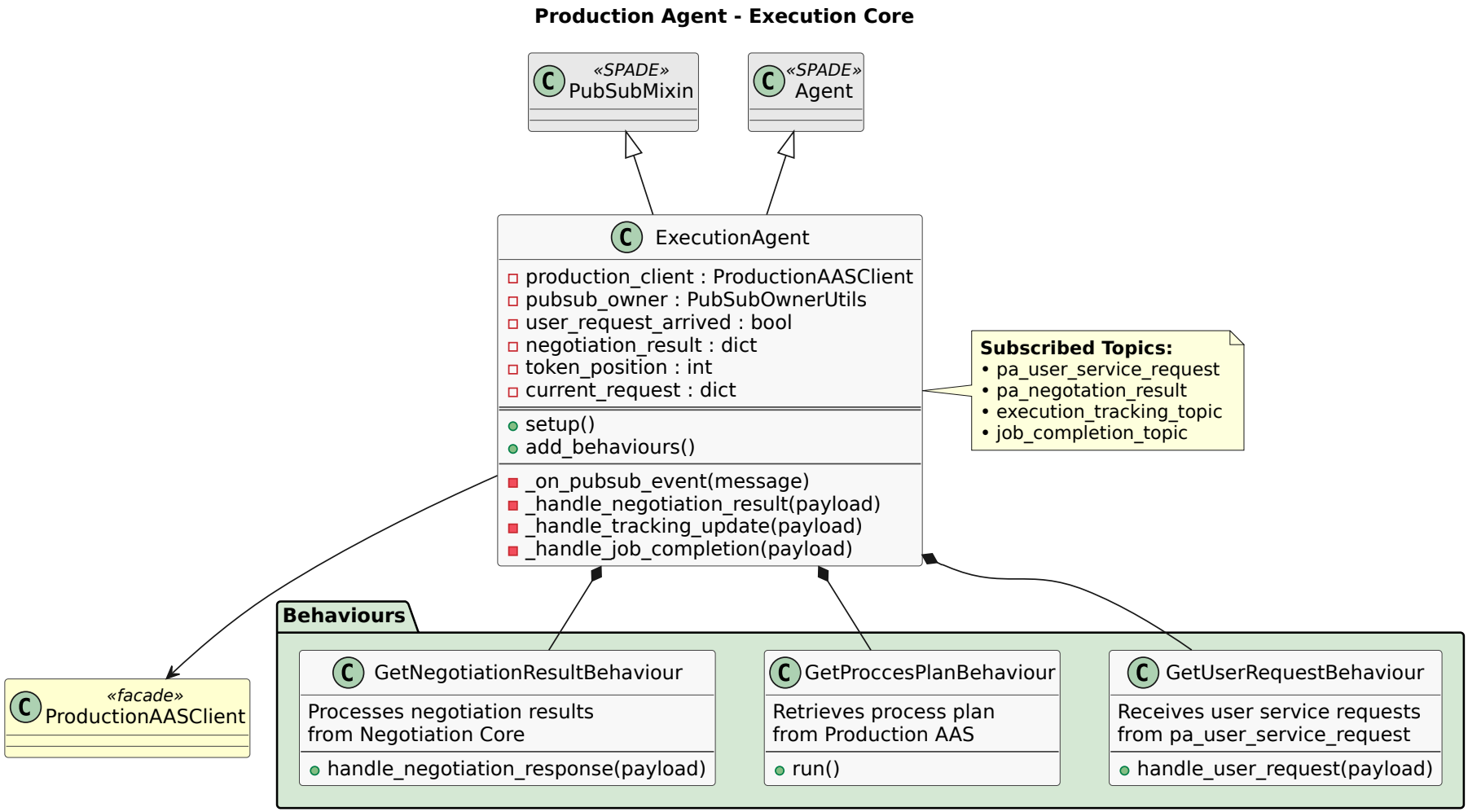


Figure 4.20: Execution Core Class Architecture

ExecutionAgent Class

The `ExecutionAgent` class serves as the central coordinator for production execution. It maintains several key attributes:

- `production_client`: Reference to the ProductionAASClient for accessing submodel data
- `pubsub_owner`: Utility for managing XMPP PubSub node ownership and subscriptions
- `negotiation_result`: Stores the selected resource and parameters from negotiation
- `token_position`: Tracks execution progress through the process plan (Petri net-style counter)
- `current_request`: Stores the active production request for reference during tracking

During setup, the agent initializes the ProductionAASClient and subscribes to four XMPP topics:

- `pa_user_service_request`: Receives new production requests
- `pa_negotation_result`: Receives negotiation outcomes from the Negotiation Core
- `execution_tracking_topic`: Receives progress updates from resource agents
- `job_completion_topic`: Receives task completion notifications

Event Handling

The `_on_pubsub_event()` method routes incoming messages to appropriate handlers based on the topic:

User Request Handling. When a message arrives on `pa_user_service_request`, the agent delegates to `GetUserRequestbehavior`, which parses the CFP message and forwards it to the Negotiation Core.

Negotiation Result Handling. The `_handle_negotiation_result()` method processes the selected resource information, logs the negotiation summary, and triggers execution preparation.

Tracking Update Handling. The `_handle_tracking_update()` method receives progress events from resource agents during execution. For each update, it increments the token position, updates the Execution Tracking Submodel via the ProductionAASClient, and logs the operation status.

Job Completion Handling. The `_handle_job_completion()` method finalizes the production run by logging completion status and saving execution reports.

behaviors

The Execution Core uses three behaviors to handle different aspects of request processing:

GetUserRequestbehavior. Receives and parses user service requests from the `pa_user_service_request` topic. It extracts the skill requirements, time slot preferences, and input parameters, then publishes the request to the Negotiation Core via `pa_execution_service_topic`.

GetProccesPlanbehavior. Retrieves the process plan from the Production AAS, providing the Execution Core with information about the sequence of operations, entry/exit points, and required capabilities.

GetNegotiationResultbehavior. Processes negotiation results received from the Negotiation Core. Upon receiving a positive result with a selected resource, it prepares the execution context and waits for tracking updates as the resource agent executes the task.

4.3.4 Production Agent Negotiation Core

The Negotiation Core implements the Contract Net Protocol initiator role, broadcasting Call-For-Proposals to resource agents and selecting the best candidate for task execution. Like the Execution Core, it operates as a SPADE agent with XMPP publish-subscribe capabilities. Figure 4.21 illustrates the class architecture.

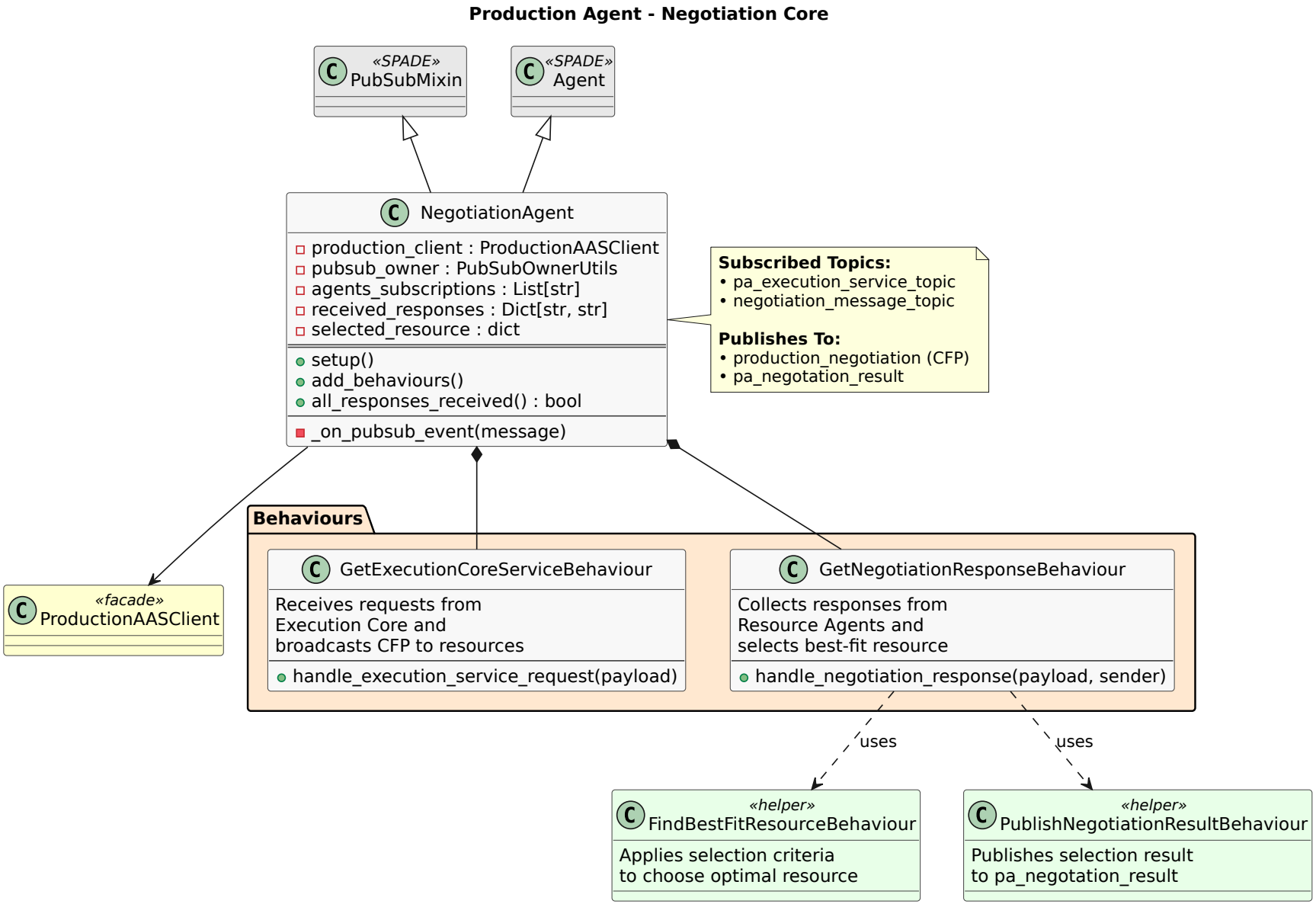


Figure 4.21: Negotiation Core Class Architecture

NegotiationAgent Class

The `NegotiationAgent` class manages the negotiation process with resource agents. It maintains several key attributes:

- `production_client`: Reference to the ProductionAASClient for accessing submodel data
- `pubsub_owner`: Utility for querying XMPP PubSub node subscribers
- `agents_subscriptions`: List of resource agent identifiers subscribed to the negotiation topic
- `received_responses`: Dictionary tracking responses received from each resource agent
- `selected_resource`: Stores the chosen resource after selection completes

During setup, the agent initializes the ProductionAASClient, queries the subscriber list from the `production_negotiation` topic, and subscribes to two XMPP topics:

- `pa_execution_service_topic`: Receives negotiation requests from the Execution Core
- `negotiation_message_topic`: Receives proposal responses from resource agents

The agent publishes to:

- `production_negotiation`: Broadcasts CFP messages to all subscribed resource agents
- `pa_negotation_result`: Publishes the selected resource back to the Execution Core

Response Tracking

The `all_responses_received()` method checks whether all expected resource agents have responded. It compares the set of received response keys against the set of subscribed agents, returning true only when all expected responses have arrived. This ensures the selection algorithm has complete information before making a decision.

behaviors

The Negotiation Core uses two main behaviors and two helper behaviors:

GetExecutionCoreServicebehavior. Receives negotiation requests from the Execution Core via `pa_execution_service_topic`. Upon receiving a request, it:

1. Parses the CFP message containing skill requirements and parameters
2. Broadcasts the CFP to all resource agents via the `production_negotiation` topic
3. Initializes the response tracking dictionary

GetNegotiationResponsebehavior. Collects and processes responses from resource agents. For each incoming response on `negotiation_message_topic`, it:

1. Extracts the sender identifier and response payload
2. Stores the response in the `received_responses` dictionary
3. Checks if all expected responses have been received
4. When complete, triggers the resource selection process

FindBestFitResourcebehavior. Applies the selection criteria to choose the optimal resource from the collected responses. The selection algorithm:

1. Filters out responses with violations (invalid proposals)
2. Among valid proposals, evaluates based on criteria such as time slot availability
3. Returns the best-fit resource identifier

PublishNegotiationResultbehavior. Publishes the selection result to `pa_negotation_result`, informing the Execution Core of the chosen resource. The result includes the selected resource identifier, allocated time slot, and original request parameters.

Contract Net Protocol Flow

The complete negotiation flow follows the Contract Net Protocol pattern:

1. Execution Core publishes a service request to `pa_execution_service_topic`
2. Negotiation Core receives the request and broadcasts CFP to `production_negotiation`
3. Resource agents evaluate the CFP against their beliefs and respond with proposals or rejections
4. Negotiation Core collects all responses via `negotiation_message_topic`
5. Once all responses are received, the best-fit resource is selected
6. The selection result is published to `pa_negotation_result`
7. Execution Core receives the result and monitors execution via tracking topics

4.4 System Interaction

This section describes the interaction between the Production Agent's dual cores and the Resource Agents during the complete production cycle. The interaction follows a coordinated flow from user request through negotiation to task execution and completion. Figure 4.22 illustrates the complete interaction state machine.

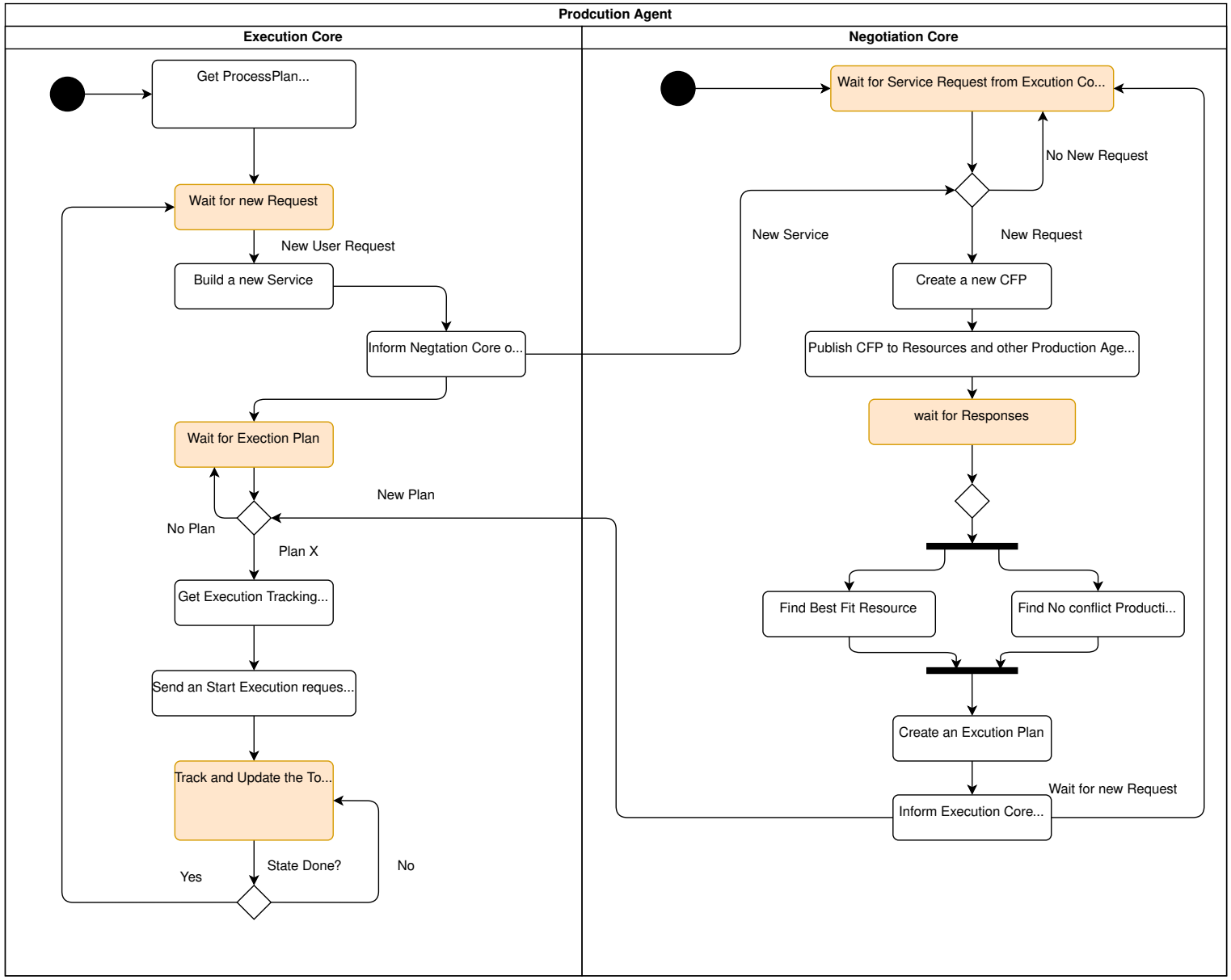


Figure 4.22: Production Agent - Resource Agent - System Interaction State Machine

4.4.1 Execution Core Flow

The Execution Core operates as the entry point for production requests and manages the overall execution lifecycle:

Request Reception. The flow begins when a user request arrives via the `pa_user_service_request` topic. The Execution Core receives and parses the request, extracting the required skills, time slot preferences, and input parameters.

Service Delegation. Upon receiving a valid request, the Execution Core publishes a service request to the `pa_execution_service_topic`, delegating the negotiation task to the Negotiation Core. The Execution Core then enters a waiting state for the negotiation result.

Execution Monitoring. After receiving a positive negotiation result, the Execution Core monitors task execution by subscribing to tracking events from the selected Resource Agent. It updates the Execution Tracking Submodel as operations progress through the process plan.

Completion Handling. When the Resource Agent signals task completion via `job_completion_topic`, the Execution Core finalizes the production run and returns to waiting for the next user request.

4.4.2 Negotiation Core Flow

The Negotiation Core implements the Contract Net Protocol initiator role:

CFP Broadcast. Upon receiving a service request from the Execution Core, the Negotiation Core broadcasts a Call-For-Proposal to all Resource Agents subscribed to the `production_negotiation` topic.

Response Collection. The Negotiation Core waits for responses from all subscribed Resource Agents. Each response is stored in a tracking dictionary, and the core checks whether all expected responses have been received.

Parallel Response Handling. Resource Agents respond independently and potentially in parallel. The state machine shows two parallel response paths converging at a synchronization bar, representing the collection of multiple proposals.

Resource Selection. Once all responses are collected, the Negotiation Core

applies the best-fit selection algorithm:

1. Filter responses without violations (valid proposals)
2. Evaluate remaining proposals based on selection criteria
3. Select the optimal resource for task execution

Execution Plan Creation. After selecting a resource, the Negotiation Core creates an execution plan containing the selected resource identifier, allocated time slot, and task parameters.

Result Publication. The Negotiation Core informs the Execution Core of the selection result via `pa_negotation_result` and returns to waiting for the next negotiation request.

4.4.3 Inter-Core Communication

The two cores communicate through XMPP publish-subscribe topics, maintaining loose coupling:

Table 4.7: Production Agent Internal Communication Topics

Topic	Publisher	Subscriber
pa_execution_service_topic	Execution Core	Negotiation Core
pa_negotation_result	Negotiation Core	Execution Core

4.4.4 Resource Agent Interaction

The Production Agent interacts with Resource Agents through external XMPP topics:

Table 4.8: Production-Resource Communication Topics

Topic	Direction	Purpose
production_negotiation	PA → RA	CFP broadcast
negotiation_message_topic	RA → PA	Proposal responses
execution_tracking_topic	RA → PA	Progress updates
job_completion_topic	RA → PA	Task completion

This event-driven architecture enables asynchronous coordination between the Production Agent and multiple Resource Agents, supporting scalable multi-resource scenarios without tight coupling between components.

4.5 Submodel Design Considerations

This section addresses design decisions and scope limitations of the proposed submodel structures. It differentiates between generic (reusable) and scenario-specific elements, discusses current limitations, and provides guidance for extending the system.

4.5.1 Generic vs. Scenario-Specific Elements

The submodel structures are designed with separation between reusable patterns and scenario-specific content. Table 4.9 identifies which elements are transferable to other domains.

Table 4.9: Generic vs. Scenario-Specific Submodel Elements

Submodel	Generic (Reusable)	Scenario-Specific
Operational State	Current_State, Free_Slots, Booked_Slots structure	–
Knowledge	Constraints collection pattern, Constraint_Type structure	Max_Depth, Min_Depth (drilling)
Capability	Operation with invocationDelegation, Supported_Skills list	Drill_Capability, MoveXY_Capability
Interaction	Time_Slots structure, XMPP_Endpoint	Specific time slot values
Process Plan	Nodes, Edges, Entry/Exit_Point, Required_Capabilities	Move_Node, Drill_Node
Execution Tracking	Current_Node, Token_Position, Step_Status	–

Integrating New Resources. To add a new resource with different capabilities (e.g., a welding machine), the following steps are required:

1. **Define Capability Operations**: Create new `Operation` elements (e.g., `Welding_Capability`) with appropriate input/output parameters and `invocationDelegation` endpoints.

2. **Implement Skill Invoker**: Create a simulator or hardware interface that responds to the delegated operation calls.

3. **Configure Knowledge Constraints**: Define resource-specific constraints (e.g., `Max_Weld_Length`, `Supported_Materials`).

4. **Register Capability String**: Add the capability identifier to `Supported_Skills` using the same naming convention.

5. **Create AgentSpeak Plans**: Add plans for the new skill execution (e.g., `+excute_skills(welding_capability)`).

The submodel *structure* remains unchanged; only the *content* is adapted.

4.5.2 Capability Matching Mechanism

The current implementation uses **string-based capability matching**. The Process Plan specifies required capabilities as strings (e.g., `"Drill_Capability"`), and Resource Agents advertise their capabilities in the `Supported_Skills` property. During negotiation, string comparison determines compatibility.

Limitation: This approach is susceptible to errors:

- Typographical errors (e.g., `"Drill_Capbility"` vs. `"Drill_Capability"`)
- Inconsistent naming conventions across resources
- No semantic understanding of capability equivalence

Recommended Extension: Future implementations should adopt a formal capability ontology or taxonomy. Options include:

- **ECLASS**: Standardized classification for products and services
- **IEC CDD**: Common Data Dictionary for industrial terminology

- **Custom OWL Ontology**: Domain-specific capability definitions with semantic reasoning

Using semantic identifiers (IRIs) instead of plain strings would enable:

- Capability subsumption (e.g., `HighSpeedDrilling` is-a `Drilling`)
- Equivalence detection across different naming conventions
- Automated validation of process plan feasibility

4.5.3 Process Plan Structure and Limitations

Linear vs. Precedence Graph Execution

The current implementation supports **linear process plans** with sequential node execution. The Process Plan Submodel stores nodes and edges, but the execution logic processes nodes in a fixed sequence (Entry_Point → ... → Exit_Point).

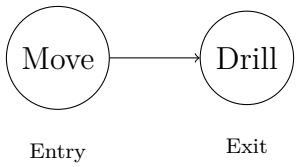


Figure 4.23: Current Implementation: Linear Process Plan

Limitation: True precedence graphs with parallel branches are **not supported** in the current prototype:

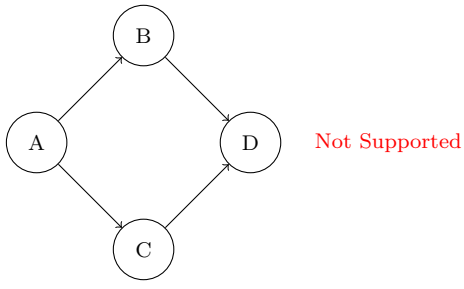


Figure 4.24: Precedence Graph with Parallel Branches (Not Implemented)

Extending to full precedence graph support would require:

- Multiple token positions (see Section 4.5.4)

- Fork/join synchronization logic
- Parallel resource allocation and negotiation

Capability-Node Association

In the current design, `Required_Capabilities` is a property at the **Process Plan level**, not associated with individual nodes:

```
sm_required_capabilities = model.Property(
    id_short="Required_Capabilities",
    value="[Drill_Capability, MoveXY_Capability]"  # Plan-
        ↪ level
)
```

Limitation: This does not explicitly link which capability is needed for which node. The association is implicit through naming conventions (`Move_Node` ↔ `MoveXY_Capability`).

Recommended Extension: Each node should explicitly reference its required capability:

```
sm_move_node = model.SubmodelElementCollection(
    id_short="Move_Node",
    value=[
        model.Property(id_short="Required_Capability",
                       value="MoveXY_Capability"),
        model.Property(id_short="Sequence_Number", value="1")
            ↪ ,
        ...
    ]
)
```

4.5.4 Token Position and Execution State

The Execution Tracking Submodel uses a **single** `Token_Position` and `Current_Node` property:

```
model.Property(id_short="Token_Position", value="1")
```

```
model.Property(id_short="Current_Node", value="Move_Node")
```

Design Decision: This models a single execution thread, analogous to a Petri net with one token.

Limitation: In a full Petri net or precedence graph, multiple tokens can exist simultaneously (parallel execution). The current design cannot represent:

- Concurrent operations on different resources
- Fork states where execution splits
- Join states requiring synchronization

Justification: For the proof-of-concept with linear plans, a single token is sufficient. Extending to multiple tokens would require refactoring `Token_Position` into a collection of token objects.

4.5.5 Knowledge Submodel Simplifications

The Knowledge Submodel stores constraints as simple min/max value pairs:

```
model.Property(id_short="Max_Depth", value="100.0")
model.Property(id_short="Min_Depth", value="1.0")
model.Property(id_short="Constraints_Type", value="[Max_Depth
    ↪ ,␣Min_Depth]")
```

Limitation: This cannot express **functional dependencies** between parameters. For example:

- A crane's maximum load depends on the boom extension
- Drilling speed limits depend on material hardness
- Tool wear affects achievable tolerances over time

Recommended Extension: Complex constraints could be modeled as:

- Mathematical expressions (e.g., `MaxLoad = f(BoomLength)`)
- Lookup tables stored as `SubmodelElementCollection`
- References to external constraint models (e.g., FMU/FMI)

4.5.6 Assembly Processes and Bill of Materials

The current submodels include a `Preconditions` property with value `"Material Available"`, but **do not model explicit Bill of Materials (BOM)** structures.

Scope Limitation: The proof-of-concept focuses on machining operations (drilling, positioning) where a single workpiece is processed. Assembly scenarios with:

- Multiple input parts per operation
- Part tracking through production stages
- Component-product relationships

are **not covered** by the current submodel design.

Extension Approach: Supporting assembly would require:

- BOM Submodel with part references
- Material tracking in Operational State
- Precondition evaluation logic checking part availability

4.5.7 User Request Definition

A **User Request** is a production order submitted to the Production Agent. In the implementation, it is a message containing:

Table 4.10: User Request Message Structure

Field	Type	Description
`skills_required`	String	Capability needed (e.g., `"drill_capability"`)
`at_time`	String	Requested time slot (e.g., `"10:30-11:00"`)
`input_arguments`	Dictionary	Operation parameters (X, Y, Depth, RPM)

The User Request is published to the `pa_user_service_request` XMPP topic and received by the Execution Core, which initiates the negotiation process.

4.5.8 Concrete Example: Drill Operation

To illustrate how the submodels work together, consider a complete drill operation request:

Step 1: User Request

```
{
  "skills_required": "drill_capability",
  "at_time": "10:30-11:00",
  "input_arguments": {"X": 60, "Y": 50, "Depth": 5, "RPM":
    ↪ 1500}
}
```

Step 2: Process Plan Lookup

The Production Agent reads the Process Plan Submodel:

- `Entry_Point`: `"Move_Node"`
- `Exit_Point`: `"Drill_Node"`
- `Edges`: `"Move_Node to Drill_Node"`
- `Required_Capabilities`: `"[Drill_Capability, MoveXY_Capability]"`

Step 3: Resource Discovery

The Negotiation Core broadcasts a CFP. Resource Agent 1 checks its Capability Submodel:

- `Supported_Skills`: `"drill_capability, movexy_capability"` ✓

And its Interaction Submodel:

- `Free_Slots`: includes `"10:30-11:00"` ✓

And its Knowledge Submodel:

- `Max_Depth`: 100mm $\geq$ requested 5mm ✓

Step 4: Execution

After selection, the Resource Agent executes:

1. Invoke `MoveXY_Capability` with X=60, Y=50
2. Update `Current_Node` = `"Move_Node"`, `Step_Status` = `"running"`
3. On completion: `Step_Status` = `"completed"`
4. Invoke `Drill_Capability` with Depth=5, RPM=1500
5. Update `Current_Node` = `"Drill_Node"`, `Step_Status` = `"running"`
6. On completion: `Step_Status` = `"completed"`

Step 5: Completion

The Execution Tracking Submodel final state:

- `Token_Position`: 2 (at Exit_Point)
- `Current_Node`: `"Drill_Node"`
- `Step_Status`: `"completed"`

This example demonstrates how the generic submodel structures are populated with scenario-specific content to execute a concrete production task.

Chapter 5

Implementation

This chapter describes the implementation of the Type 3 AAS prototype, focusing on key architectural decisions and solutions to technical challenges. Detailed configuration instructions and code listings are provided in Appendix 8.2.

5.1 Infrastructure Overview

The implementation uses a combination of open-source tools and frameworks to realize the Type 3 AAS architecture. Figure 5.1 illustrates the deployment architecture, and Table 5.1 summarizes the technology stack.

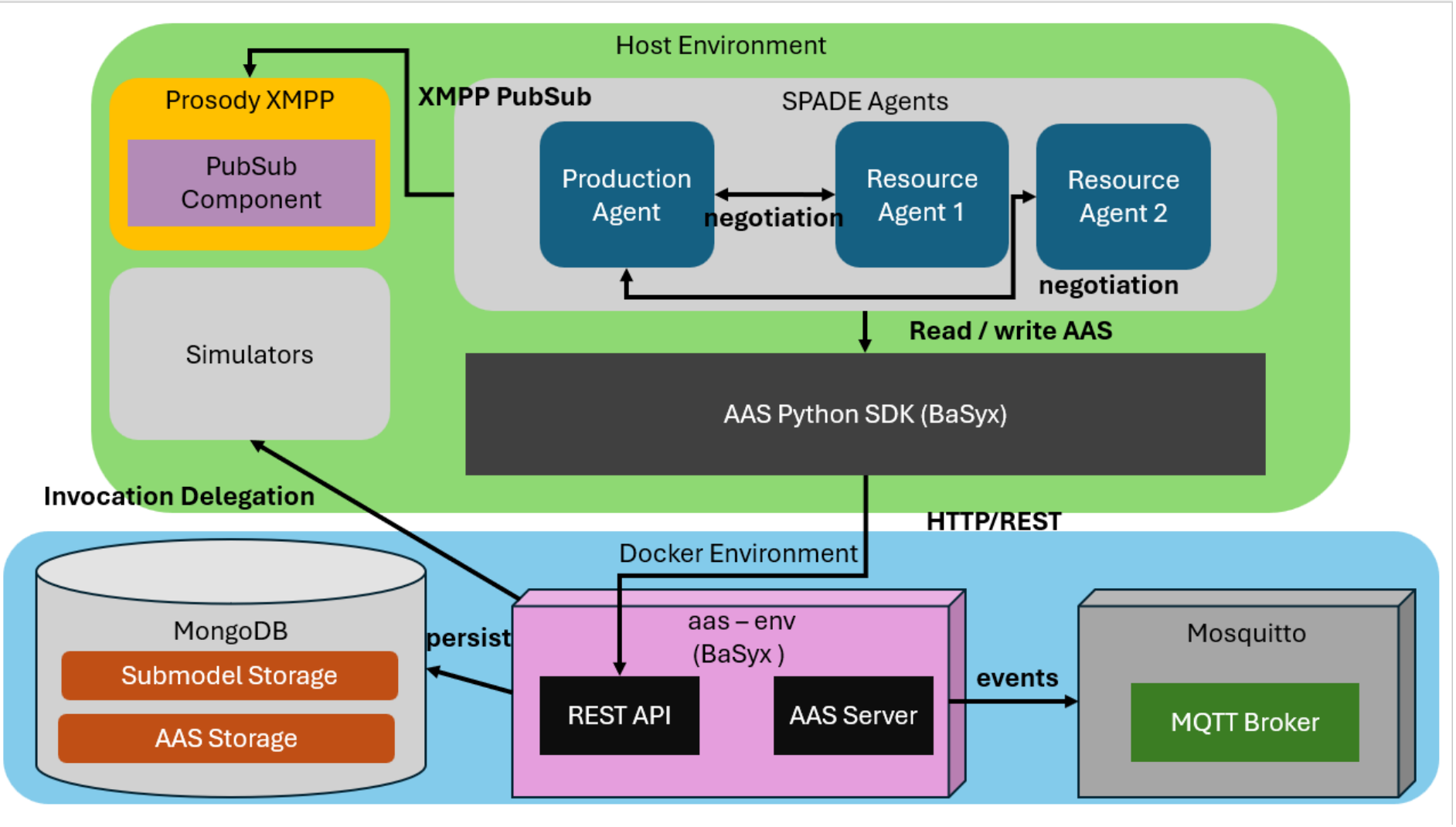


Figure 5.1: System Infrastructure Overview

Table 5.1: Implementation Technology Stack

Component	Technology	Purpose
AAS Server	Eclipse BaSyx (Java)	AAS hosting and REST API
AAS SDK	BaSyx Python SDK	Submodel creation and access
Agent Framework	SPADE + spade_bdi	BDI agent implementation
XMPP Server	Prosody	Agent communication
Message Broker	Prosody PubSub	Event distribution
Database	MongoDB	AAS persistence
Event Bus	MQTT (Mosquitto)	AAS change notifications
Simulators	FastAPI + SimPy	Machine simulation
Containerization	Docker Compose	Service orchestration

The system consists of containerized services (BaSyx, MongoDB, MQTT) orchestrated via Docker Compose, while the SPADE agents and Prosody XMPP server run on the host system. This hybrid approach simplifies development while maintaining production-like service isolation.

5.2 Asset Administration Shell Server

The implementation uses the Eclipse BaSyx AAS server, which provides MongoDB-based persistence, MQTT event notification on data changes, REST API endpoints compliant with the AAS specification, and a web-based monitoring interface. The server is deployed as a Docker container for consistent environments.

5.2.1 Operation Invocation Delegation

A key implementation pattern is the `invocationDelegation` qualifier, which enables AAS Operations to delegate execution to external services. When an Operation is invoked via the AAS REST API, the BaSyx server forwards the request to an external HTTP endpoint specified in the qualifier.

This pattern connects the standardized AAS interface to the physical asset simulators (or real PLCs in production). The invoker services are implemented as FastAPI applications that:

1. Receive the operation request with input parameters

2. Execute the corresponding machine operation

3. Return results in AAS-compliant format

The detailed configuration for Docker networking and invoker implementation is provided in Appendix 8.2.1.

5.3 Agent Communication Infrastructure

The implementation uses Prosody as the XMPP server for agent communication. Prosody was selected over alternatives (e.g., ejabberd) due to better compatibility with the SPADE Python framework and straightforward PubSub configuration.

The server provides:

- Point-to-point messaging between agents
- Publish-subscribe functionality for event distribution
- SSL/TLS encryption for secure communication

Agents are registered as XMPP users and configured with administrator privileges to enable PubSub node creation. The detailed server configuration is provided in Appendix 8.2.2.

5.4 Agent Implementation Challenges

Several technical challenges were encountered during agent implementation. This section documents the issues and their solutions, as they provide insights for future implementations.

5.4.1 BDI vs. Finite State Machine

During initial development, an attempt was made to implement agent behavior using a Finite State Machine (FSM) pattern. This approach proved problematic for BDI agents:

- **Rigidity**: FSMs enforce fixed state transitions, preventing dynamic plan selection based on beliefs and goals.
- **Loss of Reactivity**: BDI agents should react to belief changes opportunistically; FSMs constrain behavior to explicit transitions.
- **Commitment Conflict**: FSMs require commitment to a predefined structure, contradicting the BDI principle of deliberation over intentions.

The final implementation uses AgentSpeak plans triggered by belief changes, allowing context-dependent plan selection rather than predetermined paths.

5.4.2 SPADE Belief Storage Limitation

A limitation was discovered in the SPADE BDI implementation: storing list-like strings as belief values causes incorrect parsing due to internal string splitting. For example, a belief containing `"[08:00-08:30, 08:30-09:00]"` is truncated at the first bracket.

Workaround: Complex data structures are stored in AAS submodels rather than directly in beliefs. The agent retrieves structured data through the ResourceAASClient when needed, ensuring data integrity.

5.4.3 PubSub Subscriber Crosstalk

The SPADE PubSub implementation uses a single global callback for all subscriptions. Multiple calls to `set_on_item_published()` overwrite previous handlers, causing messages from different topics to be delivered to incorrect handlers.

Solution: A single callback is registered that dispatches messages based on the topic node identifier. This pattern ensures each topic's messages reach the appropriate handler without interference. The implementation is shown in Appendix 8.2.3.

5.5 Deployment Architecture

The complete system is deployed using Docker Compose for containerized services and host-based execution for agents. Table 5.2 summarizes the deployment configuration.

Table 5.2: Deployment Components

Component	Deployment	Communication
BaSyx AAS Server	Docker container	REST API (port 8081)
AAS Registry	Docker container	REST API (port 8082)
MongoDB	Docker container	Internal (port 27017)
Mosquitto MQTT	Docker container	MQTT (port 1883)
Prosody XMPP	Host system	XMPP (port 5222)
Resource Agents	Host system	XMPP + REST
Production Agent	Host system	XMPP + REST
Simulators	Host system	HTTP (port 8090+)

The startup sequence ensures dependencies are available before dependent services start: Docker services first, then Prosody, followed by AAS initialization, simulators, and finally agents. The complete Docker Compose configuration and startup scripts are provided in Appendix 8.2.4.

5.6 Summary

This chapter presented the implementation of the Type 3 AAS prototype, highlighting:

- The hybrid deployment architecture combining Docker containers with host-based agents
- The `invocationDelegation` pattern for connecting AAS Operations to external execution
- Technical challenges in BDI agent implementation and their solutions

The implementation demonstrates that integrating BDI agents with AAS infrastructure is feasible using existing open-source tools, though framework-specific

workarounds are required. Detailed configuration files and code listings are provided in Appendix 8.2 for reproducibility.

Chapter 6

Evaluation

This chapter evaluates the Type 3 AAS prototype through functional testing, execution analysis, and requirements validation. The evaluation uses automated report generation to capture system behavior during production runs.

6.1 Test Environment

The evaluation was conducted on a development workstation with the following configuration: The test scenario involves a production request for a drill operation

Table 6.1: Test Environment Configuration

Component	Specification
Operating System	Ubuntu 24.04 LTS
Docker	Docker Compose v2.x
Python	3.12
XMPP Server	Prosody
AAS Server	Eclipse BaSyx (Docker)

requiring two sequential skills: positioning (MoveXY) and drilling (Drill). Two Resource Agents compete for the task through Contract Net Protocol negotiation.

6.2 Report Generation System

The implementation includes an automated report generation system that captures events throughout the production lifecycle. Two report types are generated:

6.2.1 Production Reports

The `ReportCollector` class aggregates events from the Production Agent and generates comprehensive reports for each production run. Reports are saved in both human-readable text format and machine-parseable JSON format.

Event Types Captured:

- `SYSTEM_START` / `JOB_COMPLETED`: Production run boundaries
- `CFP_RECEIVED` / `CFP_RESPONSE`: Negotiation events
- `RESOURCE_SELECTED`: Winner selection
- `EXECUTION_STARTED` / `OPERATION_COMPLETED`: Task execution
- `AAS_UPDATED`: Submodel modifications

6.2.2 Simulator Reports

Each simulator generates independent reports tracking machine utilization, operation counts, and processing times. These reports enable verification of skill execution and timing analysis.

6.3 Functional Test Results

This section presents the functional testing of the Type 3 AAS prototype. The evaluation documents: (1) AAS submodel changes during execution, (2) multi-agent negotiation with competing resources, and (3) multi-node process plan execution.

6.3.1 Test Scenarios

Three test scenarios were designed to validate different aspects of the system. Table 6.2 provides an overview of each scenario, its purpose, and the requirements it validates.

Table 6.2: Test Scenario Overview

ID	Scenario	Purpose	Validates
TS-1	Single Request, Resource Selection	Verify CNP negotiation selects appropriate resource	REQ_20, REQ_31
TS-2	AAS State Tracking	Document all submodel changes during execution	REQ_30, Bidirectional Integration
TS-3	Multi-Node Process Plan	Execute sequential Move → Drill operations	REQ_27, Process Plan

6.3.2 Test Scenario TS-1: Multi-Agent Negotiation

Test Configuration

The negotiation test was configured with the following components:

- **Production Agent**: 1 instance (Execution Core + Negotiation Core)
- **Resource Agents**: 2 instances (resource_agent_1, resource_agent_2)
- **Request**: `drill_capability` at time slot `10:30-11:00`

Test Execution

Four production runs were executed with identical requests. The negotiation outcomes are summarized in Table 6.3.

Table 6.3: Negotiation Test Results (TS-1)

Run ID	Participants	Selected	Reason	Duration
20251209_105841	RA1, RA2	resource_agent_1	Best fit	9.00s
20251209_110134	RA1, RA2	resource_agent_1	Best fit	9.00s
20251209_110327	RA1, RA2	resource_agent_1	Best fit	9.00s
20251209_110515	RA1, RA2	resource_agent_2	Best fit	9.45s

Observation

As shown in Table 6.3, Run 20251209_110515 selected `resource_agent_2` instead of `resource_agent_1`. Analysis of the negotiation data shows that after previous

runs, `resource_agent_1`'s time slot `10:30-11:00` was marked as booked, causing the Negotiation Core to select the next available resource. This demonstrates:

- The CNP correctly broadcasts to all registered resources
- Resources correctly report their availability based on `Booked_Slots`
- The selection algorithm chooses an alternative when the preferred resource is unavailable

6.3.3 Test Scenario TS-2: AAS State Documentation

This scenario documents all AAS submodel changes during a complete production run to validate bidirectional integration.

Initial AAS State

Before execution begins, the AAS submodels contain the initial configuration. Table 6.4 shows the ResourceAAS state, and Table 6.5 shows the ProductionAAS state.

Table 6.4: ResourceAAS Submodel State Before Execution

Submodel	Element	Value
Operational State	Current_State	idle
Operational State	Free_Slots	[08:00-08:30, 08:30-09:00, ..., 10:30-11:00]
Operational State	Booked_Slots	[]
Capability	Supported_Skills	drill_capability, movexy_capability
Knowledge	Max_Depth	100.0
Knowledge	Min_Depth	1.0
Interaction	XMPP_Endpoint	resource_agent_1@localhost

AAS Updates During Execution

During execution of Run 20251209_110327, seven submodel modifications were recorded. Table 6.6 documents each update with its timestamp relative to the production run start.

Table 6.5: ProductionAAS Submodel State Before Execution

Submodel	Element	Value
Process Plan	Entry_Point	Move_Node
Process Plan	Exit_Point	Drill_Node
Process Plan	Required_Capabilities	[Drill_Capability, MoveXY_Capability]
Execution Tracking	Current_Node	N/A
Execution Tracking	Token_Position	0
Execution Tracking	Step_Status	(empty)

Table 6.6: Detailed AAS Updates During Execution (TS-2)

#	Time (ms)	Submodel	Element	Value
1	+3787	ExecutionTracking	Current_Node	move_xy
2	+3787	ExecutionTracking	Step_Status	running
3	+5888	ExecutionTracking	Step_Status	completed
4	+5953	ExecutionTracking	Current_Node	drill
5	+5953	ExecutionTracking	Step_Status	running
6	+8933	ExecutionTracking	Step_Status	completed
7	+8933	ExecutionTracking	Token_Position	2

Final AAS State

After execution completes, the AAS submodels reflect the updated state. Table 6.7 shows the changed values.

Table 6.7: AAS Submodel State After Execution

Submodel	Element	Value (Changed)
Operational State	Current_State	idle
Operational State	Free_Slots	[...] (10:30-11:00 removed)
Operational State	Booked_Slots	[10:30-11:00]
Execution Tracking	Current_Node	Drill_Node
Execution Tracking	Token_Position	2
Execution Tracking	Step_Status	completed

The results documented in Tables 6.4 through 6.7 demonstrate that:

- The AAS is updated in real-time during execution (7 updates recorded)
- Execution Tracking correctly reflects the process plan progress
- Resource availability (time slots) is updated after task completion
- The bidirectional flow works: Agent reads beliefs → executes → updates AAS

6.3.4 Test Scenario TS-3: Multi-Node Process Plan

This scenario validates execution of a process plan with multiple sequential nodes.

Process Plan Structure

The test process plan consists of two nodes connected by a single edge, as illustrated in Figure 6.1.

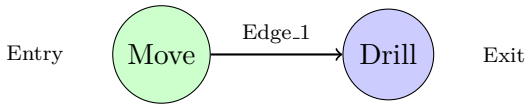


Figure 6.1: Test Process Plan: Move → Drill

The process plan configuration includes:

- **Nodes**: Move_Node, Drill_Node (2 nodes)
- **Edges**: Move_Node → Drill_Node
- **Entry Point**: Move_Node
- **Exit Point**: Drill_Node

Execution Trace

The operation sequence from Run 20251209_110327 is documented in Table 6.8.

Table 6.8: Process Plan Execution Trace (TS-3)

Step	Node	Operation	Status	Parameters	Duration
1	Move_Node	move_xy	started	X=60, Y=50	–
2	Move_Node	move_xy	completed	Result=OK	2.09s
3	Drill_Node	drill	started	Depth=5, RPM=1500	–
4	Drill_Node	drill	completed	Result=OK	2.98s

Token Progression

The token position advances through the process plan as execution progresses. Table 6.9 shows the token state at each phase.

The execution trace in Table 6.8 and token progression in Table 6.9 demonstrate:

Table 6.9: Token Position During Execution

Phase	Token_Position	Current_Node	Step_Status
Before Start	0	N/A	(empty)
Move Started	1	Move_Node	running
Move Completed	1	Move_Node	completed
Drill Started	2	Drill_Node	running
Drill Completed	2	Drill_Node	completed

- The process plan with 2 nodes executes correctly in sequence
- Token position advances through the graph ($0 \rightarrow 1 \rightarrow 2$)
- Each operation updates the Execution Tracking submodel
- The agent follows the edge from Move_Node to Drill_Node

6.3.5 Simulator Verification

To verify that operations were correctly delegated to the physical asset simulators, the simulator-side logs were analyzed. Table 6.10 summarizes the recorded operations.

Table 6.10: Simulator Operation Logs

Simulator	Operation	Parameters	Result	Time
MoveXY_Machine_1	move_xy	X=60, Y=50	Success	2.09s
DrillMachine_1	drill	Depth=5, RPM=1500	5.0	3.00s

The simulator reports in Table 6.10 confirm:

- The `invocationDelegation` pattern correctly routes requests
- Input parameters are passed correctly from AAS to simulator
- Results are returned and logged

6.3.6 Test Limitations

Table 6.11 documents the limitations of the current test suite and identifies areas for future work.

Table 6.11: Test Limitations and Future Work

Limitation	Reason / Future Work
No concurrent competing orders	Would require queue management; deferred to future work
Linear process plan only	Parallel branches not implemented (see Section 4.5.3)
Simulated hardware only	Real PLC integration planned for production deployment
No failure scenarios tested	Error recovery requires additional implementation

6.3.7 Summary

The functional tests validate the core Type 3 AAS capabilities:

- **TS-1**: Multi-agent negotiation correctly selects resources based on availability (see Table 6.3)
- **TS-2**: AAS submodels are updated bidirectionally during execution with 7 updates documented (see Table 6.6)
- **TS-3**: Multi-node process plans execute sequentially with correct token progression (see Table 6.9)

All test runs completed successfully, demonstrating the feasibility of the proposed architecture.

6.4 Requirements Validation

The implementation was validated against the requirements defined in Appendix 8.1.2. Each requirement table in the appendix includes a *Status* column indicating fulfillment. The coverage across all requirement categories is summarized in Table 6.12.

As shown in Table 6.12, the implementation achieves 95% requirements coverage (36 of 38 requirements fulfilled).

Fulfilled Requirements. All system-level, ResourceAAS, MAS/BDI, and non-functional requirements were successfully implemented. The BDI agent architecture integrates with the AAS infrastructure as specified, and the Contract Net Protocol

Table 6.12: Requirements Coverage Summary

Category	Total	Fulfilled	Coverage
System Specifications (REQ_1–8)	8	8	100%
ResourceAAS (REQ_10–20)	11	11	100%
ProductionAAS (REQ_21–31)	11	9	82%
MAS/BDI (REQ_32–34)	3	3	100%
Non-Functional (NFR_1–5)	5	5	100%
Total	**38**	**36**	**95%**

negotiation operates within the required performance bounds (<2 seconds vs. the 5-second target in NFR_5).

Partial Implementations. Two requirements use alternative approaches:

- REQ_15 and REQ_24 (FIPA ACL): XMPP PubSub was used instead of pure FIPA ACL, while maintaining equivalent Contract Net Protocol semantics.

Deferred Requirements. Two ProductionAAS submodels were not implemented:

- REQ_26 (Identification Submodel): Provides metadata not essential for core Type 3 behavior.
- REQ_28 (Services Catalog): Cataloging functionality deferred as out of scope for the proof-of-concept.

These deferrals do not impact the validation of the Type 3 AAS concept, as the core autonomous behavior, negotiation, and execution tracking capabilities are fully demonstrated.

6.5 Validation of Thesis Contributions

The evaluation results validate the three main contributions of this thesis as stated in Chapter 1.

6.5.1 Contribution 1: Bidirectional AAS Integration

The test runs demonstrate that agents both read from and write to the AAS. During initialization, Resource Agents load beliefs from the Operational State, Knowledge, and Capability Submodels (see Section 4.2.3). During execution, agents invoke Operations via the `invocationDelegation` pattern (e.g., `invoke_drill()`, `invoke_movexy()`), and the Production Agent updates the Execution Tracking Submodel in real-time.

As documented in Table 6.6, seven submodel updates were recorded during a single production run, confirming the bidirectional data flow between agents and their AAS representations. This validates that:

- Agents can use AAS submodels as a source of beliefs
- Agents can invoke AAS Operations to execute intentions
- The AAS serves as a live digital twin updated during execution

6.5.2 Contribution 2: Foundational Submodels for Proactive Behavior

The implemented submodels successfully support agent-based decision-making while maintaining compliance with the AAS meta-model. Table 6.13 summarizes how each submodel enables proactive behavior.

Table 6.13: Submodel Support for Proactive Agent Behavior

Submodel	Proactive Behavior Enabled
Capability	Skill descriptions used to respond to CFP requests
Process Plan	Goals (Entry/Exit_Point) and required capabilities guide orchestration
Knowledge	Resource constraints inform negotiation decisions
Interaction	Time slot availability enables scheduling negotiations
Execution Tracking	Real-time progress monitoring and belief updates

All submodel structures use standard AAS elements (Property, SubmodelElementCollection, Operation), demonstrating that proactive agent behavior can be

supported without extending the AAS specification. The generic vs. scenario-specific separation described in Section 4.5.1 ensures these submodels are transferable to other domains.

6.5.3 Contribution 3: Proof-of-Concept with Production Resources

The drill machine scenario validates the feasibility of implementing BDI-based Type 3 AAS in a production context. The test results demonstrate:

- **Multi-Agent Negotiation**: Successful Contract Net Protocol negotiations between two Resource Agents, with dynamic resource selection based on availability (see Table 6.3)
- **Process Plan Execution**: Coordinated task execution following a two-node process plan (Move → Drill), with correct token progression (see Table 6.9)
- **Real-Time Tracking**: Execution state updates propagated to the AAS during operation (see Table 6.6)

While the evaluation uses discrete-event simulators rather than physical hardware, the simulators implement the same AAS interfaces (`invocationDelegation` via HTTP) that would be used with real PLCs or machine controllers. This design ensures that transitioning to physical hardware requires only endpoint URL changes, not architectural modifications.

6.5.4 Summary

Table 6.14 summarizes the validation status of each thesis contribution.

All three contributions have been validated through the functional tests, demonstrating the feasibility of the proposed Type 3 AAS architecture.

Table 6.14: Thesis Contribution Validation Summary

Contribution	Status	Key Evidence
Bidirectional AAS Integration	✓	7 AAS updates per run
Foundational Submodels	✓	5 submodels enabling BDI
Proof-of-Concept	✓	4 successful test runs

6.6 Discussion

6.6.1 Achievements

The evaluation demonstrates that the Type 3 AAS concept is feasible and can be implemented using existing open-source tools. Key achievements include:

- **BDI-AAS Integration**: Successfully integrated BDI agent reasoning with AAS data models, enabling agents to use submodel data as beliefs and update submodels based on intentions.
- **Contract Net Protocol**: Implemented multi-agent negotiation for resource allocation, with the Production Agent successfully coordinating with multiple Resource Agents.
- **Real-time AAS Updates**: Demonstrated that the AAS can serve as a live digital twin, with submodel elements updated during task execution.
- **Skill Invocation**: Validated the invocationDelegation pattern for connecting AAS Operations to external execution systems.

6.6.2 Limitations

Several limitations were identified during evaluation:

- **Simulation Only**: The evaluation used simulated machines rather than physical hardware, limiting validation of real-world timing and error handling.
- **Single Production Scenario**: Only the drill-and-move scenario was tested; more complex process plans would require additional validation.

- **SPADE Framework Constraints**: The SPADE BDI implementation has limitations (e.g., belief storage), requiring workarounds documented in Chapter 5.
- **No Performance Benchmarking**: The evaluation focused on functional correctness rather than performance metrics like throughput or latency under load.

6.6.3 Comparison with Type 1 and Type 2 AAS

The implemented Type 3 AAS provides capabilities beyond passive (Type 1) and reactive (Type 2) shells are to be found in Table 6.15.

Table 6.15: AAS Type Comparison

Capability	Type 1	Type 2	Type 3
Data storage	✓	✓	✓
Event notification	–	✓	✓
Operation invocation	–	✓	✓
Autonomous decision-making	–	–	✓
Multi-agent negotiation	–	–	✓
Goal-oriented behavior	–	–	✓

6.6.4 Validation of Thesis Contributions

The evaluation results validate the three main contributions of this thesis:

Contribution 1: Bidirectional AAS Integration. The test runs demonstrate that agents both read from and write to the AAS. During initialization, Resource Agents load beliefs from the Operational State, Knowledge, and Capability Submodels. During execution, agents invoke Operations via the `invocationDelegation` pattern (e.g., `invoke_drill()`, `invoke_movexy()`), and the Production Agent updates the Execution Tracking Submodel in real-time. Table **??** shows eight submodel updates during a single production run, confirming the bidirectional data flow between agents and their AAS representations.

Contribution 2: Foundational Submodels for Proactive Behavior. The implemented submodels successfully support agent-based decision-making while maintaining compliance with the AAS meta-model. The Capability Submodel provides skill descriptions that agents use to respond to CFP requests. The Process Plan Submodel defines goals (Entry_Point, Exit_Point) and required capabilities that guide the Production Agent's orchestration logic. The Knowledge and Interaction Submodels store behavioral parameters such as resource constraints and time slot availability that inform negotiation responses. All submodel structures use standard AAS elements (Property, SubmodelElementCollection, Operation), demonstrating that proactive agent behavior can be supported without extending the AAS specification.

Contribution 3: Proof-of-Concept with Production Resources. The drill machine scenario validates the feasibility of implementing BDI-based Type 3 AAS in a production context. The test runs show successful Contract Net Protocol negotiations between multiple Resource Agents, coordinated task execution following a process plan (Move $\rightarrow$ Drill), and real-time tracking updates. While the evaluation uses discrete-event simulators rather than physical hardware, the simulators implement the same AAS interfaces (invocationDelegation via HTTP) that would be used with real PLCs or machine controllers. This design ensures that transitioning to physical hardware requires only endpoint URL changes, not architectural modifications.

6.7 Summary

The evaluation validates that the Type 3 AAS prototype meets its functional and non-functional requirements. The automated report generation system provides comprehensive traceability of system behavior, and all test runs completed successfully. While limitations exist, particularly regarding physical hardware testing and performance benchmarking, the prototype demonstrates the feasibility of integrating BDI agents with the Asset Administration Shell for autonomous manufacturing coordination.

Chapter 7

Outlook

This section identifies opportunities for extending the Type 3 AAS concept beyond the current proof-of-concept implementation.

7.1 Physical Hardware Integration

The current implementation uses simulated machines for skill execution. Future work should validate the architecture with physical manufacturing equipment:

- Integration with real PLCs and industrial controllers via OPC UA
- Handling of real-world timing constraints and communication latencies
- Error recovery and fault tolerance with actual hardware failures
- Sensor data integration for real-time belief updates

7.2 Enhanced Negotiation Mechanisms

The Contract Net Protocol implementation uses time availability as the primary selection criterion. Extensions could include:

- Multi-criteria negotiation considering energy consumption, quality metrics, and cost

- Auction-based mechanisms for competitive resource allocation
- Learning-based approaches where agents improve negotiation strategies over time
- Support for re-negotiation when execution conditions change

7.3 Scalability and Performance

The proof-of-concept was tested with two Resource Agents. Scaling to production environments requires:

- Performance benchmarking with larger agent populations (10+, 100+ agents)
- Distributed XMPP broker clusters for high message throughput
- Hierarchical agent organizations to reduce negotiation complexity
- Caching strategies for frequently accessed AAS data

7.4 Advanced BDI Capabilities

The BDI implementation could be extended with more sophisticated reasoning:

- Dynamic plan generation based on process plan analysis
- Learning from past executions to optimize future behavior
- Collaborative goal achievement across multiple agents
- Integration with external knowledge bases for enhanced decision-making

7.5 Standardization Alignment

As the AAS ecosystem evolves, the implementation should align with emerging standards:

- IDTA submodel templates for capability and skill descriptions
- VDI/VDE 2193 for agent-based communication in Industry 4.0
- Integration with Gaia-X and Manufacturing-X data spaces
- Compliance with upcoming Type 3 AAS specifications from IDTA

7.6 Security and Access Control

Production deployments require robust security measures:

- Authentication and authorization for agent interactions
- Encrypted communication channels for sensitive production data
- Audit logging for compliance and traceability
- Role-based access control for AAS submodel elements

7.7 Visualization and Monitoring

Enhanced tooling for system observation and debugging:

- Real-time dashboard for agent state and negotiation visualization
- Integration with existing Manufacturing Execution Systems (MES)
- Predictive analytics based on historical execution data
- Digital twin synchronization with 3D visualization

Chapter 8

Appendix

8.1 Detailed Use Cases and Requirements

This appendix provides the detailed specifications for use cases and system requirements summarized in Chapter 3.

8.1.1 Use Case Specifications

Table 8.1: Use Case UC1: System Startup

Attribute	Description
Identifier	UC1
Name	System Startup
Description	The SmartDrillSystem initializes all components and prepares for operation.
Actors	Operator
Preconditions	The SmartDrillSystem is shut down.
Trigger	The Operator starts the SmartDrillSystem.

Continued on next page

Table 8.1 – UC1: System Startup (continued)

Attribute	Description
Standard Procedure	1. The Operator starts the SmartDrillSystem (via GUI or CLI). 2. The SmartDrillSystem initializes all system components. 3. The SmartDrillSystem triggers UC2: AAS Register. 4. The SmartDrillSystem displays “ResourceAAS Register Message”. 5. The SmartDrillSystem continues with UC3: DrillingOrder.
Alternative Procedure	2a. If initialization fails → display “SystemComponentsFailMessage”. 2b. Retry initialization. 3a. If registration fails → display “AASRegistryFailureMessage”. 3b. Restart UC1 at step 2.
Postconditions	SmartDrillSystem is in UC3: DrillingOrder OR retrying UC1.
Related Use Cases	UC2: AAS Register

Table 8.2: Use Case UC3: Drilling Order

Attribute	Description
Identifier	UC3
Name	Drilling Order

Continued on next page

Table 8.2 – UC3: Drilling Order (continued)

Attribute	Description
Description	The Operator submits a DrillingOrder to the SmartDrillSystem, which coordinates execution through agent negotiation.
Actors	Operator
Preconditions	The SmartDrillSystem is initialized.
Trigger	The Operator initiates a DrillingOrder.
Standard Procedure	1. The SmartDrillingSystem displays "ReadyForOrder" message. 2. The System prompts the Operator to input DrillingParameters. 3. The Operator inputs DrillingParameters (depth, diameter, material). 4. The Operator submits the DrillingOrder. 5. The System delegates the DrillingOrder to the ProductionAAS. 6. The ProductionAAS retrieves registered ResourceAAS from Registry. 7. The ProductionAAS negotiates with all ResourceAAS. 8. The ProductionAAS sends ProductionPlan to the winning ResourceAAS. 9. The ResourceAAS starts executing the ProductionPlan. 10. The ResourceAAS triggers UC4: Drill. 11. The ResourceAAS publishes DrillingStatusEvent. 12. Upon UC4 completion, ResourceAAS publishes "DoneMessage". 13. The System continues with UC3 at step 1.

Continued on next page

Table 8.2 – UC3: Drilling Order (continued)

Attribute	Description
Alternative Procedure	4a. Invalid DrillingParameters → display "InvalidInputTypeMessage" → return to step 3. 11a. UC4 fails → publish "FailMessage" → display "FailToDrillMessage" → continue at step 3.
Postconditions	SmartDrillSystem is in UC3: DrillingOrder OR UC5: Shutdown.
Related Use Cases	UC2: AAS Register, UC4: Drill

Table 8.3: Use Case UC5: System Shutdown

Attribute	Description
Identifier	UC5
Name	System Shutdown
Description	The SmartDrillSystem performs a graceful shutdown of all components.
Actors	Operator
Preconditions	SmartDrillSystem is in UC1: Startup OR UC3: DrillingOrder.
Trigger	The Operator initiates system shutdown.
Standard Procedure	1. The Operator initiates shutdown of the SmartDrillSystem. 2. The System stops all active ProductionAAS and pending DrillingOrders. 3. The System deregisters all ResourceAAS from the Registry. 4. The System displays "ShutdownMessage".

Continued on next page

Table 8.3 – UC5: System Shutdown (continued)

Attribute	Description
Postconditions	SmartDrillingSystem is shut down.
Related Use Cases	n/A

8.1.2 System Specifications

Table 8.4: System Specification Requirements

ID	Title	Description	Severit	UC	Status
REQ_1	System Components	The System shall contain: 1. One or more ResourceAAS 2. One or more ProductionAAS 3. EventBroker 4. AAS Registry	High	UC1	✓
REQ_2	AAS Communication	The System shall have standardized communication between AASs.	High	UC3	✓
REQ_3	AAS Events	The System shall allow AASs to communicate over Events via an EventBroker.	High	UC3, UC4	✓
REQ_4	AAS Registry	The System shall register all AASs.	High	UC2	✓
REQ_5	Resource-Asset Communication	The System shall create standardized communication between ResourceAAS and PhysicalAsset.	High	UC4	✓
REQ_6	System Autonomy	The System's AAS components shall react autonomously to each other and to external triggers.	High	UC3, UC4	✓
REQ_7	System Intelligence	The System shall contain decision-making capabilities.	High	UC3	✓
REQ_8	System Collaboration	The System's AAS components shall perform interdependent processes.	High	UC3	✓

8.1.3 ResourceAAS Specifications

Table 8.5: ResourceAAS Requirements

ID	Title	Description	Severit	UC	Status
REQ_10	ResourceAAS Components	The ResourceAAS shall contain: 1. Agent Interface (FIPA ACL) 2. Agent Runtime 3. AAS Submodels 4. AAS Asset Interface	High	UC1	✓
REQ_1	ResourceAAS Event	The ResourceAAS shall publish and subscribe to events on system level.	High	UC3, UC4	✓
REQ_12	ResourceAAS Submodels	The ResourceAAS shall contain: 1. Operational State Submodel 2. Knowledge Submodel 3. Capability Submodel 4. Interaction and Scheduling Submodel	Mid	UC4	✓
REQ_1	ResourceAAS Runtime	The ResourceAAS shall have a BDI Agent Runtime.	High	UC3, UC4	✓
REQ_14	Asset Interface	The ResourceAAS shall communicate with the Physical Asset via suited protocol.	High	UC4	✓
REQ_1	Agent Interface	The ResourceAAS shall communicate via FIPA ACL protocol.	High	UC3	*
REQ_16	Operational State Submodel	Contains: current state, performance KPIs, historical data.	Mid	UC4	✓

Continued on next page

Table 8.5 – ResourceAAS Requirements (continued)

ID	Title	Description	Severit	UC	Status
REQ_1	Knowledge Submodel	Contains: resource constraints, environmental conditions.	Low	UC4	✓
REQ_18	Capability Submodel	Contains: skills, service descriptions.	Mid	UC3	✓
REQ_1	Interaction Submodel	Contains: communication endpoints, collaboration protocols.	Mid	UC3	✓
REQ_20	Negotiation	ResourceAAS shall negotiate using time availability as weight.	High	UC3	✓

* *XMPP PubSub used instead of pure FIPA ACL; message semantics follow CNP pattern.*

8.1.4 ProductionAAS Specifications

Table 8.6: ProductionAAS Requirements

ID	Title	Description	Severit	UC	Status
REQ_21	ProductionAAS Components	The ProductionAAS shall contain: 1. Agent Interface 2. Agent Runtime 3. AAS Submodels	High	UC1	✓
REQ_2	ProductionAAS Events	Shall publish and subscribe to events on system level.	High	UC3	✓

Continued on next page

Table 8.6 – ProductionAAS Requirements (continued)

ID	Title	Description	Severit	UC	Status
REQ_23	ProductionAAS Submodels	Shall contain: 1. Identification Submodel 2. Process Plan Submodel 3. Services Catalog Submodel 4. Interface and Endpoints Submodel 5. Execution State and Tracking Submodel	High	UC3	†
REQ_2	Agent Interface	Shall communicate via FIPA ACL protocol.	High	UC3	*
REQ_25	Runtime	Shall have an Agent Runtime.	High	UC3	✓
REQ_2	Identification Submodel	Contains: IDs, version, owner, product context.	Mid	UC3	–
REQ_27	Process Plan Submodel	Contains: nodes (tasks), edges, entry/exit, capabilities.	Mid	UC3	✓
REQ_2	Services Catalog	Contains: machine skills with callable operations.	Mid	UC3	–
REQ_29	Interface Submodel	Contains: REST/OPC UA/MQTT endpoints, schemas.	Mid	UC3	✓
REQ_3	Execution State Submodel	Contains: current node, timestamps, step status.	Mid	UC3, UC4	✓
REQ_31	Negotiation	Shall negotiate using time availability as weight.	High	UC3	✓

† *3 of 5 submodels implemented (Process Plan, Interface, Execution State).*

8.1.5 MAS/BDI Specifications

Table 8.7: MAS/BDI Requirements

ID	Title	Description	Severit	UC	Status
REQ_32	Agent Beliefs	The AAS shall contain a defined Agent belief set for each ResourceAAS.	High	UC4	✓
REQ_3	Agent Desires	The AAS shall contain defined Agent objectives for each ResourceAAS.	High	UC3	✓
REQ_34	Agent Intentions	The AAS shall contain defined Agent plans for each ResourceAAS.	High	UC4	✓

8.1.6 Non-Functional Requirements

Table 8.8: Non-Functional Requirements

ID	Category	Description	Severit	Status
NFR_1	Interoperability	All AAS components shall comply with IDTA AAS specification (Part 1 and Part 2).	High	✓
NFR_2	Scalability	The system shall support adding new ResourceAAS without modifying existing components.	Mid	✓
NFR_3	Reliability	The system shall handle component failures gracefully with appropriate error messages.	High	✓
NFR_4	Maintainabilit	The system architecture shall follow modular design principles.	Mid	✓

NFR_5	Performance	Agent negotiation shall complete within < 5 seconds for the proof-of-concept.	Mid	✓

Legend: ✓ = Fulfilled, – = Not implemented (out of scope), * = Partially fulfilled with alternative approach

8.2 Implementation Details

This appendix provides detailed configuration files, code listings, and installation instructions for reproducing the Type 3 AAS prototype described in Chapter 5.

8.2.1 Operation Invocation Delegation

Docker Network Configuration

To enable the containerized BaSyx server to communicate with host-based invoker services, the Docker Compose configuration requires network bridging:

Listing 8.1: Docker Compose Network Configuration

```
services:
  aas-env:
    extra_hosts:
      - "host.docker.internal:host-gateway"
  aas-web-ui:
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

This configuration maps `host.docker.internal` to the host machine's gateway address, allowing containers to reach services running on the host.

Qualifier Configuration

Operations in submodels are configured with an `invocationDelegation` qualifier that specifies the target endpoint:

Listing 8.2: Operation Invocation Delegation Qualifier

```
qualifier=[model.Qualifier(
    kind=model.QualifierKind.CONCEPT_QUALIFIER,
    type_="invocationDelegation",
    value_type=datatypes.String,
    value="http://host.docker.internal:8090/drill_invocation"
)]
```

The hostname `host.docker.internal` must remain unchanged, while the port (e.g., 8090) can be configured per invoker service.

Invoker Implementation

External invoker services are implemented as FastAPI applications that receive operation requests from the BaSyx server and return AAS-compliant responses:

Listing 8.3: Operation Invoker Endpoint Structure

```
@app.post("/drill_invocation")
async def invoke_operation(request: Request):
    body = await request.json()
    properties = _opvars_to_properties(body)

    # Execute operation logic
    result = execute_drill(properties)

    # Return AAS-compliant response
    return [{
        "modelType": "OperationVariable",
        "value": {
            "modelType": "Property",
            "idShort": "Drill_Result",
            "value": str(result),
            "valueType": "xs:string"
        }
    }]
```

The invoker must parse the incoming `OperationVariable` array, execute the requested operation, and return results in the same format.

8.2.2 XMPP Server Configuration

Prosody Installation

Listing 8.4: Prosody Installation

```
sudo apt update
sudo apt install prosody
```

Server Configuration

The Prosody configuration file (`/etc/prosody/prosody.cfg.lua`) requires the following modifications:

Localhost Binding:

Listing 8.5: Prosody Host Configuration

```
Host "*"
    c2s_interface = "127.0.0.1"
    s2s_interface = "127.0.0.1"
```

PubSub Component:

Listing 8.6: PubSub Component Configuration

```
Component "pubsub.localhost" "pubsub"
```

Admin Configuration:

Listing 8.7: Admin Configuration

```
admins = {
    "resource_agent_1@localhost",
    "resource_agent_2@localhost",
    "negotiation_agent@localhost",
    "execution_agent@localhost"
}
```

SSL Certificate Generation

Listing 8.8: Certificate Generation

```
cd /etc/prosody/certs
openssl genrsa -out localhost.key 2048
openssl req -x509 -nodes -days 365 -new \
    -key localhost.key -out localhost.crt -subj "/CN=
       ↪ localhost"
sudo chown root:prosody /etc/prosody/certs/localhost.key
sudo chmod 640 /etc/prosody/certs/localhost.key
```

User Management

Agents must be registered as users on the XMPP server:

Listing 8.9: XMPP User Creation

```
prosodyctl shell
> user:create("resource_agent_1@localhost", "password123")
> user:create("negotiation_agent@localhost", "password123")
> user:list("localhost")
```

8.2.3 PubSub Dispatch Pattern

To avoid subscriber crosstalk when subscribing to multiple PubSub topics, implement a single callback that dispatches based on the topic node:

Listing 8.10: PubSub Topic Dispatch Pattern

```
async def callback(self, message):
    event = message['pubsub_event']
    node = event['items']['node']
    payload = event['items']['substanzas'][0]['payload'].text

    if node == 'production_negotiation':
        await self._handle_cfp(payload)
```

```
        elif node == 'negotiation_message_topic':
            await self._handle_response(payload)
        elif node == 'execution_tracking_topic':
            await self._handle_tracking(payload)
```

8.2.4 Deployment Configuration

Docker Compose Services

Listing 8.11: Docker Compose Configuration (excerpt)

```
version: '3.8'
services:
  mongodb:
    image: mongo:latest
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db

  mosquitto:
    image: eclipse-mosquitto:latest
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto/config:/mosquitto/config

  aas-env:
    image: eclipsebasyx/aas-environment:2.0.0-
      ↪ milestone-03
    ports:
      - "8081:8081"
    environment:
```

```
      - BASYX_MONGODB_HOST=mongodb
      - BASYX_MQTT_HOST=mosquitto
    extra_hosts:
      - "host.docker.internal:host-gateway"
    depends_on:
      - mongodb
      - mosquitto

  aas-registry:
    image: eclipsebasyx/aas-registry-kafka-mongodb
      ↪ :2.0.0
    ports:
      - "8082:8080"
    depends_on:
      - mongodb

  aas-web-ui:
    image: eclipsebasyx/aas-gui:v2-240801
    ports:
      - "3000:3000"
    extra_hosts:
      - "host.docker.internal:host-gateway"

volumes:
  mongodb_data:
```

Startup Sequence

The system requires the following startup order:

1. Start Docker Compose services:

```
docker compose up -d
```

2. Start Prosody XMPP server:

```
sudo systemctl start prosody
```

3. Initialize AAS shells and submodels:

```
python -m aastype3.Core.Submodels_base.Resource_Base.
    ↪ Entry_Point
python -m aastype3.Core.Submodels_base.Production_Base.
    ↪ Entry_Point
```

4. Start simulators:

```
python -m aastype3.code.Simulators.DrillMachine1 &
python -m aastype3.code.Simulators.MoveXYMachine1 &
```

5. Start Resource Agents:

```
python -m aastype3.Core.Resource_Agent.Agent_Core.
    ↪ Resource_Agent &
```

6. Start Production Agent:

```
python -m aastype3.Core.Production_Agent.Agent_Core.
    ↪ agent_entrypoint
```

Bibliography

[1] M. Lindner, L. Bank, J. Schilp, and M. Weigold, "Digital twins in manufacturing: A rami 4.0 compliant concept," *Sci*, vol. 5, no. 4, p. 40, 2023.

[2] Industrial Digital Twin Association, "Specification of the asset administration shell: Part 1 - metamodel," IDTA, Tech. Rep., 2023, version 3.0.

[3] Platform Industrie 4.0 and Industrial Digital Twin Association, "Details of the asset administration shell - part 1: The exchange of information between partners in the value chain of industrie 4.0," Platform Industrie 4.0 and IDTA, Berlin, Germany, Tech. Rep., 2023, specification Document. [Online]. Available: https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part1_V3.pdf

[4] S. Bader, B. Berres, B. Boss, A. G. Gatterburg, M. Hoffmeister, Y. Kogan, A. Köpke, M. Lieske, T. Miny, J. Neidig, A. Orzelski, S. Pollmeier, M. Sauer, D. Schel, T. Schröder, M. Thron, T. Usländer, J. Vialkowitsch, F. Vollmar, J. Wende, and C. Ziesche, "Details of the asset administration shell – part 2: Interoperability at runtime – exchanging information via application programming interfaces, version 1.0," Plattform Industrie 4.0, Berlin, Germany, Tech. Rep. Version 1.0, Nov. 2021, accessed: 2025-12-21. [Online]. Available: https://www.plattform-i40.de/IP/Redaktion/EN/Downloads/Publikation/Details_of_the_Asset_Administration_Shell_Part2_V1.pdf?__blob=publicationFile&v=1

[5] L. Sakurada, F. D. la Prieta, and P. Leitão, "The role of multi-agent systems in realizing asset administration shell type 3," *Future Internet*,

vol. 17, no. 7, p. 270, 2025, accessed: 2025-12-21. [Online]. Available: https://www.mdpi.com/1999-5903/17/7/270

[6] P. Leitão, T. I. Strasser, S. Karnouskos, L. Ribeiro, J. Barbosa, and V. Huang, "Recommendation of best practices for industrial agent systems based on the ieee 2660.1 standard," in *2021 22nd IEEE international conference on industrial technology (ICIT)*, vol. 1. IEEE, 2021, pp. 1157–1162.

[7] A. Pokahr and L. Braubach, *Jadex 0.96x User Guide: Chapter 2. Concepts of the Jadex BDI Reasoning Engine*, Actoron / Distributed Systems Group, University of Hamburg, Hamburg, Germany, Jun. 2007, accessed: 2025-12-21; part of Jadex 0.96x User Guide. [Online]. Available: https://download.actoron.com/docs/releases/jadex-0.96x/userguide/concepts.html

[8] Plattform Industrie 4.0, "Information model for capabilities, skills & services: Definition of terminology and proposal for a technology-independent information model," ZVEI/VDMA/BITKOM, Tech. Rep., 2022. [Online]. Available: https://www.plattform-i40.de/

[9] H. Lasi, P. Fettke, H.-G. Kemper, T. Feld, and M. Hoffmann, "Industry 4.0," *Business & information systems engineering*, vol. 6, no. 4, pp. 239–242, 2014.

[10] R. Y. Zhong, X. Xu, E. Klotz, and S. T. Newman, "Intelligent manufacturing in the context of industry 4.0: a review," *Engineering*, vol. 3, no. 5, pp. 616–630, 2017.

[11] S. Vaidya, P. Ambad, and S. Bhosle, "Industry 4.0–a glimpse," *Procedia manufacturing*, vol. 20, pp. 233–238, 2018.

[12] F. Fraile, R. Sanchis, R. Poler, and A. Ortiz, "Reference models for digital manufacturing platforms," *Applied Sciences*, vol. 9, no. 20, p. 4433, 2019.

[13] E. VanDerHorn and S. Mahadevan, "Digital twin: Generalization, characterization and implementation," *Decision support systems*, vol. 145, p. 113524, 2021.

[14] M. Attaran, S. Attaran, and B. G. Celik, "The impact of digital twins on the evolution of intelligent manufacturing and industry 4.0," *Advances in Computational Intelligence*, vol. 3, no. 3, p. 11, 2023.

[15] M. A. Inigo, A. Porto, B. Kremer, A. Perez, F. Larrinaga, and J. Cuenca, "Towards an asset administration shell scenario: a use case for interoperability and standardization in industry 4.0," in *NOMS 2020-2020 IEEE/IFIP Network Operations and Management Symposium*. IEEE, 2020, pp. 1–6.

[16] K. Wei, J. Sun, and R. Liu, "A review of asset administration shell," in *2019 IEEE International Conference on Industrial Engineering and Engineering Management (IEEM)*. IEEE, 2019, pp. 1460–1465.

[17] S. Wang, J. Wan, D. Li, and C. Zhang, "Implementing smart factory of industrie 4.0: an outlook," *International journal of distributed sensor networks*, vol. 12, no. 1, p. 3159805, 2016.

[18] O. Antons and J. C. Arlinghaus, "Designing distributed decision-making authorities for smart factories–understanding the role of manufacturing network architecture," *International Journal of Production Research*, vol. 62, no. 1-2, pp. 204–222, 2024.

[19] P. Leitão *et al.*, "The role of multi-agent systems in realizing asset administration shell type 3," *Future Internet*, vol. 17, no. 7, p. 270, 2025.

[20] C. E. Pereira *et al.*, "Asset administration shell tool comparison: A case study with real digital twins used in petrochemical industry," *Sensors*, 2025, pMC11991629.

[21] A. S. Rao, M. P. Georgeff *et al.*, "Bdi agents: From theory to practice." in *Icmas*, vol. 95, 1995, pp. 312–319.

[22] J. Arm, T. Benesl, P. Marcon, Z. Bradac, T. Schröder, A. Belyaev, T. Werner, V. Braun, P. Kamensky, F. Zezulka, C. Diedrich, and P. Dohnal, "Automated design and integration of asset administration shells in components of industry 4.0," *Sensors*, vol. 21, no. 6, p. 2004, 2021.

[23] A. López, O. Casquero, E. Estévez, A. Armentia, D. Orive, and M. Marcos, "An industrial agent-based customizable platform for i4.0 manufacturing systems," *Computers in Industry*, vol. 146, p. 103859, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S016636152300009X

[24] B. Vogel-Heuser, F. Ocker, and T. Scheuer, "An approach for leveraging digital twins in agent-based production systems," *at-Automatisierungstechnik*, vol. 69, no. 12, pp. 1026–1039, 2021.

[25] L. Monostori, J. Váncza, and S. R. Kumara, "Agent-based systems for manufacturing," *CIRP annals*, vol. 55, no. 2, pp. 697–720, 2006.

[26] M. Wooldridge, *An introduction to multiagent systems*. John wiley & sons, 2009.

[27] M. Wooldridge and N. R. Jennings, "Intelligent agents: Theory and practice," *The Knowledge Engineering Review*, vol. 10, no. 2, pp. 115–152, 1995.

[28] P. Leitao, S. Karnouskos, L. Ribeiro, J. Lee, T. Strasser, and A. W. Colombo, "Smart agents in industrial cyber–physical systems," *Proceedings of the IEEE*, vol. 104, no. 5, pp. 1086–1101, 2016.

[29] R. A. Brooks, "A robust layered control system for a mobile robot," *IEEE Journal on Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.

[30] Foundation for Intelligent Physical Agents, "Fipa acl message structure specification," FIPA, Tech. Rep. SC00061G, 2002. [Online]. Available: http://www.fipa.org/specs/fipa00061/SC00061G.html

[31] R. G. Smith, "The contract net protocol: High-level communication and control in a distributed problem solver," *IEEE Transactions on Computers*, vol. C-29, no. 12, pp. 1104–1113, 1980.

[32] J. F. Hübner, J. S. Sichman, and O. Boissier, "Developing organised multiagent systems using the moise+ model: Programming issues at the system and agent levels," *International Journal of Agent-Oriented Software Engineering*, vol. 1, no. 3/4, pp. 370–395, 2007.

[33] M. E. Bratman, *Intention, Plans, and Practical Reason.* Cambridge, MA: Harvard University Press, 1987.

[34] R. H. Bordini, J. F. Hübner, and M. Wooldridge, *Programming Multi-Agent Systems in AgentSpeak Using Jason*, ser. Wiley Series in Agent Technology. Wiley, 2007.

[35] E. Yan, S. Burattini, J. F. Hübner, and A. Ricci, "A multi-level explainability framework for engineering and understanding bdi agents," *Autonomous Agents and Multi-Agent Systems*, vol. 39, no. 1, p. 9, 2025.

[36] Eclipse Foundation, "Eclipse basyx: Open source platform for industry 4.0," 2023, accessed: 2025-12-21. [Online]. Available: https://www.eclipse.org/basyx/

[37] Fraunhofer IOSB, "Fa³st service: Fraunhofer advanced asset administration shell tools," 2023, accessed: 2025-12-21. [Online]. Available: https://github.com/FraunhoferIOSB/FAAAST-Service

[38] Industrial Digital Twin Association, "Aasx package explorer," 2023, accessed: 2025-12-21. [Online]. Available: https://github.com/admin-shell-io/aasx-package-explorer

[39] Telecom Italia, "Jade: Java agent development framework," 2023, accessed: 2025-12-21. [Online]. Available: https://jade.tilab.com/

[40] M. E. Gregori *et al.*, "Spade: Smart python agent development environment," 2023, accessed: 2025-12-21. [Online]. Available: https://spade-mas.readthedocs.io/

[41] Actoron GmbH, "Jadex active components," 2023, accessed: 2025-12-21. [Online]. Available: https://www.2activecomponents.2org/

[42] L. Sakurada, P. Leitao, and F. De La Prieta, "Engineering a multi-agent systems approach for realizing collaborative asset administration shells," in *2022 IEEE International Conference on Industrial Technology (ICIT).* IEEE, 2022, pp. 1–6.

[43] V. Siatras, E. Bakopoulos, P. Mavrothalassitis, N. Nikolakis, and K. Alexopoulos, "Production scheduling based on a multi-agent system and digital twin: a bicycle industry case," *Information*, vol. 15, no. 6, p. 337, 2024.

[44] A. Sidorenko, W. Motsch, M. van Bekkum, N. Nikolakis, K. Alexopoulos, and A. Wagner, "The mas4ai framework for human-centered agile and smart manufacturing," *Frontiers in Artificial Intelligence*, vol. 6, p. 1241522, 2023.

[45] S. Jungbluth, J. Hermann, W. Motsch, M. Pourjafarian, A. Sidorenko, M. Volkmann, K. Zoltner, C. Plociennik, and M. Ruskowski, "Dynamic replanning using multi-agent systems and asset administration shells," in *2022 IEEE 27th International Conference on Emerging Technologies and Factory Automation (ETFA)*. IEEE, 2022, pp. 1–8.

[46] Industrial Digital Twin Association, "Submodel template for control component instance," Industrial Digital Twin Association (IDTA), Frankfurt am Main, Germany, Tech. Rep. IDTA 02016-1-0, April 2023, version 1.0. [Online]. Available: https://industrialdigitaltwin.org/wp-content/uploads/2023/04/IDTA-02016-1-0-_Submodel_ControlComponentInstance.pdf

[47] I. D. T. Association *et al.*, "Idta 02017-1-0 asset interfaces description," *Industrial Digital Twin Association*, 2024.

[48] ——, "Idta 02008-1-1 time series data," *Industrial Digital Twin Association: Frankfurt am Main, Germany*, 2023.

[49] M. Fowler, *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[50] "SimPy: Discrete event simulation for Python," https://simpy.readthedocs.io/, 2023.